

DYNAMIC EARLY EXIT IN REASONING MODELS

Chenxu Yang^{✉*}, Qingyi Si^{✉*}, Yongjie Duan[✉], Zheliang Zhu[✉], Chenyu Zhu[✉],
 Qiaowei Li[✉], Minghui Chen[✉], Zheng Lin^{✉†}, Weiping Wang[✉],
[✉]Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[✉]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[✉]Huawei Technologies Co., Ltd.
 {yangchenxu,linzheng}@iie.ac.cn, siqingyi@huawei.com

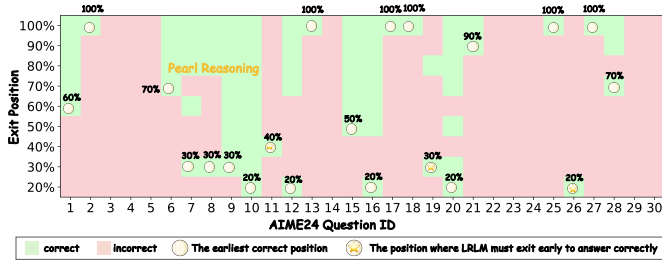


Figure 1: Correctness statistics for early exits at various reasoning steps.

ABSTRACT

Recent advances in large reasoning language models (LRMs) rely on test-time scaling, which extends long chain-of-thought (CoT) generation to solve complex tasks. However, overthinking in long CoT not only slows down the efficiency of problem solving, but also risks accuracy loss due to the extremely detailed or redundant reasoning steps. We propose a simple yet effective method that allows LLMs to self-truncate CoT sequences by early exit during generation. Instead of relying on fixed heuristics, the proposed method monitors model behavior at potential reasoning transition points and dynamically terminates the next reasoning chain’s generation when the model exhibits high confidence in a trial answer. Our method requires no additional training and can be seamlessly integrated into existing o1-like reasoning LLMs. Experiments on 10 reasoning benchmarks (e.g., GSM8K, MATH-500, AMC, GPQA, AIME and LiveCodeBench) show that the proposed method is consistently effective on 11 cutting-edge reasoning LLMs of varying series and sizes, reducing the length of CoT sequences by an average of 19.1% to 80.1% while improving accuracy by 0.3% to 5.0%.¹

1 INTRODUCTION

The emergence of large reasoning models (Xu et al., 2025a), such as DeepSeek-R1 (DeepSeek-AI et al., 2025) and GPT-O1 (OpenAI, 2025), has marked a significant breakthrough in natural language processing, particularly in solving complex and intricate tasks (WANG et al., 2025). These models leverage the test-time scaling (Snell et al., 2024) law by generating a longer CoT (Wei et al., 2023) with rich and diverse reasoning paths, unleashing the potential of their reasoning ability.

* Equal Contribution. Work done when Chenxu Yang was an intern at Huawei.

† Corresponding Author.

¹ The code is available at <https://github.com/iie-ycx/DEER>

推理模型中的动态早期退出

Chenxu Yang^{✉*}, Qingyi Si^{✉*}, Yongjie Duan[✉], Zheliang Zhu[✉], Chenyu Zhu[✉],
 Qiaowei Li[✉], Minghui Chen[✉], Zheng Lin^{✉†}, Weiping Wang[✉],
[✉]中国科学院信息工程研究所, 北京, 中国
[✉]中国科学院大学网络空间安全学院, 北京, 中国
[✉]华为技术有限公司
 {yangchenxu,linzheng}@iie.ac.cn, siqingyi@huawei.com

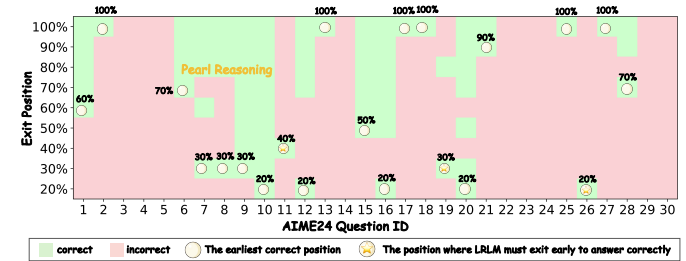


图 1: 不同推理步骤早期退出的正确率统计。

Abstract

大型推理语言模型 (LRMs) 的最新进展依赖于测试时扩展, 即生成长链思维 (CoT) 来解决复杂任务。然而, 长链思维中的过度思考不仅会降低问题解决的效率, 而且由于极其详细或冗余的推理步骤, 还存在准确性损失的风险。我们提出了一种简单而有效的方法, 允许 LLMs 在生成过程中通过提前退出来自我截断 CoT 序列。该方法不依赖固定的启发式规则, 而是在潜在的推理转换点监控模型行为, 当模型对试探性答案表现出高置信度时, 动态终止下一个推理链的生成。我们的方法无需额外训练, 可以无缝集成到现有的类 o1 推理 LLMs 中。在 10 个推理基准测试 (例如 GSM8K、MATH-500、AMC、GPQA、AIME 和 LiveCodeBench) 上的实验表明, 所提出的方法在 11 个不同系列和规模的尖端推理 LLMs 上持续有效, 将 CoT 序列的长度平均减少了 19.1% 至 80.1%, 同时将准确性提高了 0.3% 至 5.0%。¹

1 Introduction

大型推理模型 (Large Reasoning Models, LRMs) 的出现 (Xu et al., 2025a), 例如 DeepSeek-R1 (DeepSeek-AI et al., 2025) 和 GPT-O1 (OpenAI, 2025), 标志着自然语言处理领域, 特别是在解决复杂任务方面 (WANG et al., 2025), 取得了重大突破。这些模型通过生成

* 同等贡献。Chenxu Yang 在华为实习期间完成的工作。

† 通讯作者。

¹ 代码可在 <https://github.com/iie-ycx/DEER> 获取。

However, the generation of overlong CoT significantly increases computational overload and reasoning latency, which hinders their deployment in computationally sensitive applications. Moreover, recent research (Chen et al., 2025b; Team et al., 2025a) reveals an intrinsic overthinking problem in LRMs: These models persistently generate verbose reasoning sequences (Wu et al., 2025; Cuadron et al., 2025), introducing irrelevant information and unnecessary thought steps. Such redundant processing not only wastes computational resources but also leads to accuracy degradation by de-railing from correct reasoning paths to erroneous ones (see Questions 11, 19 and 26 in Fig. 1. This redundancy can be attributed to the design of the Supervised Fine-Tuning (Achiam et al., 2023; Wei et al., 2021; Ouyang et al., 2022) or Reinforcement Learning (Bai et al., 2022; Ouyang et al., 2022; Schulman et al., 2017; Ramesh et al., 2024) stage, where the ability to dynamically adjust its reasoning length during generation is overlooked, leaving a gap in the inference efficiency of LRMs.

Intuitively, as the number of reasoning paths increases, more information is referenced when generating conclusions. If we can identify the critical point where the reasoning information becomes just sufficient (termed **Pearl Reasoning**) and force the model to stop further thinking and directly output conclusions at this point, we can achieve both accuracy and efficiency. This paper aims to *find such pearls in long CoT sequences*. To validate our motivation, we forced the model to switch from thinking to directly generating answers, at different transition points in the thought process. If the answers obtained are correct, the existence of such pearl reasoning is verified. As shown in Fig. 1, about 75% samples contain such pearls (early exit yields correct answers), even 36.7% samples required only less than half of the original reasoning paths to reach correct conclusions. Therefore, how to find the pearl reasoning is a valuable topic to achieve efficient reasoning.

To this end, we propose a novel, training-free approach **DEER** that allows large reasoning language models to achieve **Dynamic Early Exit in Reasoning**. It regards the key moments when the model switches thought chains in reasoning as chances of early exit, and prompting LRMs to stop thinking and generate trial answers at these moments. The confidence of each trial answer is the decision-making reference of early exit in reasoning. Specifically, the proposed method contains three actions: 1) **Reasoning Transition Monitoring**. During the generation of long CoTs, DEER monitors the positions of reasoning transitions through either linguistic marker-based (such as "Wait") or entropy-based methods. When the reasoning transition points are found, the action of 2) **Trial Answer Inducing** is triggered: we replace it with "*final answer*" tokens to induce the model to immediately generate a trial answer, which will be used for 3) **Confidence Evaluating**. If the confidence is sufficiently high, set the model to stop further thinking and generate a conclusion based on the generated thoughts. Otherwise, the action of Trial Answer Inducing is revoked, and the model continues reasoning along the original path. Moreover, Considering the potential sensitivity of models to answer inducing prompts, we propose **DEER-Pro** (a Parallel and Robust variant of DEER), which performs multiple parallel answer inductions at potential early-exit points and calibrates confidence based on the aggregated results, thereby further ensuring DEER's robustness.

Our method is simple yet effective, and can be seamlessly extended to eleven reasoning models of varying architectures and sizes, achieving excellent results in the ten reasoning benchmarks, including mathematical tasks (e.g., AIME 2024, AMC 2023 and MATH-500), scientific tasks (e.g., GPQA Diamond) and programming tasks (e.g., BigCodeBench). Specifically, our method, when integrated into cutting-edge reasoning models, can reduce the length of CoT sequences by an average of 19.1% to 80.1% while improving accuracy by 0.3% to 5.0% across different reasoning benchmarks. Our DEER offers a plug-and-play solution for improving both the efficiency and accuracy of LRMs.

2 MOTIVATIONS AND OBSERVATIONS

In this section, we analyze the overthinking phenomenon in LRMs and investigate the impact of static early exits on model performance. We define "pearl reasoning" as the critical juncture where reasoning information becomes precisely sufficient for accurate problem-solving. Our analysis in Figure 1 reveals that approximately 75% of samples contain such pearls (where early exit yields correct answers). Furthermore, we identified a subset of samples for which correct answers are exclusively obtainable through early exits (exemplified by Questions 11, 19, and 26 in Figure 1). Quantitative analysis presented in Figure 2(a) further demonstrates that 60.8% and 35.1% of correctly answered samples in MATH-500 and GPQA, respectively, maintain their accuracy when employing early exits after completing merely 20% of the reasoning steps. These empirical findings

更长、包含丰富多样推理路径的思维链 (CoT) (Wei et al., 2023), 利用测试时缩放 (Snell et al., 2024) 定律, 释放了其推理能力的潜力。

然而, 生成长度过长的思维链会显著增加计算负荷和推理延迟, 这阻碍了它们在计算敏感型应用中的部署。此外, 最近的研究 (Chen et al., 2025b; Team et al., 2025a) 揭示了大型推理模型存在一个固有的过度思考问题: 这些模型会持续生成冗长的推理序列 (Wu et al., 2025; Cuadron et al., 2025), 引入无关信息和不必要的思考步骤。这种冗余处理不仅浪费计算资源, 还会导致准确率下降, 因为它使模型偏离正确的推理路径, 转向错误的路径 (参见图1中的问题 11、19 和 26)。这种冗余可归因于监督微调 (Achiam et al., 2023; Wei et al., 2021; Ouyang et al., 2022) 或强化学习 (Bai et al., 2022; Ouyang et al., 2022; Schulman et al., 2017; Ramesh et al., 2024) 阶段的设计, 其中忽略了在生成过程中动态调整其推理长度的能力, 从而在大型推理模型的推理效率上留下了空白。

直观上, 随着推理路径数量的增加, 生成结论时参考的信息也越多。如果我们能识别出推理信息恰好足够的关键点 (称为 Pearl Reasoning), 并迫使模型在此点停止进一步思考, 直接输出结论, 那么我们就实现同时实现准确性和效率。本文旨在 *find such pearls in long CoT sequences*。为了验证我们的动机, 我们迫使模型在思维过程的不同转换点从思考切换到直接生成答案。如果获得的答案是正确的, 则验证了这种“珍珠推理”的存在。如图1所示, 大约 75% 的样本包含这种珍珠 (提前退出能产生正确答案), 甚至有 36.7% 的样本仅需要不到原始推理路径的一半就能得出正确结论。因此, 如何找到珍珠推理是实现高效推理的一个有价值的研究课题。

当发现推理转换点时, 会触发动作 2) Trial Answer Inducing: 我们将其替换为 "final answer" 标记, 以诱导模型立即生成一个试验性答案, 该答案将用于 3) Confidence Evaluating。如果置信度足够高, 则设置模型停止进一步思考, 并根据已生成的思路得出结论。否则, 撤销“诱导试验性答案”的动作, 模型继续沿着原始路径进行推理。此外, 考虑到模型对答案诱导提示的潜在敏感性, 我们提出了 DEER-Pro (DEER 的 P 行且 Ro 健变体), 它在潜在的早期退出点执行多个并行的答案诱导, 并根据聚合结果校准置信度, 从而进一步确保 DEER 的鲁棒性。

我们的方法简单而有效, 并且可以无缝扩展到十一种不同架构和规模的推理模型, 在十个推理基准测试中取得了优异的结果, 包括数学任务 (例如 AIME 2024、AMC 2023 和 MATH-500)、科学任务 (例如 GPQA Diamond) 和编程任务 (例如 BigCodeBench)。具体来说, 我们的方法在集成到前沿推理模型中时, 可以将思维链序列的长度平均减少 19.1% 到 80.1%, 同时在不同推理基准测试中将准确率提高 0.3% 到 5.0%。我们的 DEER 为提升 LRM 的效率和准确性提供了一个即插即用的解决方案。

2 Motivations and Observations

在本节中, 我们分析 LRM 中的过度思考现象, 并研究静态早期退出对模型性能的影响。我们将“珍珠推理”定义为推理信息恰好足以准确解决问题的关键节点。我们在图1中的分析显示, 大约 75% 的样本包含此类珍珠 (即早期退出能产生正确答案)。此外, 我们识别出了一个样本子集, 对于这些样本, 正确答案只能通过早期退出获得 (如图1中的问题 11、19 和 26 所示)。图2(a) 中呈现的定量分析进一步表明, 在 MATH-500 和 GPQA 中, 分别有 60.8% 和 35.1% 的正确回答样本, 在仅完成 20% 的推理步骤后采用早期退出时, 仍能保持其准确性。这些实证结果证实了我们的假设: LRM 有潜力通过策略性的早期终止, 在计算效率和预测准确性上同时实现改进。

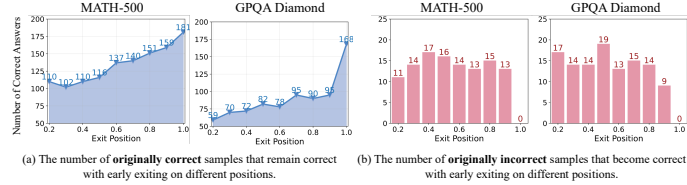


Figure 2: Quantitative pilot experiment results. Please refer to Appendix A for setups.

substantiate our hypothesis that LRMs possess the potential to achieve simultaneous improvements in both computational efficiency and prediction accuracy through strategic early termination.

Fig. 2(b) illustrates that exiting at different positions corrects varying proportions of wrong answers. For the MATH dataset, the highest correction rate is achieved when exiting at 40% of the reasoning steps, whereas for the GPQA dataset, the optimal correction occurs when exiting at 50%. The optimal early exit point varies for each problem and is closely related to the inherent difficulty of the problem itself. Therefore, it is intuitive that relying on a static early exit strategy based on fixed heuristics is suboptimal, underscoring the necessity of designing a dynamic early exit mechanism.

3 METHOD

3.1 THE GENERATION PATTERN OF LARGE REASONING MODELS

In contrast to traditional large language models (*System 1*), large reasoning models (*System 2*) (Li et al., 2025b) exhibit distinct generation patterns during the inference stage. (1) LRMs use delimiters to divide the output into two processes: slow thinking and conclusion. LRMs conduct systematic and thorough reasoning in the slow thinking, ultimately summarizing the thought process and providing the final answer in the conclusion. (2) During the slow thinking process, LRMs engage in complex thinking actions (thoughts), such as problem comprehension, approach exploration, and result verification (Luo et al., 2025b). Within each reasoning action (thought), the model performs specific procedural action execution, while transitions between different reasoning actions are typically marked by action transition points (ATP), such as "Wait", "Alternatively".

$$\text{System 1: [Prompt] + [Completion],} \quad (1)$$

$$\text{System 2: [Prompt] + \langle \text{think} \rangle + [\text{Slow Thinking}] + \langle / \text{think} \rangle + [\text{Conclusion}],} \quad (2)$$

$$[\text{Slow Thinking}] : [\text{Action Execution}] + (\text{ATP}) + [\text{Action Execution}] + (\text{ATP}) + \dots, \quad (3)$$

where $\langle \text{think} \rangle$ and $\langle / \text{think} \rangle$ are begin-of-thinking and end-of-thinking delimiters respectively.

3.2 DYNAMIC EARLY EXIT IN REASONING

In this section, we introduce the Dynamic Early Exit in Reasoning (DEER) method to determine optimal positions for early exits (pearl reasoning path), thereby alleviating the overthinking issue.

The core idea behind DEER is that a model’s confidence in its trial answer dynamically indicates whether the thinking information required for LRMs to generate the final answer is sufficient. We observe that when the model’s reasoning process is incomplete or flawed, the trial answer tends to exhibit significantly lower confidence. Conversely, when the reasoning is comprehensive and logically sound, the model generates answers with higher confidence, as illustrated in Fig. 17. This suggests that the model implicitly recognizes when **pearl reasoning** occurs, but lacks an explicit mechanism during inference to leverage this awareness for early termination. DEER aims to bridge this gap by converting implicit awareness into explicit early-exit decisions.

As shown in Fig. 3, DEER involves three designs to determine whether to exit early: reasoning transition monitor, answer inducer, and confidence evaluator.

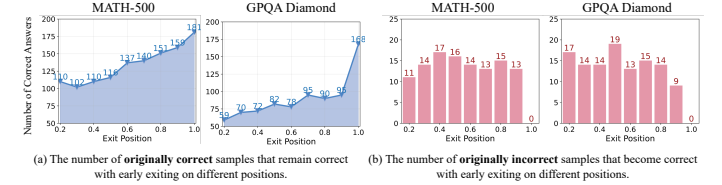


图 2: 定量试点实验结果。具体设置请参阅附录 A。

图2(b) 表明，在不同位置退出能纠正不同比例的错误答案。对于 MATH 数据集，在 40% 的推理步骤处退出时，纠正率最高；而对于 GPQA 数据集，最佳纠正率在 50% 处退出时发生。每个问题的最佳早期退出点各不相同，并且与问题本身的内在难度密切相关。因此，直观上，依赖于固定启发式的静态早期退出策略是次优的，这突显了设计动态早期退出机制的必要性。

3 Method

3.1 The Generation Pattern of Large Reasoning Models

与传统的大型语言模型（System 1）相比，大型推理模型（System 2）(Li et al., 2025b) 在推理阶段展现出独特的生成模式。(1) LRMs 使用分隔符将输出划分为两个过程：慢思考和结论。LRMs 在慢思考中进行系统且彻底的推理，最终总结思考过程并在结论中给出最终答案。(2) 在慢思考过程中，LRMs 进行复杂的思考动作（thoughts），例如问题理解、方法探索和结果验证 (Luo et al., 2025b)。在每个推理动作（thought）内部，模型执行特定的程序性动作，而不同推理动作之间的转换通常由动作转换点（ATP）标记，例如“Wait”、“Alternatively”。

$$\text{System 1: [Prompt] + [Completion],} \quad (1)$$

$$\text{System 2: [Prompt] + \langle \text{think} \rangle + [\text{Slow Thinking}] + \langle / \text{think} \rangle + [\text{Conclusion}],} \quad (2)$$

$$[\text{Slow Thinking}] : [\text{Action Execution}] + (\text{ATP}) + [\text{Action Execution}] + (\text{ATP}) + \dots, \quad (3)$$

其中 $\langle \text{think} \rangle$ 和 $\langle / \text{think} \rangle$ 分别是思考开始和思考结束的分隔符。

在本节中，我们介绍动态推理早期退出（DEER）方法，以确定早期退出（珍珠推理路径）的最佳位置，从而缓解过度思考问题。

DEER 背后的核心思想是，模型对其试探性答案的置信度动态地表明 LRMs 生成最终答案所需的思考信息是否充足。我们观察到，当模型的推理过程不完整或有缺陷时，试探性答案的置信度往往显著较低。相反，当推理全面且逻辑合理时，模型会生成置信度更高的答案，如图17所示。这表明模型隐含地认识到何时发生 pearl reasoning，但在推理过程中缺乏明确的机制来利用这种意识进行早期终止。DEER 旨在通过将隐含意识转化为明确的早期退出决策来弥合这一差距。

如图3所示，DEER 涉及三种设计来决定是否提前退出：推理转换监控器、答案诱导器和置信度评估器。Reasoning transition monitor. 在 DEER 框架内，我们提出了两种可选的监控器设计策略：(i) linguistic marker-based，以及 (ii) entropy-based 监控。对于第一种策略，如第 3.1 节所述，LRMs 明确利用 ATPs 来标记不同思维之间的边界。这一特性使得 DEER 能够将 ATPs 识别为潜在的提前退出机会。在第二种策略中，DEER 使用 “\n\n” 作为分

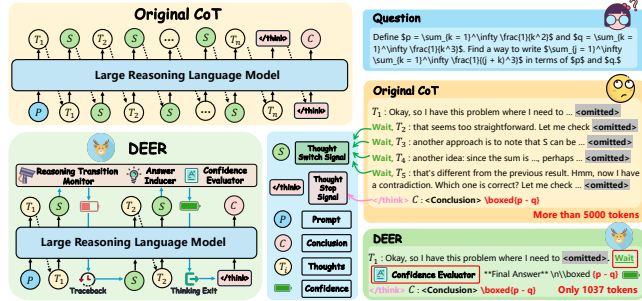


Figure 3: An overview of the Dynamic Early Exit in Reasoning (DEER) method.

Reasoning transition monitor. Within the DEER framework, we propose two alternative monitor design strategies: (i) **linguistic marker-based**, and (ii) **entropy-based** monitoring. For the first strategy, as mentioned in Section 3.1, LRMs explicitly utilize ATPs to mark boundaries between different thoughts. This feature enables DEER to recognize ATPs as potential early-exit opportunities. In the second strategy, DEER employs “ $\backslash n \backslash n$ ” as delimiters to demarcate reasoning steps. Following each reasoning step, DEER computes the entropy of the initial token, denoted as $H(p(\cdot|x_{<t}))$. Low entropy values indicate that the model is engaged in procedural action execution, characterized by stable reasoning patterns. Conversely, high entropy values suggest that the model is deliberating on its subsequent reasoning action, with multiple potential pathways being activated concurrently. These positions exhibiting high entropy are identified as candidate points for early-exit.

Our subsequent experiments in Section 4.3 demonstrate that DEER with external linguistic markers satisfies similar properties as the second internal state-based approach while achieving comparable performance. When applied to English LRMs, existing models consistently exhibit a pattern of generating such linguistic markers. Following the Occam’s Razor principle (Rasmussen & Ghahramani, 2000), we recommend adopting the first strategy. For non-English reasoning scenarios, the alternative second strategy can also accurately capture early-exit points, demonstrating the generality and robustness of DEER.

Answer inducer. When the LRM pauses at a potential early exit point, the trial answer inducer module prompts the model to generate an intermediate answer based on the reasoning content produced so far. We incorporated the answer delimiters ($\backslash boxed \{ \}$) into the prompt to facilitate a more precise identification of the trial answers, as follows: $A = \text{LRM}(P, T, I)$ where P denotes the input prompt, T denotes the generated thoughts, I denotes the answer inducer prompt, and $A = [a_0, a_1, \dots, a_n]$ is the trial answer.

Confidence evaluator. The confidence evaluator computes the confidence of the induced trial answer. It takes the maximum predicted probability of each token as its confidence. For multi-token trial answers, the overall confidence is computed as their mean score across all tokens as follows:

$$p(a_t) = \text{softmax}(\mathcal{M}(P, T, I, a_{<t})), \quad C = \left(\prod_{a_t \in \mathcal{V}} p(a_t) \right)^{1/n} \quad (4)$$

where $\mathcal{M}$ is the language model head at the final layer of the LRM. The calculation of C employs the geometric mean, which better aligns with the multiplicative nature of joint probabilities and exhibits greater sensitivity to low probability values, thereby providing enhanced robustness.

Finally, the comparison between the obtained confidence and the empirical threshold λ determines whether to exit early. If $C > \lambda$, we consider the reasoning information currently generated by the LRM to be sufficient, indicating that the model has reached the **pearl reasoning**. At this point, DEER stops further reasoning actions and proceeds to deliver the conclusion. Otherwise, the model reverts to the previous transition point to generate the next thoughts.

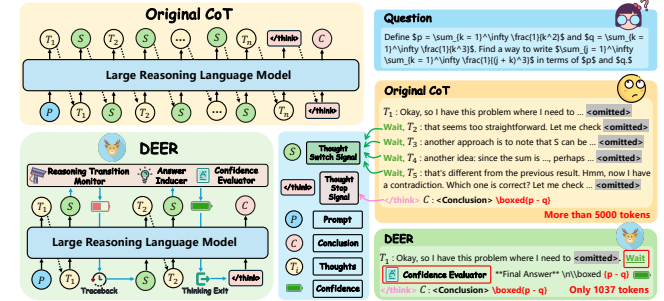


图 3: 动态推理早期退出 (DEER) 方法概述。

隔符来划分推理步骤。在每个推理步骤之后，DEER 计算初始标记的熵，记为 $H(p(\cdot|x_{<t}))$ 。低熵值表明模型正在进行程序性动作执行，其特点是推理模式稳定。相反，高熵值表明模型正在思考其后续的推理动作，多个潜在路径被同时激活。这些表现出高熵的位置被识别为提前退出的候选点。

我们在第 ?? 节中的后续实验表明，使用外部语言标记的 DEER 满足与第二种基于内部状态的方法相似的性质，同时实现了可比的性能。当应用于英语 LRMs 时，现有模型一致表现出生成此类语言标记的模式。遵循奥卡姆剃刀原则 (Rasmussen & Ghahramani, 2000)，我们建议采用第一种策略。对于非英语推理场景，备选的第二种策略也能准确捕捉提前退出点，这证明了 DEER 的通用性和鲁棒性。

Answer inducer. 当 LRM 在一个潜在的提前退出点暂停时，试验答案诱导器模块会提示模型根据目前已生成的推理内容生成一个中间答案。我们将答案分隔符 ($\backslash boxed \{ \}$) 纳入提示中，以便更精确地识别试验答案，如下所示： $A = \text{LRM}(P, T, I)$

(4)

其中 P 表示输入提示， T 表示生成的思维， I 表示答案诱导器提示，而 $A = [a_0, a_1, \dots, a_n]$ 是试验答案。Confidence evaluator. 置信度评估器计算诱导出的试验答案的置信度。它取每个词元的最高预测概率作为其置信度。对于多词元的试验答案，其整体置信度计算为所有词元得分的平均值，如下所示：

$$p(a_t) = \text{softmax}(\mathcal{M}(P, T, I, a_{<t})), \quad C = \left(\prod_{a_t \in \mathcal{V}} \max_{a_t \in \mathcal{V}} p(a_t) \right)^{1/n} \quad (5)$$

其中 $\mathcal{M}$ 是 LRM 最后一层的语言模型头。 C 的计算采用几何平均数，这更好地契合了联合概率的乘法性质，并对低概率值表现出更高的敏感性，从而提供了更强的鲁棒性。

最后，将获得的置信度与经验阈值 λ 进行比较，以决定是否提前退出。如果 $C > \lambda$ ，我们认为 LRM 当前生成的推理信息已足够，表明模型已达到 pearl reasoning。此时，DEER 停止进一步的推理动作，并继续给出结论。否则，模型将回退到上一个转换点以生成下一个思考。

DEER-PRo. 为了进一步提高珍珠推理识别的可靠性和准确性，我们引入了 DEER-PRo，它是 DEER 的一个 P 并行且 Ro 鲁棒的变体。通过在提前退出点使用不同的提示进行答案诱导，DEER-PRo 计算多个置信度值的平均值和平均绝对偏差 (MAD)，从而得出一个经过

DEER-Pro. To further improve the reliability and accuracy of pearl reasoning identification, we introduce **DEER-Pro**, a Parallel and Robust variant of DEER. Through answer elicitation using varied prompts at early-exit points, DEER-Pro calculates both the mean and Mean Absolute Deviation (MAD) of multiple confidence values, deriving a calibrated confidence score C_{cali} as follows:

$$C_{cali} = C_{avg} - \alpha \cdot C_{MAD}, \quad C_{avg} = \frac{1}{N} \sum_{i=1}^N C_i, \quad C_{MAD} = \frac{1}{N} \sum_{i=1}^N |C_i - C_{avg}| \quad (5)$$

where $C_i = \left(\prod_{a_i \in \mathcal{V}} \max_{a_i \in \mathcal{V}} \text{softmax}(\mathcal{M}(P, T, I_i, a_i^t)) \right)^{1/n}$ denotes the confidence score obtained using a specific answer inducing prompt I_i , N denotes the number of inducing attempts, and α is the fluctuation penalty strength coefficient. The introduced conservative bias C_{MAD} effectively prevents erroneous early exits caused by overestimated confidence scores resulting from positive noise in prompts, where the estimated confidence exceeds the true confidence. We demonstrate in the Appendix B that C_{cali} effectively eliminates the influence of the model’s sensitivity to answer inducer prompts on early-exit accuracy, thus substantially improving DEER’s robustness.

3.3 BRANCH-PARALLEL DECODING ACCELERATION

Intuitively, the computation of the answer inducer and confidence evaluator in DEER introduces additional latency during inference, particularly in code generation tasks where trial answers remain lengthy. This overhead diminishes the efficiency gains achieved through substantial reduction of generated CoT sequences. To address this challenge, we integrate DEER with a branch-parallel acceleration strategy (Fig. 9) that mitigates these efficiency limitations through two key mechanisms: (1) linearization of multiple branches into a single sequence for parallel generation using a specialized causal attention mask, and (2) dynamic KV cache management via confidence-based pruning. This strategy facilitates temporal overlap between trial answer evaluation and concurrent reasoning-chain generation, thereby optimizing overall inference efficiency.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Benchmarks, Metrics and Implementations. We evaluate model performance across 10 benchmarks, including 6 mathematical reasoning benchmarks: GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021), AMC 2023 (AI-MO, 2024), AIME 2024, AIME 2025 (MAA Committees), OlympiadBench (He et al., 2024), one scientific reasoning benchmark: GPQA Diamond (Rein et al., 2023), and 3 code reasoning benchmarks: HumanEval (Chen et al., 2021), BigCodeBench (Zhao et al., 2024), and LiveCodeBench (Jain et al., 2024). Among the six mathematical reasoning benchmarks, GSM8K, MATH-500, and AMC 2023 are generally considered to be relatively simple reasoning tasks, whereas AIME 2024, AIME 2025, and OlympiadBench are regarded as more challenging. Given the extensive set of evaluation benchmarks, we selectively present the most popular ones (GSM8K, MATH-500, AMC 2023, AIME 2024 and GPQA Diamond) in the main experiment. More experimental results are provided in the Appendix K. We selected *Accuracy* (**Acc**), *Token Number* (**Tok**), and *Compression Rate* (**CR**) as the evaluation metrics. **Acc** denotes the final answer accuracy. **Tok** denotes the average generation length per sample to evaluate the cost. **CR** is defined as the ratio of the average response length to that of the original model, with lower values indicating higher compression. Given the limited number of samples in datasets AMC 2023, AIME 2024, and AIME 2025, we conduct 4 sampling rounds per instance and average the results across all metrics to ensure stability and reliability. We have implemented DEER using both HuggingFace Transformers (Wolf et al., 2020) and the vLLM inference acceleration framework (Kwon et al., 2023). The experimental results presented in this paper are based on the vLLM implementation. We set the hyperparameter λ to 0.95 ($\lambda = 0.95$). For entropy-based DEER, following the 80/20 principle proposed in (Wang et al., 2025b), we designate reasoning step termination positions with entropy values exceeding 0.672 as early-exit points. For DEER-Pro, we set $N = 4$ and $\alpha = 1$. More experimental setup details are placed in Appendix C.

Backbone LLMs and Baselines. We conducted experiments on the open-source DeepSeek-R1-Distill-Qwen series of models (1.5B, 7B, 14B, and 32B)(DeepSeek-AI et al., 2025), Qwen3 series of

校准的置信度分数 C_{cali} , 如下所示:

$$C_{cali} = C_{avg} - \alpha \cdot C_{MAD}, \quad C_{avg} = \frac{1}{N} \sum_{i=1}^N C_i, \quad C_{MAD} = \frac{1}{N} \sum_{i=1}^N |C_i - C_{avg}| \quad (6)$$

其中 $C_i = \left(\prod_{a_i \in \mathcal{V}} \max_{a_i \in \mathcal{V}} \text{softmax}(\mathcal{M}(P, T, I_i, a_i^t)) \right)^{1/n}$ 表示使用特定答案诱导提示 I_i 获得的置信度分数, N 表示诱导尝试的次数, α 是波动惩罚强度系数。引入的保守偏差 C_{MAD} 有效地防止了由于提示中的正向噪声导致置信度分数被高估 (即估计置信度超过真实置信度) 而引起的错误提前退出。我们在附录 ?? 中证明, C_{cali} 有效地消除了模型对答案诱导提示的敏感性对提前退出准确性的影响, 从而显著提高了 DEER 的鲁棒性。

直观上, DEER 中答案诱导器和置信度评估器的计算在推理过程中引入了额外的延迟, 尤其是在代码生成任务中, 试验答案仍然很长。这种开销削弱了通过大幅减少生成的 CoT 序列所实现的效率提升。为了解决这一挑战, 我们将 DEER 与分支并行加速策略 (图9) 相结合, 该策略通过两个关键机制缓解了这些效率限制: (1) 使用专门的因果注意力掩码将多个分支线性化为单个序列以进行并行生成, 以及 (2) 通过基于置信度的剪枝进行动态 KV 缓存管理。该策略促进了试验答案评估与并发推理链生成之间的时间重叠, 从而优化了整体推理效率。

4 Experiments

4.1 Experimental Setup

Benchmarks, Metrics and Implementations. 我们在 10 个基准测试上评估模型性能, 包括 6 个数学推理基准: GSM8K (Cobbe et al., 2021)、MATH-500 (Hendrycks et al., 2021)、AMC 2023 (AI-MO, 2024)、AIME 2024、AIME 2025 (MAA Committees)、OlympiadBench (He et al., 2024), 一个科学推理基准: GPQA Diamond (Rein et al., 2023), 以及 3 个代码推理基准: HumanEval (Chen et al., 2021)、BigCodeBench (Zhao et al., 2024) 和 LiveCodeBench (Jain et al., 2024)。在六个数学推理基准中, GSM8K、MATH-500 和 AMC 2023 通常被认为是相对简单的推理任务, 而 AIME 2024、AIME 2025 和 OlympiadBench 则被视为更具挑战性。鉴于评估基准集广泛, 我们在主要实验中选择性呈现最流行的几个 (GSM8K、MATH-500、AMC 2023、AIME 2024 和 GPQA Diamond)。更多实验结果在附录 ?? 中提供。我们选择 *Accuracy* (**Acc**)、*Token Number* (**Tok**) 和 *Compression Rate* (**CR**) 作为评估指标。Acc 表示最终答案的准确率。Tok 表示每个样本的平均生成长度, 用于评估成本。CR 定义为平均响应长度与原始模型平均响应长度的比值, 数值越低表示压缩率越高。鉴于数据集 AMC 2023、AIME 2024 和 AIME 2025 中的样本数量有限, 我们对每个实例进行 4 轮采样, 并对所有指标的结果取平均值, 以确保稳定性和可靠性。我们已使用 HuggingFace Transformers (Wolf et al., 2020) 和 vLLM 推理加速框架 (Kwon et al., 2023) 实现了 DEER。本文呈现的实验结果基于 vLLM 实现。我们将超参数 λ 设置为 0.95 ($\lambda = 0.95$)。对于基于熵的 DEER, 遵循 (Wang et al., 2025b) 中提出的 80/20 原则, 我们将熵值超过 0.672 的推理步骤终止位置指定为提前退出点。对于 DEER-Pro, 我们设置 $N = 4$ 和 $\alpha = 1$ 。更多实验设置细节置于附录 B 中。

Backbone LLMs and Baselines. 我们在开源的 DeepSeek-R1-Distill-Qwen 系列模型 (1.5B、7B、14B 和 32B) (DeepSeek-AI et al., 2025)、Qwen3 系列模型 (1.7B、4B、8B、14B、32B) (Qwen et al., 2025)、QwQ-32B (Team, 2025) 以及 DeepSeek-R1 (Liu et al., 2025c) 上进行了实验。由于评估的模型数量众多, 我们在主实验中选取 DeepSeek-R1-Distill-Qwen-7B、Qwen3-14B 和 QwQ-32B 作为代表性示例。更多实验结果在附录 B 中提供。我们将 DEER

Table 1: Experimental results across various types of reasoning models. "Acc" denotes accuracy, "Tok" denotes token count, and "CR" denotes compression rate. ↑ indicates that higher values are better, while ↓ indicates that lower values are better. The best results are highlighted in **bold**.

Method	GSM8K			MATH			MATH-500			AMC23			AIME24			SCIENCE GPQA-D			Overall	
	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	CR↓
DeepSeek-R1-Distill-Qwen-7B																				
Vanilla	89.6	1,484	100%	87.4	3,858	100%	78.8	6,792	100%	41.7	13,765	100%	23.7	10,247	100%	64.2	100%			
TCC	88.0	892	60.1%	89.2	3,864	100.2%	82.5	6,491	95.6%	48.4	10,603	77.0%	27.3	8,442	82.4%	67.1	83.0%			
CoD	84.7	298	20.1%	83.2	1,987	51.5%	77.5	4,440	65.4%	40.0	10,519	76.4%	37.9	6,431	62.8%	64.7	55.3%			
NoThinking	87.1	284	19.1%	80.6	834	21.6%	65.0	1,911	28.1%	26.7	4,427	32.2%	29.8	724	7.1%	57.8	21.6%			
Dynasor-CoT	89.6	1,285	86.6%	89.0	2,971	77.0%	85.0	5,980	88.0%	46.7	12,695	92.2%	30.5	7,639	74.5%	68.2	83.7%			
SEAL	88.4	811	54.6%	89.4	2,661	69.0%	-	-	-	-	-	-	-	-	-	-	-	-	-	-
DEER	90.6	917	61.8%	93.8	2,143	55.5%	85.0	4,451	65.5%	49.2	9,839	71.5%	31.3	5,469	53.4%	69.2	61.5%			
DEER-Pro	91.0	989	66.7%	90.2	2,391	62.0%	87.5	4,877	71.8%	49.2	10,046	73.0%	30.6	5,682	55.5%	69.7	65.8%			
Qwen3-14B																				
Vanilla	95.1	2,047	100%	93.8	4,508	100%	95.0	7,139	100%	70.0	10,859	100%	60.1	7,339	100%	82.8	100%			
TCC	95.7	1,241	60.6%	94.6	4,484	99.5%	95.0	7,261	101.7%	70.8	11,573	106.6%	60.1	7,138	97.3%	83.3	93.1%			
CoD	85.7	648	31.7%	75.2	2,359	52.3%	72.5	4,122	57.7%	60.0	10,768	99.2%	51.0	1,177	16.0%	68.9	51.4%			
NoThinking	94.8	286	14.0%	85.0	1,228	27.2%	77.5	2,133	29.9%	26.7	7,337	67.6%	30.5	2,320	31.6%	66.9	34.1%			
Dynasor-CoT	95.6	1,483	72.4%	93.8	4,063	90.1%	95.6	6,582	92.2%	73.3	10,369	95.5%	59.6	5,968	81.3%	83.6	86.3%			
DEER	95.3	840	41.0%	94.0	3,074	68.2%	95.0	4,763	66.7%	76.7	7,619	70.2%	57.6	2,898	39.5%	83.7	57.1%			
DEER-Pro	95.3	926	45.2%	94.4	3,260	72.3%	95.6	4,905	68.7%	75.0	8,135	74.9%	61.2	4,062	55.4%	84.3	63.3%			
QwQ-32B																				
Vanilla	96.7	1,427	100%	93.8	4,508	100%	92.5	6,792	100%	66.7	10,821	100%	63.1	7,320	100%	82.6	100%			
TCC	95.8	1,348	94.5%	94.4	4,315	95.7%	90.0	6,818	100.4%	60.0	11,263	104.1%	61.6	7,593	103.7%	80.4	99.7%			
CoD	96.0	627	43.9%	94.0	3,630	80.5%	92.5	5,943	87.5%	60.0	10,731	99.2%	62.6	6,039	82.5%	81.0	78.7%			
NoThinking	96.2	1,113	78.0%	94.8	3,930	87.2%	87.5	6,908	101.7%	66.7	10,859	100.4%	63.6	7,668	104.8%	81.8	94.4%			
Dynasor-CoT	95.2	1,095	76.7%	94.2	4,176	92.6%	93.8	6,544	96.3%	63.3	11,156	103.1%	64.1	7,024	96.0%	82.1	93.0%			
DEER	96.3	977	68.5%	94.6	3,316	73.6%	95.0	5,782	85.1%	70.0	10,097	93.3%	64.1	6,163	84.2%	84.0	80.9%			
DEER-Pro	96.2	1032	72.3%	94.8	3,650	80.9%	95.0	5,811	85.6%	70.0	10,264	94.9%	64.7	6,201	84.7%	84.1	83.7%			

models (1.7B, 4B, 8B, 14B, 32B) (Qwen et al., 2025), QwQ-32B (Team, 2025), and DeepSeek-R1 (Liu et al., 2025c). Due to the large number of models evaluated, we selectively present DeepSeek-R1-Distill-Qwen-7B, Qwen3-14B, and QwQ-32B as representative examples in the main experiment. More experimental results are provided in the Appendix C. We compare DEER against existing prompt-based and output-based efficient reasoning approaches, including *Vanilla*, *TCC* (Muennighoff et al., 2025), *CoD* (Xu et al., 2025c), *NoThinking* (Ma et al., 2025a), *Dynasor-CoT* (Fu et al., 2025), and *SEAL* (Chen et al., 2025a). *Vanilla* performs direct evaluation of the LRM without any intervention. Token-Conditional Control (*TCC*) specifies a fixed token count in the system prompt to enforce a token budget; in our experiments, we set this limit based on the actual token length generated by DEER. Chain-of-Draft (*CoD*) reduce verbosity by limiting the number of words used in each reasoning step, focusing only on the essential calculations or transformations needed to progress. *NoThinking* prompts the model to skip the reasoning phase and directly generate the final answer. *Dynasor-CoT* periodically prompts the model to produce intermediate answers at fixed token intervals and triggers early exit when three consecutive answers are consistent. *SEAL* trains a steering vector to calibrate the CoT process, guiding the model toward more reliable reasoning.

4.2 MAIN RESULTS

Overall Performance. Due to space constraints, Tab.1 presents five widely adopted reasoning benchmarks, evaluated across three state-of-the-art reasoning models specifically covering three model scales, which comprehensively demonstrates DEER’s superior performance. We also provide more results across 10 datasets covering 11 models ranging from 1.5B to 671B parameters in the Appendix. It can be found that DEER demonstrates strong adaptability across various reasoning models and tasks, achieving accuracy improvements of 0.9 to 4.8 points while reducing sequence length by 19.1% to 42.9% compared to vanilla models. DEER-Pro achieves higher accuracy with only a marginal increase in generation length ranging from 2.8% to 6.2% compared to DEER. We conducted comparative experiments between DEER and DEER-Pro on additional smaller-scale models. The experimental results in Table 2 demonstrate that DEER-Pro achieves more significant accuracy improvements. It indicates that DEER-Pro effectively addresses the prompt sensitivity issues in smaller models, demonstrating its superior robustness.

Comparison with Efficient Reasoning SoTAs. Tab. 1 presents comparisons between DEER and recent efficient reasoning methods. It can be observed that DEER consistently outperforms all baselines, whereas baselines either struggle to generalize across tasks and base models, or must trade off accuracy for efficiency. Specifically, while TCC Muennighoff et al. (2025) achieve reasonable

表 1: 各类推理模型的实验结果。“Acc”表示准确率,“Tok”表示词元数量,“CR”表示压缩率。↑表示数值越高越好,而↓表示数值越低越好。最佳结果以 bold 高亮显示。

Method	GSM8K			MATH			MATH-500			AMC23			AIME24			Science GPQA-D			Overall	
	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	Tok↓	CR↓	Acc↑	CR↓
DeepSeek-R1-Distill-Qwen-7B																				
Vanilla	89.6	1,484	100%	87.4	3,858	100%	78.8	6,792	100%	41.7	13,765	100%	23.7	10,247	100%	64.2	100%			
TCC	88.0	892	60.1%	89.2	3,864	100.2%	82.5	6,491	95.6%	48.4	10,603	77.0%	27.3	8,442	82.4%	67.1	83.0%			
CoD	84.7	298	20.1%	83.2	1,987	51.5%	77.5	4,440	65.4%	40.0	10,519	76.4%	37.9	6,431	62.8%	64.7	55.3%			
NoThinking	87.1	284	19.1%	80.6	834	21.6%	65.0	1,911	28.1%	26.7	4,427	32.2%	29.8	724	7.1%	57.8	21.6%			
Dynasor-CoT	89.6	1,285	86.6%	89.0	2,971	77.0%	85.0	5,980	88.0%	46.7	12,695	92.2%	30.5	7,639	74.5%	68.2	83.7%			
SEAL	88.4	811	54.6%	89.4	2,661	69.0%	-	-	-	-	-	-	-	-	-	-	-	-	-	-
DEER	90.6	917	61.8%	93.8	2,143	55.5%	85.0	4,451	65.5%	49.2	9,839	71.5%	31.3	5,469	53.4%	69.2	61.5%			
DEER-Pro	91.0	989	66.7%	90.2	2,391	62.0%	87.5	4,877	71.8%	49.2	10,046	73.0%	30.6	5,682	55.5%	69.7	65.8%			
Qwen3-14B																				
Vanilla	95.1	2,047	100%	93.8	4,508	100%	95.0	7,139	100%	70.0	10,859	100%	60.1	7,339	100%	82.8	100%			
TCC	95.7	1,241	60.6%	94.6	4,484	99.5%	95.0	7,261	101.7%	70.8	11,573	106.6%	60.1	7,138	97.3%	83.3	93.1%			
CoD	85.7	648	31.7%	75.2	2,359	52.3%	72.5	4,122	57.7%	60.0	10,768	99.2%	51.0	1,177	16.0%	68.9	51.4%			
NoThinking	94.8	286	14.0%	85.0	1,228	27.2%	77.5	2,133	29.9%	26.7	7,337	67.6%	30.5	2,320	31.6%	66.9	34.1%			
Dynasor-CoT	95.6	1,483	72.4%	93.8	4,063	90.1%	95.6	6,582	92.2%	73.3	10,369	95.5%	59.6	5,968	81.3%	83.6	86.3%			
DEER	95.3	840	41.0%	94.0	3,074	68.2%	95.0	4,763	66.7%	76.7	7,619	70.2%	57.6	2,898	39.5%	83.7	57.1%			
DEER-Pro	95.3	926	45.2%	94.4	3,260	72.3%	95.6	4,905	68.7%	75.0	8,135	74.9%	61.2	4,062	55.4%	84.3	63.3%			
QwQ-32B																				
Vanilla	96.7	1,427	100%	93.8	4,508	100%	92.5	6,792	100%	66.7	10,821	100%	63.1	7,320	100%	82.6	100%			
TCC	95.8	1,348	94.5%	94.4	4,315	95.7%	90.0	6,818	100.4%	60.0	11,263	104.1%	61.6	7,593	103.7%	80.4	99.7%			
CoD	96.0	627	43.9%	94.0	3,630	80.5%	92.5	5,943	87.5%	60.0	10,731	99.2%	62.6	6,039	82.5%	81.0	78.7%			
NoThinking	96.2	1,113	78.0%	94.8	3,930	87.2%	87.5	6,908	101.7%	66.7	10,859	100.4%	63.6	7,668	104.8%	81.8	94.4%			
Dynasor-CoT	95.2	1,095	76.7%	94.2	4,176	92.6%	93.8	6,544	96.3%	63.3	11,156	103.1%	64.1	7,024	96.0%	82.1	93.0%			
DEER	96.3	977	68.5%	94.6	3,316	73.6%	95.0	5,782	85.1%	70.0	10,097	93.3%	64.1	6,163	84.2%	84.0	80.9%			
DEER-Pro	96.2	1032	72.3%	94.8	3,650	80.9%	95.0	5,811	85.6%	70.0	10,264	94.9%	64.7	6,201	84.7%	84.1	83.7%			

与现有的基于提示和基于输出的高效推理方法进行比较,包括 *Vanilla*、*TCC* (Muennighoff et al., 2025)、*CoD* (Xu et al., 2025c)、*NoThinking* (Ma et al., 2025a)、*Dynasor-CoT* (Fu et al., 2025) 以及 *SEAL* (Chen et al., 2025a)。*Vanilla* 对 LRM 进行直接评估,不做任何干预。令牌条件控制 (*TCC*) 在系统提示中指定固定的令牌数量以强制执行令牌预算;在我们的实验中,我们根据 DEER 生成的实际令牌长度来设置此限制。链式草稿 (*CoD*) 通过限制每个推理步骤中使用的词语数量来减少冗长,仅关注推进所需的必要计算或转换。*NoThinking* 提示模型跳过推理阶段,直接生成最终答案。*Dynasor-CoT* 定期提示模型在固定的令牌间隔处生成中间答案,并在三个连续答案一致时触发提前退出。*SEAL* 训练一个转向向量来校准 CoT 过程,引导模型进行更可靠的推理。

4.2 Main Results

Overall Performance. 由于篇幅限制,表1展示了五个广泛采用的推理基准,在三个最先进的推理模型上进行了评估,这些模型专门涵盖了三种模型规模,全面证明了 DEER 的优越性能。我们还在附录中提供了涵盖 10 个数据集、11 个参数规模从 1.5B 到 671B 的模型的更多结果。可以发现,DEER 在各种推理模型和任务中表现出强大的适应性,与原始模型相比,在将序列长度减少 19.1% 至 42.9% 的同时,实现了 0.9 到 4.8 个百分点的准确率提升。与 DEER 相比,DEER-Pro 仅以 2.8% 至 6.2% 的生成长度边际增加,就实现了更高的准确率。我们在额外的较小规模模型上进行了 DEER 与 DEER-Pro 的对比实验。表2中的实验结果表明,DEER-Pro 实现了更显著的准确率提升。这表明 DEER-Pro 有效解决了较小模型中的提示敏感性问题,展现了其卓越的鲁棒性。

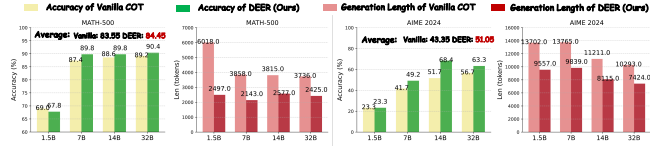


Figure 4: Experimental results of DEER compared to Vanilla CoT across DeepSeek-R1-Distill-Qwen-Series models of varying sizes on MATH-500 and AIME 2024.

efficiency-accuracy tradeoffs on simpler tasks like GSM8K by incorporating token budgets into prompts, it fails on complex problems (such as AIME24) where models ignore prompts' length constraints and generate even longer responses than vanilla CoT. As for *NoThinking* and *CoD*, while achieving dramatic length reduction, they severely compromises models' inherent reasoning capabilities. In contrast, Dynasor-CoT preserves reasoning quality but suffers from late termination due to its conservative early-exit condition, resulting in minimal length reduction. Notably, nearly all baselines fail completely on QwQ-32B due to the sporadic invalidation of its end-of-thinking delimiter `</think>` where the model continues generating reasoning steps after it and often produces duplicate `</think>` tokens (as shown in Appendix Fig. 18). Remarkably, DEER still achieves a 19.1% length reduction on QwQ-32B despite these challenges, further demonstrating its robustness.

Performance on Programming Tasks. Tab. 3 reports DEER's evaluation results across three programming tasks, completing our comprehensive coverage of reasoning models' three primary domains: mathematics, science, and programming. It demonstrates DEER's consistent effectiveness across varying programming tasks and model sizes, achieving smaller compression ratios compared to math and science tasks (average 19.9% vs. 61.5%). This enhanced compression likely originates from the inherent characteristics of code generation, where each reasoning step typically produces verbose code segments containing substantial redundant tokens.

Performance Trends across Model Sizes and Reasoning Difficulty. Fig. 4, 10 presents evaluation results on MATH-500 and AIME 2024 datasets to examine DEER's performance gains across different model sizes. It can be seen that DEER consistently enhances accuracy while reducing token consumption across all model sizes. A key observation is that smaller models (e.g., 1.5B) tend to generate significantly longer reasoning sequences with more severe overthinking phenomena. This stems from their limited reasoning capacity in discovering the correct reasoning steps during CoT generation. Consequently, our method achieves greater length reduction for these smaller models. Fig. 4 utilizes the MATH-500 (simple reasoning) and AIME 2024 (challenging reasoning) datasets as representative benchmarks. The results demonstrate DEER's dual capability: it achieves more superior compression ratios on simpler problems while delivering more substantial accuracy gains on complex tasks. This precisely addresses two critical needs in reasoning systems: the efficiency demands in simple scenarios and the growing accuracy requirements in challenging scenarios.

4.3 ABLATION STUDY

Performance Trends across Token Budes. Fig. 5 evaluates DEER's performance across varying token budgets (controlled by different max length settings). In plots (a) and (f), the x-axis represents the actual length of model-generated CoT sequences, while the y-axis indicates model accuracy. The optimal balance between accuracy and efficiency is demonstrated by curves positioned closer to the top-left corner. The blue shaded regions quantitatively represent DEER's performance gains: vertical height corresponds to accuracy improvement and horizontal width to token compression benefit. It can be seen that DEER consistently outperforms vanilla methods, as all points located upper-left to vanilla ones. As shown in the four-column plots on the right, we observe that vanilla models generate longer sequences with higher accuracy as token budgets increase, confirming test-time scaling. Notably, DEER demonstrates an adaptive tradeoff: under constrained token budgets, it achieves greater gains in accuracy but reduced benefits in length compression. Conversely, the opposite trend is observed with larger token budgets. This indicates that our method can dynamically adjust token budgets to meet varying requirements for accuracy-efficiency in different scenarios.

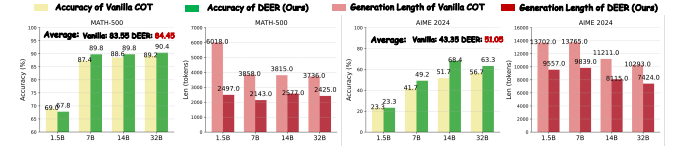


图 4: DEER 与 Vanilla CoT 在不同规模的 DeepSeek-R1-Distill-Qwen-Series 模型上，在 MATH-500 和 AIME 2024 上的实验结果对比。

Comparison with Efficient Reasoning SoTAs. 表 1 展示了 DEER 与近期高效推理方法之间的比较。可以观察到，DEER 始终优于所有基线方法，而基线方法要么难以在不同任务和基础模型之间泛化，要么必须在准确性和效率之间进行权衡。具体来说，虽然 TCC Muennighoff et al. (2025) 通过将令牌预算纳入提示，在 GSM8K 等较简单的任务上实现了合理的效率-准确性权衡，但在复杂问题（如 AIME24）上却失败了，因为模型忽略了提示的长度限制，生成了比普通 CoT 更长的回答。至于 NoThinking 和 CoD，虽然实现了显著的长度缩减，但它们严重损害了模型固有的推理能力。相比之下，Dynasor-CoT 保持了推理质量，但由于其保守的提前退出条件导致终止过晚，导致长度缩减微乎其微。值得注意的是，几乎所有基线方法在 QwQ-32B 上都完全失败，原因是其思维结束分隔符 `</think>` 会偶发性失效，模型在其后继续生成推理步骤，并经常产生重复的 `</think>` 令牌（如附录图 18 所示）。值得注意的是，尽管存在这些挑战，DEER 在 QwQ-32B 上仍然实现了 19.1% 的长度缩减，进一步证明了其鲁棒性。

Performance on Programming Tasks. 表 3 报告了 DEER 在三个编程任务上的评估结果，完成了我们对推理模型三个主要领域（数学、科学和编程）的全面覆盖。它展示了 DEER 在不同编程任务和模型规模上的一致有效性，与数学和科学任务相比，实现了更小的压缩率（平均 19.9% 对比 61.5%）。这种增强的压缩可能源于代码生成的固有特性，其中每个推理步骤通常会包含大量冗余令牌的冗长代码段。

Performance Trends across Model Sizes and Reasoning Difficulty. 图 4、10 展示了在 MATH-500 和 AIME 2024 数据集上的评估结果，以检验 DEER 在不同模型规模上的性能增益。可以看出，DEER 在所有模型规模上都能持续提高准确性，同时减少令牌消耗。一个关键的观察是，较小的模型（例如 1.5B）倾向于生成明显更长的推理序列，并表现出更严重的过度思考现象。这源于它们在 CoT 生成过程中发现正确推理步骤的有限能力。因此，我们的方法对这些较小的模型实现了更大的长度缩减。图 4 使用 MATH-500（简单推理）和 AIME 2024（挑战性推理）数据集作为代表性基准。结果表明了 DEER 的双重能力：它在更简单的问题上实现了更优越的压缩率，同时在复杂任务上带来了更显著的准确性提升。这恰好解决了推理系统中的两个关键需求：简单场景下的效率需求和挑战性场景下日益增长的准确性需求。

Performance Trends across Token Budes. 图 5 评估了 DEER 在不同令牌预算（由不同的最大长度设置控制）下的性能。在子图 (a) 和 (f) 中，x 轴表示模型生成的思维链序列的实际长度，而 y 轴表示模型准确率。准确率与效率之间的最佳平衡由靠近左上角的曲线所展示。蓝色阴影区域定量地表示了 DEER 的性能增益：垂直高度对应准确率提升，水平宽度对应令牌压缩收益。可以看出，DEER 始终优于原始方法，因为所有点都位于原始方法对应点的左上方。如右侧四列子图所示，我们观察到随着令牌预算增加，原始模型生成长度更长、准确率更高的序列，这证实了测试时缩放效应。值得注意的是，DEER 展示了一种自适应的权衡：在受限的令牌预算下，它在准确率上获得更大增益，但在长度压缩方面的收益减

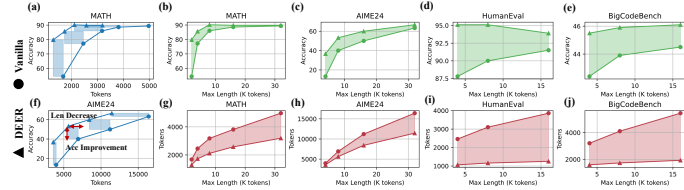


Figure 5: Performance comparison between DEER and baselines based on the DeepSeek-R1-Distill-Qwen-14B model across four datasets under different token budget settings.

Impact of Reasoning Transition Monitor Choices. In the main experiments, we employ "Wait" as the early-exit monitoring signal, denoted as DEER(W). This simple linguistic marker-based approach yields promising results. To compare the impact of different early-exit signals on DEER performance, we conduct additional experiments using "Alternatively" as the signal, as well as entropy-based monitoring for early-exit detection. The corresponding results are presented in Tab. 5 and 6. Tab. 5 collects statistics on the number and average length of reasoning chunks obtained by dividing the original CoT with potential exit points. The chunk numbers indicate that DEER(Ent) presents the most early-exit opportunities while DEER(A) offers the fewest, exhibiting a negative correlation with average generation length. The results in Tab. 6 across additional datasets and models demonstrate that both entropy-based and linguistic marker-based monitoring exhibit comparable superior performance, significantly outperforming the baseline. In large-scale real-world deployments, we advocate for the linguistic marker-based approach given its implementation simplicity and efficiency. Appendix I provides further exploration between these two monitoring strategies.

Robustness of threshold λ . Fig. 6 shows the performance of DEER on MATH-500 dataset with different threshold λ . The results indicate that when the threshold is set too low, a minor additional reduction in reasoning length leads to a significant drop in accuracy, reflecting an overcorrection of overthinking. Conversely, when the threshold is set too high, the model exits reasoning too late, resulting in prolonged reasoning lengths with a decline in accuracy. Moreover, it can be seen that our method is robust to λ within the range of 0.9-0.97, eliminating the need for hyperparameter tuning. Tab. 4 presents our robustness investigation of the threshold λ across additional datasets using Qwen3. The experimental results demonstrate that Qwen3 exhibits superior robustness to λ , maintaining consistently strong performance within the range of 0.8-0.97. Additionally, the experimental results in the appendix, conducted across 11 models and 10 datasets, uniformly employ 0.95 as the threshold value. The consistently strong results further demonstrate DEER’s generalization capability and robustness. Appendix Section J reveals that the underlying source of DEER’s robustness originates from confidence polarization phenomenon.

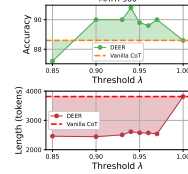


Figure 6: Impact of λ .

4.4 DISCUSSION

Efficiency Improvement. To accurately verify the gains brought by DEER and its Branch-Parallel accelerated variant in end-to-end inference efficiency, we measured the average latency on MATH-500 and AMC 2023 datasets based on huggingface transformers (Wolf et al., 2020). As shown in Fig. 7 (a), original DEER reduces the latency by 27.9% to 40.1% while the proposed branch-parallel decoding variant reduces the latency by 36.3% to 58.6%. This suggests that Branch-Parallel DEER achieves further speed improvements by efficiently reducing the latency of trial answer inducing and confidence evaluation. Additionally, we mapped the latency speedup against the length savings for every sample on MATH-500. Fig. 7 (b) illustrates that the

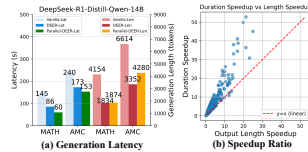


Figure 7: Efficiency Improvement.

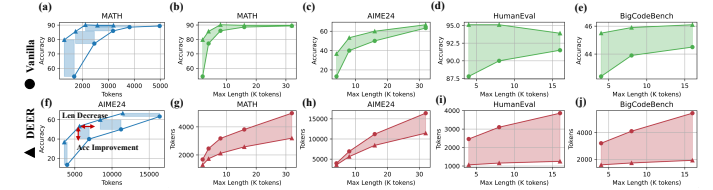


图 5: 基于 DeepSeek-R1-Distill-Qwen-14B 模型, DEER 与基线方法在四个数据集上、不同令牌预算设置下的性能比较。

少。相反, 在更大的令牌预算下则观察到相反的趋势。这表明我们的方法可以动态调整令牌预算, 以满足不同场景下对准确率与效率的不同要求。

4.3 Ablation Study

Impact of Reasoning Transition Monitor Choices. 在主要实验中, 我们采用 "Wait" 作为早期退出的监控信号, 记作 DEER(W)。这种基于简单语言标记的方法取得了有希望的结果。为了比较不同早期退出信号对 DEER 性能的影响, 我们进行了额外的实验, 使用 "Alternatively" 作为信号, 以及基于熵的监控来进行早期退出检测。相应的结果展示在表 5 和 6 中。表 5 收集了通过将原始 CoT 在潜在退出点处划分所得推理块的数量和平均长度的统计数据。块的数量表明 DEER(Ent) 提供了最多的早期退出机会, 而 DEER(A) 提供的最少, 这与平均生成长度呈负相关。表 6 中在额外数据集和模型上的结果表明, 基于熵的监控和基于语言标记的监控都表现出可比且优越的性能, 显著优于基线。在大规模实际部署中, 鉴于其实现的简单性和高效性, 我们提倡使用基于语言标记的方法。附录 G 对这两种监控策略进行了进一步的探讨。

Robustness of threshold λ . 图 6 展示了 DEER 在 MATH-500 数据集上使用不同阈值 λ 的性能。结果表明, 当阈值设置过低时, 推理长度的微小额外减少会导致准确率显著下降, 这反映了对过度思考的过度校正。相反, 当阈值设置过高时, 模型退出推理过晚, 导致推理时间延长且准确率下降。此外, 可以看出我们的方法在 0.9-0.97 范围内对 λ 具有鲁棒性, 无需进行超参数调优。表 4 展示了我们使用 Qwen3 在额外数据集上对阈值 λ 的鲁棒性研究。实验结果表明, Qwen3 对 λ 表现出卓越的鲁棒性, 在 0.8-0.97 范围内始终保持强劲性能。此外, 附录中在 11 个模型和 10 个数据集上进行的实验结果, 均统一采用 0.95 作为阈值。持续强劲的结果进一步证明了 DEER 的泛化能力和鲁棒性。附录第 H 节揭示了 DEER 鲁棒性的根本来源在于置信度极化现象。

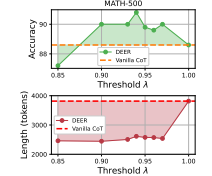


图 6: λ 的影响。

4.4 Discussion

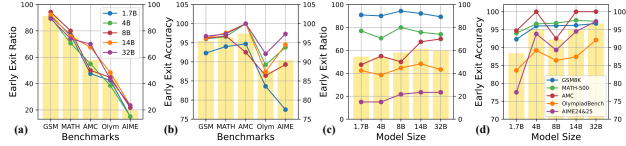


Figure 8: Early-exit rates and accuracy of early-exited samples of DEER. Figures (a) and (b) share a common legend, as do figures (c) and (d). The height of each bar reflects the average value.

ratio between latency speedup and length savings exhibits a superlinear trend, reinforcing the significance of DEER in enhancing inference speed. In Section H of the Appendix, we theoretically prove the efficiency of DEER and explain the underlying cause of the superlinear speedup.

Exploring the Effectiveness of DEER’s Early-Exit Mechanism. Fig. 8 presents the early-exit rate and the accuracy of early-exited samples across Qwen3-series models of varying sizes. As shown in Fig. 8(a), DEER’s early-exit rate decreases with increasing task difficulty, which accounts for its relatively lower compression performance on complex tasks compared to simpler ones. Fig. 8(b) reveals that, although early-exit accuracy declines somewhat as task difficulty increases, it remains consistently high—ranging from 88% to 98%. In a concurrent study, Zhang et al. (2025a) trained a probe to decide whether to early exit. However, their probe achieves an accuracy of only around 80% on MATH-500, which is significantly lower than DEER’s 95%. This indicates that the model inherently possesses the ability to assess answer correctness, and that DEER effectively harnesses this capability. As illustrated in Figures 8(c) and (d), although minor differences exist across tasks of varying difficulty, there is a general trend toward higher early-exit rates and improved accuracy as model size increases. This observation implies a positive relationship between DEER’s performance and the capacity of the model: larger models yield more accurate confidence estimates, which in turn lead to better early-exit decisions. We further investigate the reasons behind DEER’s accuracy gains. Fig. 11 indicates that DEER corrects more answers (green bars) than it alters incorrectly (red bars) through early exits. This suggests that DEER not only saves computational cost by exiting early on questions it could correctly answer, but also corrects problematic thinking.

Case Study. Fig. 13 shows that both DEER and vanilla CoT arrive at the correct answer during the first reasoning step, as shown in the green box. The difference lies in the fact that DEER exits early after evaluating the confidence of the trial answer as sufficiently high, thus producing the correct result. In contrast, the vanilla CoT proceeds to the next reasoning action. After double-checking and switching reasoning approaches, the model becomes trapped in an endless cycle of verification due to inconsistent answers from the two approaches, ultimately failing to provide a final answer. Besides, Fig. 17 shows that LRMs implicitly know when to leave early, and our method is simple and effective to realize such potential of the model itself. Please refer to Appendix L for details.

5 RELATED WORK

Following the taxonomy of efficient reasoning established in (Sui et al., 2025; Wang et al., 2025a), we categorize related work into three classes: **post-training based** methods use SFT (Yu et al., 2024; Kang et al., 2025; Xia et al., 2025; Ma et al., 2025b; Munkhbat et al., 2025; Liu et al., 2024; Han et al., 2024) with variable-length CoT data or incorporate length rewards (Team et al., 2025b; Luo et al., 2025a; Aggarwal & Welleck, 2025; Arora & Zanette, 2025; Yeo et al., 2025; Shen et al., 2025b; Qu et al., 2025; Cui et al., 2025) in reinforcement learning to enable the model to adaptively generate chains of thought of different lengths, which is beyond our training-free scope. **Prompt-based** methods (Han et al., 2024; Xu et al., 2025b; Lee et al., 2025; Renze & Guven, 2024; Chen et al., 2024) use varying prompts to enforce reasoning models to generate concise CoT with less unnecessary reasoning steps. **Output-based** methods aim to accelerate reasoning generation during the model’s decoding phase, and DEER falls into this category. However, most prior works (Xie et al., 2023; Liao et al., 2025; Li et al., 2024; Manvi et al., 2024; Aggarwal et al., 2023) focus on optimizing best-of-N sampling, which is irrelevant to our study. Instead, we select three recent concurrent works Nothinking (Ma et al., 2025a), Dynasor-CoT (Fu et al., 2025), and SEAL (Chen et al., 2025a) as baselines for comparison. More related works can be seen in Appendix M.

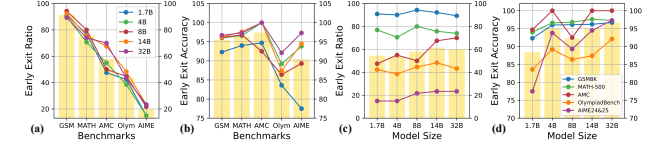


图 8: DEER 的提前退出率及提前退出样本的准确率。图 (a) 和 (b) 共享一个图例，图 (c) 和 (d) 亦然。每个条形的高度反映了平均值。

Efficiency Improvement. 为了准确验证 DEER 及其分支并行加速变体在端到端推理效率上带来的增益，我们基于 huggingface transformers (Wolf et al., 2020) 在 MATH-500 和 AMC 2023 数据集上测量了平均延迟。如图 7 (a) 所示，原始 DEER 将延迟降低了 27.9% 至 40.1%，而提出的分支并行解码变体将延迟降低了 36.3% 至 58.6%。这表明分支并行 DEER 通过有效减少

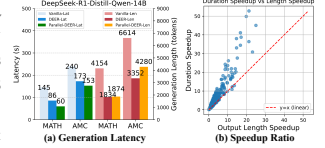


图 7: 效率提升。

试验答案生成和置信度评估的延迟，实现了进一步的加速提升。此外，我们针对 MATH-500 上的每个样本，将延迟加速比与长度节省量进行了映射。图 7 (b) 表明，延迟加速比与长度节省量之间的比率呈现出超线性趋势，这强化了 DEER 在提升推理速度方面的重要性。在附录的章节 F 中，我们从理论上证明了 DEER 的效率，并解释了超线性加速的根本原因。Exploring the Effectiveness of DEER’s Early-Exit Mechanism. 图 8 展示了不同规模的 Qwen3 系列模型的早退率以及早退样本的准确率。如图 8(a) 所示，DEER 的早退率随着任务难度的增加而降低，这解释了其在复杂任务上相比简单任务压缩性能相对较低的原因。图 8(b) 显示，尽管随着任务难度增加，早退准确率有所下降，但其始终保持在较高水平——范围从 88% 到 98%。在一项同期研究中，Zhang et al. (2025a) 训练了一个探针来决定是否早退。然而，他们的探针在 MATH-500 上的准确率仅为 80% 左右，显著低于 DEER 的 95%。这表明模型本身具备评估答案正确性的能力，而 DEER 有效地利用了这种能力。如图 8(c) 和 (d) 所示，尽管在不同难度的任务间存在微小差异，但总体上存在随着模型规模增大，早退率更高、准确率提升的趋势。这一观察结果意味着 DEER 的性能与模型容量之间存在正相关关系：更大的模型能产生更准确的置信度估计，从而做出更好的早退决策。我们进一步研究了 DEER 获得准确率提升的原因。图 11 表明，DEER 通过早退纠正的答案（绿色条）多于其错误改变的答案（红色条）。这表明 DEER 不仅通过在能正确回答的问题上早退来节省计算成本，还能纠正有问题的思维过程。

Case Study. 图 13 显示，DEER 和原始的 CoT 都在第一个推理步骤中得出了正确答案，如绿色框所示。区别在于，DEER 在评估试验答案的置信度足够高后提前退出，从而产生了正确结果。相比之下，原始的 CoT 则继续进行下一个推理动作。在双重检查并切换推理方法后，由于两种方法得出的答案不一致，模型陷入了无休止的验证循环，最终未能提供最终答案。此外，图 17 表明，

6 CONCLUSION

This paper verifies the rationale behind the early exit motivation in CoT generation, and accordingly proposes a training-free dynamic early exit algorithm, which makes the reasoning model withdraw from subsequent thinking when the thinking amount is just enough. Our method comprehensively evaluated across reasoning models of varying model sizes and demonstrates superior performance with fewer tokens on ten classical reasoning benchmarks, which offers a win-win solution to the trade-off between accuracy and efficiency commonly encountered in test-time scaling.

7 ETHICS STATEMENT

This work adheres to the ICLR Code of Ethics. We affirm that our research has been conducted with integrity, honesty, and respect for ethical principles throughout all stages of the work.

- All findings presented in this paper are reported accurately and honestly. We have not fabricated, falsified, or misrepresented any data or results. Our methods and experimental procedures are described transparently to ensure reproducibility.
- All datasets used in this research were obtained and utilized in accordance with their licenses and terms of use. For any data involving personal information, we ensured compliance with privacy regulations and obtained appropriate ethical approvals where necessary.
- All contributions to this work have been properly acknowledged. We have appropriately cited all sources and prior work that influenced our research. All co-authors have made substantial contributions to the work and have agreed to the submission.
- We have carefully considered the broader implications of our work. While our research aims to advance the field positively, we acknowledge potential dual-use concerns and encourage responsible deployment of our methods in real-world applications.

8 REPRODUCIBILITY STATEMENT

To ensure the reproducibility of our results, we have provided detailed descriptions of our experimental setup, hyperparameters, and implementation details. Code and supplementary materials are made available where possible to facilitate verification and future research.

REFERENCES

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.
- Pranjal Aggarwal, Aman Madaan, Yiming Yang, et al. Let’s sample step by step: Adaptive-consistency for efficient reasoning and coding with llms. *arXiv preprint arXiv:2305.11860*, 2023.
- AI-MO. Amc 2023, 2024. URL <https://huggingface.co/datasets/AI-MO/aimo-validation-amc>.
- Daman Arora and Andrea Zanette. Training language models to reason efficiently. *arXiv preprint arXiv:2502.04463*, 2025.
- Simon A Aytes, Jinheon Baek, and Sung Ju Hwang. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*, 2025.
- Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

LRMs 隐性地知道何时提前退出，而我们的方法简单有效，能够实现模型自身的这种潜力。详情请参阅附录 I。

5 Related Work

遵循 (Sui et al., 2025; Wang et al., 2025a) 中建立的高效推理分类法，我们将相关工作分为三类：post-training based 方法使用带有可变长度思维链数据的 SFT (Yu et al., 2024; Kang et al., 2025; Xia et al., 2025; Ma et al., 2025b; Munkhbat et al., 2025; Liu et al., 2024; Han et al., 2024) 或在强化学习中引入长度奖励 (Team et al., 2025b; Luo et al., 2025a; Aggarwal & Welleck, 2025; Arora & Zanette, 2025; Yeo et al., 2025; Shen et al., 2025b; Qu et al., 2025; Cui et al., 2025)，使模型能够自适应地生成不同长度的思维链，这超出了我们无需训练的范围。Prompt-based 方法 (Han et al., 2024; Xu et al., 2025b; Lee et al., 2025; Renze & Guven, 2024; Chen et al., 2024) 使用不同的提示来强制推理模型生成简洁的思维链，减少不必要的推理步骤。Output-based 方法旨在加速模型解码阶段的推理生成，DEER 属于此类。然而，大多数先前工作 (Xie et al., 2023; Liao et al., 2025; Li et al., 2024; Manvi et al., 2024; Aggarwal et al., 2023) 侧重于优化最佳 N 采样，这与我们的研究无关。相反，我们选择了三个近期的并行工作 Nothinking (Ma et al., 2025a)、Dynasor-CoT (Fu et al., 2025) 和 SEAL (Chen et al., 2025a) 作为比较基线。更多相关工作可见附录 J。

6 Conclusion

本文验证了思维链生成中提前退出动机背后的原理，并据此提出了一种无需训练的、动态的提前退出算法，该算法使推理模型在思考量恰好足够时退出后续思考。我们的方法在不同模型规模的推理模型上进行了全面评估，并在十个经典推理基准上以更少的标记数展示了优越性能，为测试时扩展中常见的准确性与效率权衡提供了一个双赢的解决方案。

7 Ethics statement

本工作遵循 ICLR 道德准则。我们确认我们的研究在所有阶段都以诚信、诚实和尊重道德原则的方式进行。

- 本文中呈现的所有发现均被准确、诚实地报告。我们未捏造、伪造或歪曲任何数据或结果。我们的方法和实验过程描述透明，以确保可复现性。
- 本研究中使用的数据集均根据其许可证和使用条款获取和使用。对于任何涉及个人信息的数据，我们确保遵守隐私法规，并在必要时获得了适当的伦理批准。
- 对本工作的所有贡献均已得到适当承认。我们恰当地引用了所有影响我们研究的来源和先前工作。所有合著者均对本工作做出了实质性贡献，并同意提交。

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.

Qiguang Chen, Libo Qin, Jiaqi Wang, Jingxuan Zhou, and Wanxiang Che. Unlocking the capabilities of thought: A reasoning boundary framework to quantify and optimize chain-of-thought. *Advances in Neural Information Processing Systems*, 37:54872–54904, 2024.

Runjin Chen, Zhenyu Zhang, Junyuan Hong, Souvik Kundu, and Zhangyang Wang. Seal: Steerable reasoning calibration of large language models for free, 2025a. URL <https://arxiv.org/abs/2504.07986>.

Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do not think that much for 2+3=? on the overthinking of ol-like llms, 2025b. URL <https://arxiv.org/abs/2412.21187>.

Jeffrey Cheng and Benjamin Van Durme. Compressed chain of thought: Efficient reasoning through dense representations. *arXiv preprint arXiv:2412.13171*, 2024.

Yu-Neng Chuang, Helen Zhou, Prathusha Sarma, Parikshit Gopalan, John Boccio, Sara Bolouki, and Xia Hu. Learning to route llms with confidence tokens. *arXiv preprint arXiv*, 2410, 2024.

Yu-Neng Chuang, Leisheng Yu, Guanchu Wang, Lizhe Zhang, Zirui Liu, Xuanning Cai, Yang Sui, Vladimir Braverman, and Xia Hu. Confident or seek stronger: Exploring uncertainty-based on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*, 2025.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.

Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao, et al. The danger of overthinking: Examining the reasoning-action dilemma in agentic tasks. *arXiv preprint arXiv:2502.08235*, 2025.

Yingqian Cui, Pengfei He, Jingying Zeng, Hui Liu, Xianfeng Tang, Zhenwei Dai, Yan Han, Chen Luo, Jing Huang, Zhen Li, et al. Stepwise perplexity-guided refinement for efficient chain-of-thought reasoning in large language models. *arXiv preprint arXiv:2502.13260*, 2025.

Muzhi Dai, Chenxu Yang, and Qingyi Si. S-grpo: Early exit via reinforcement learning in reasoning models, 2025. URL <https://arxiv.org/abs/2505.07686>.

Renfei Dang, Shujian Huang, and Jiajun Chen. Internal bias in reasoning models leads to overthinking, 2025. URL <https://arxiv.org/abs/2505.16448>.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai

- 我们已仔细考虑了本工作更广泛的影响。虽然我们的研究旨在积极推动该领域发展，但我们承认潜在的军民两用关切，并鼓励在现实世界应用中负责任地部署我们的方法。

8 Reproducibility statement

为确保我们结果的可复现性，我们提供了实验设置、超参数和实现细节的详细描述。在可能的情况下，我们提供了代码和补充材料，以便于验证和未来的研究。

参考文献

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.
- Pranjal Aggarwal, Aman Madaan, Yiming Yang, et al. Let’s sample step by step: Adaptive-consistency for efficient reasoning and coding with llms. *arXiv preprint arXiv:2305.11860*, 2023.
- AI-MO. Amc 2023, 2024. URL <https://huggingface.co/datasets/AI-MO/aimo-validation-amc>.
- Daman Arora and Andrea Zanette. Training language models to reason efficiently. *arXiv preprint arXiv:2502.04463*, 2025.
- Simon A Aytes, Jinheon Baek, and Sung Ju Hwang. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*, 2025.
- Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam,

Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liye Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanxia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.

Razvan-Gabriel Dumitru, Darius Peteleaza, Vikas Yadav, and Liangming Pan. Conciserl: Conciseness-guided reinforcement learning for efficient reasoning models, 2025. URL <https://arxiv.org/abs/2505.17250>.

Yichao Fu, Junda Chen, Yonghao Zhuang, Zheyu Fu, Ion Stoica, and Hao Zhang. Reasoning without self-doubt: More efficient chain-of-thought through certainty probing. In *ICLR 2025 Workshop on Foundation Models in the Wild*, 2025. URL <https://openreview.net/forum?id=wPK4IMJfdX>.

Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-budget-aware llm reasoning. *arXiv preprint arXiv:2412.18547*, 2024.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024. URL <https://arxiv.org/abs/2402.14008>.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.

Yao Huang, Huanran Chen, Shouwei Ruan, Yichi Zhang, Xingxing Wei, and Yinpeng Dong. Mitigating overthinking in large reasoning models via manifold steering, 2025. URL <https://arxiv.org/abs/2505.22411>.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.

Lingjie Jiang, Xun Wu, Shaohan Huang, Qingxiu Dong, Zewen Chi, Li Dong, Xingxing Zhang, Tengchao Lv, Lei Cui, and Furu Wei. Think only when you need with large hybrid-reasoning models, 2025a. URL <https://arxiv.org/abs/2505.14631>.

Yuxuan Jiang, Dawei Li, and Frank Ferraro. Drp: Distilled reasoning pruning with skill-aware step decomposition for efficient large reasoning models, 2025b. URL <https://arxiv.org/abs/2505.13975>.

Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.

Qiguang Chen, Libo Qin, Jiaqi Wang, Jingxuan Zhou, and Wanxiang Che. Unlocking the capabilities of thought: A reasoning boundary framework to quantify and optimize chain-of-thought. *Advances in Neural Information Processing Systems*, 37:54872–54904, 2024.

Runjin Chen, Zhenyu Zhang, Junyuan Hong, Souvik Kundu, and Zhenyang Wang. Seal: Steerable reasoning calibration of large language models for free, 2025a. URL <https://arxiv.org/abs/2504.07986>.

Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhusheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Do not think that much for 2+3=? on the overthinking of o1-like llms, 2025b. URL <https://arxiv.org/abs/2412.21187>.

Jeffrey Cheng and Benjamin Van Durme. Compressed chain of thought: Efficient reasoning through dense representations. *arXiv preprint arXiv:2412.13171*, 2024.

Yu-Neng Chuang, Helen Zhou, Prathusha Sarma, Parikshit Gopalan, John Boccio, Sara Bolouki, and Xia Hu. Learning to route llms with confidence tokens. *arXiv preprint arXiv*, 2410, 2024.

Yu-Neng Chuang, Leisheng Yu, Guanchu Wang, Lizhe Zhang, Zirui Liu, Xuanning Cai, Yang Sui, Vladimir Braverman, and Xia Hu. Confident or seek stronger: Exploring uncertainty-based on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*, 2025.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.

Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao, et al. The danger of overthinking: Examining the reasoning-action dilemma in agentic tasks. *arXiv preprint arXiv:2502.08235*, 2025.

Yingqian Cui, Pengfei He, Jingying Zeng, Hui Liu, Xianfeng Tang, Zhenwei Dai, Yan Han, Chen Luo, Jing Huang, Zhen Li, et al. Stepwise perplexity-guided refinement for efficient chain-of-thought reasoning in large language models. *arXiv preprint arXiv:2502.13260*, 2025.

- Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 24312–24320, 2025.
- kvcache ai. KTransformers: A flexible framework for experiencing cutting-edge llm inference optimizations. <https://github.com/kvcache-ai/ktransformers>, 2025. URL <https://github.com/kvcache-ai/ktransformers>. GitHub repository, commit a1b2c3d, accessed 2025-05-16.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Ayeong Lee, Ethan Che, and Tianyi Peng. How well do llms compress their own chain-of-thought? a token complexity approach. *arXiv preprint arXiv:2503.01141*, 2025.
- Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning. *arXiv preprint arXiv:2401.10480*, 2024.
- Zheng Li, Qingxiu Dong, Jingyuan Ma, Di Zhang, Kai Jia, and Zhifang Sui. Selfbudgeter: Adaptive token allocation for efficient llm reasoning, 2025a. URL <https://arxiv.org/abs/2505.11274>.
- Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhijiang Guo, Le Song, and Cheng-Lin Liu. From system 1 to system 2: A survey of reasoning large language models, 2025b. URL <https://arxiv.org/abs/2502.17419>.
- Baohao Liao, Yuhui Xu, Hanze Dong, Junnan Li, Christof Monz, Silvio Savarese, Doyen Sahoo, and Caiming Xiong. Reward-guided speculative decoding for efficient llm reasoning. *arXiv preprint arXiv:2501.19324*, 2025.
- Hanbing Liu, Lang Cao, Yuanyi Ren, Mengyu Zhou, Haoyu Dong, Xiaojun Ma, Shi Han, and Dongmei Zhang. Bingo: Boosting efficient reasoning of llms via dynamic and significance-based reinforcement learning, 2025a. URL <https://arxiv.org/abs/2506.08125>.
- Tengxiao Liu, Qipeng Guo, Xiangkun Hu, Cheng Jiayang, Yue Zhang, Xipeng Qiu, and Zheng Zhang. Can language models learn to skip steps? *arXiv preprint arXiv:2411.01855*, 2024.
- Wei Liu, Ruochen Zhou, Yiyun Deng, Yuzhen Huang, Junteng Liu, Yuntian Deng, Yizhe Zhang, and Junxian He. Learn to reason efficiently with adaptive length-based reward shaping, 2025b. URL <https://arxiv.org/abs/2505.15612>.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective, 2025c. URL <https://arxiv.org/abs/2503.20783>.
- Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*, 2025a.
- Yijia Luo, Yulin Song, Xingyao Zhang, Jiaheng Liu, Weixun Wang, GengRu Chen, Wenbo Su, and Bo Zheng. Deconstructing long chain-of-thought: A structured reasoning optimization framework for long cot distillation, 2025b. URL <https://arxiv.org/abs/2503.16385>.
- Wenjie Ma, Jingxuan He, Charlie Snell, Tyler Griggs, Sewon Min, and Matei Zaharia. Reasoning models can be effective without thinking. *arXiv preprint arXiv:2504.09858*, 2025a.
- Xinyin Ma, Guangnian Wan, Runkeng Yu, Gongfan Fang, and Xinchao Wang. Cot-valve: Length-compressible chain-of-thought tuning. *arXiv preprint arXiv:2502.09601*, 2025b.
- Rohin Manvi, Anikait Singh, and Stefano Ermon. Adaptive inference-time compute: LLMs can predict if they can do better, even mid-generation. *arXiv preprint arXiv:2410.02725*, 2024.

- Muzhi Dai, Chenxu Yang, and Qingyi Si. S-grpo: Early exit via reinforcement learning in reasoning models, 2025. URL <https://arxiv.org/abs/2505.07686>.
- Renfei Dang, Shujian Huang, and Jiajun Chen. Internal bias in reasoning models leads to overthinking, 2025. URL <https://arxiv.org/abs/2505.16448>.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuan Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Razvan-Gabriel Dumitru, Darius Peteleaza, Vikas Yadav, and Liangming Pan. Conciserl: Conciseness-guided reinforcement learning for efficient reasoning models, 2025. URL <https://arxiv.org/abs/2505.17250>.
- Yichao Fu, Junda Chen, Yonghao Zhuang, Zheyu Fu, Ion Stoica, and Hao Zhang. Reasoning without self-doubt: More efficient chain-of-thought through certainty probing. In ICLR 2025 Workshop on Foundation Models in the Wild, 2025. URL <https://openreview.net/forum?id=wpK4IMJfdX>.

- MAA Committees. Aime problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. sl: Simple test-time scaling, 2025. URL <https://arxiv.org/abs/2501.19393>.
- Tergel Munkhbat, Namgyu Ho, Seo Hyun Kim, Yongjin Yang, Yujin Kim, and Se-Young Yun. Self-training elicits concise reasoning in large language models. *arXiv preprint arXiv:2502.20122*, 2025.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data, 2024. URL <https://arxiv.org/abs/2406.18665>.
- OpenAI. Learning to reason with llms. <https://openai.com/research/learning-to-reason-with-llms>, 2025. Accessed: 15 March 2025.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35: 27730–27744, 2022.
- Ziqing Qiao, Yongheng Deng, Jiali Zeng, Dong Wang, Lai Wei, Guanbo Wang, Fandong Meng, Jie Zhou, Ju Ren, and Yaoxue Zhang. Concise: Confidence-guided compression in step-by-step efficient reasoning, 2025. URL <https://arxiv.org/abs/2505.04881>.
- Yuxiao Qu, Matthew YR Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. Optimizing test-time compute via meta reinforcement fine-tuning. *arXiv preprint arXiv:2503.07572*, 2025.
- Qwen., An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yaqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Shyam Sundhar Ramesh, Yifan Hu, Iason Chailamas, Viraj Mehta, Pier Giuseppe Sessa, Haitham Bou Ammar, and Ilija Bogunovic. Group robust preference optimization in reward-free RLHF. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=PRAsjrmXXK>.
- Carl Rasmussen and Zoubin Ghahramani. Occam’s razor. In T. Leen, T. Dietterich, and V. Tresp (eds.), *Advances in Neural Information Processing Systems*, volume 13. MIT Press, 2000. URL https://proceedings.neurips.cc/paper_files/paper/2000/file/0950ca92a4dcf426067cfd2246bb5ff3-Paper.pdf.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL <https://arxiv.org/abs/2311.12022>.
- Matthew Renze and Erhan Guven. The benefits of a concise chain of thought on problem-solving in large language models. In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pp. 476–483. IEEE, 2024.
- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J Reddi. Reasoning with latent thoughts: On the power of looped transformers. *arXiv preprint arXiv:2502.17416*, 2025.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-budget-aware llm reasoning. *arXiv preprint arXiv:2412.18547*, 2024.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhit-ing Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024. URL <https://arxiv.org/abs/2402.14008>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Yao Huang, Huanran Chen, Shouwei Ruan, Yichi Zhang, Xingxing Wei, and Yinpeng Dong. Mitigating overthinking in large reasoning models via manifold steering, 2025. URL <https://arxiv.org/abs/2505.22411>.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Lingjie Jiang, Xun Wu, Shaohan Huang, Qingxiu Dong, Zewen Chi, Li Dong, Xingxing Zhang, Tengchao Lv, Lei Cui, and Furu Wei. Think only when you need with large hybrid-reasoning models, 2025a. URL <https://arxiv.org/abs/2505.14631>.
- Yuxuan Jiang, Dawei Li, and Frank Ferraro. Drp: Distilled reasoning pruning with skill-aware step decomposition for efficient large reasoning models, 2025b. URL <https://arxiv.org/abs/2505.13975>.
- Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 24312–24320, 2025.
- kvcache ai. KTransformers: A flexible framework for experiencing cutting-edge llm inference optimizations. <https://github.com/kvcache-ai/ktransformers>, 2025. URL <https://github.com/kvcache-ai/ktransformers>. GitHub repository, commit a1b2c3d, accessed 2025-05-16.

- Xuan Shen, Yizhou Wang, Xiangxi Shi, Yanzhi Wang, Pu Zhao, and Jiuxiang Gu. Efficient reasoning with hidden thinking. *arXiv preprint arXiv:2501.19201*, 2025a.
- Yi Shen, Jian Zhang, Jieyun Huang, Shuming Shi, Wenjing Zhang, Jiangze Yan, Ning Wang, Kai Wang, and Shiguo Lian. Dast: Difficulty-adaptive slow-thinking for large reasoning models. *arXiv preprint arXiv:2503.04472*, 2025b.
- Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. Codi: Compressing chain-of-thought into continuous space via self-distillation. *arXiv preprint arXiv:2502.21074*, 2025c.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Jiwon Song, Dongwon Jo, Yulhwa Kim, and Jae-Joon Kim. Reasoning path compression: Compressing generation trajectories for efficient llm reasoning, 2025. URL <https://arxiv.org/abs/2505.13866>.
- DiJia Su, Hanlin Zhu, Yingchen Xu, Jiantao Jiao, Yuandong Tian, and Qinqing Zheng. Token assorted: Mixing latent and text tokens for improved language model reasoning. *arXiv preprint arXiv:2502.03275*, 2025.
- Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models, 2025. URL <https://arxiv.org/abs/2503.16419>.
- Wenhui Tan, Jiaze Li, Jianzhong Ju, Zhenbo Luo, Jian Luan, and Ruihua Song. Think silently, think fast: Dynamic latent compression of llm reasoning chains, 2025. URL <https://arxiv.org/abs/2505.16552>.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, Haiqing Guo, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haotian Yao, Haotian Zhao, Haoyu Lu, Haoze Li, Haozhen Yu, Hongcheng Gao, Huabin Zheng, Huan Yuan, Jia Chen, Jianhang Guo, Jianlin Su, Jianzhou Wang, Jie Zhao, Jin Zhang, Jingyuan Liu, Junjie Yan, Junyan Wu, Lidong Shi, Ling Ye, Longhui Yu, Mengnan Dong, Neo Zhang, Ningchen Ma, Qiwei Pan, Qucheng Gong, Shaowei Liu, Shengling Ma, Shupeng Wei, Sihan Cao, Siying Huang, Tao Jiang, Weihao Gao, Weimin Xiong, Weiran He, Weixiao Huang, Wenhao Wu, Wenyang He, Xianghui Wei, Xianqing Jia, Xingzhe Wu, Xinran Xu, Xinxing Zu, Xinyu Zhou, Xuehai Pan, Y. Charles, Yang Li, Yangyang Hu, Yangyang Liu, Yanru Chen, Yejie Wang, Yibo Liu, Yidao Qin, Yifeng Liu, Ying Yang, Yiping Bao, Yulun Du, Yuxin Wu, Yuzhi Wang, Zaida Zhou, Zhaoji Wang, Zhaowei Li, Zhen Zhu, Zheng Zhang, Zhexu Wang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Ziyao Xu, and Zonghan Yang. Kimi k1.5: Scaling reinforcement learning with llms, 2025a. URL <https://arxiv.org/abs/2501.12599>.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1.5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025b.
- Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025. URL <https://qwenlm.github.io/blog/qwq-32b/>.
- Songjun Tu, Jiahao Lin, Qichao Zhang, Xiangyu Tian, Linjing Li, Xiangyuan Lan, and Dongbin Zhao. Learning when to think: Shaping adaptive reasoning in rl-style models via multi-stage rl, 2025. URL <https://arxiv.org/abs/2505.10832>.
- Hongru WANG, Deng Cai, Wanjun Zhong, Shijue Huang, Jeff Z. Pan, Zeming Liu, and Kam-Fai Wong. Self-reasoning language models: Unfold hidden reasoning chains with few reasoning catalyst. In *Workshop on Reasoning and Planning for Large Language Models*, 2025. URL <https://openreview.net/forum?id=p4wXiD8FX1>.

- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Ayeong Lee, Ethan Che, and Tianyi Peng. How well do llms compress their own chain-of-thought? a token complexity approach. *arXiv preprint arXiv:2503.01141*, 2025.
- Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning. *arXiv preprint arXiv:2401.10480*, 2024.
- Zheng Li, Qingxiu Dong, Jingyuan Ma, Di Zhang, Kai Jia, and Zhifang Sui. Selfbudgeter: Adaptive token allocation for efficient llm reasoning, 2025a. URL <https://arxiv.org/abs/2505.11274>.
- Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhijiang Guo, Le Song, and Cheng-Lin Liu. From system 1 to system 2: A survey of reasoning large language models, 2025b. URL <https://arxiv.org/abs/2502.17419>.
- Baohao Liao, Yuhui Xu, Hanze Dong, Junnan Li, Christof Monz, Silvio Savarese, Doyen Sahoo, and Caiming Xiong. Reward-guided speculative decoding for efficient llm reasoning. *arXiv preprint arXiv:2501.19324*, 2025.
- Hanbing Liu, Lang Cao, Yuanyi Ren, Mengyu Zhou, Haoyu Dong, Xiaojun Ma, Shi Han, and Dongmei Zhang. Bingo: Boosting efficient reasoning of llms via dynamic and significance-based reinforcement learning, 2025a. URL <https://arxiv.org/abs/2506.08125>.
- Tengxiao Liu, Qipeng Guo, Xiangkun Hu, Cheng Jiayang, Yue Zhang, Xipeng Qiu, and Zheng Zhang. Can language models learn to skip steps? *arXiv preprint arXiv:2411.01855*, 2024.
- Wei Liu, Ruochen Zhou, Yiyun Deng, Yuzhen Huang, Junteng Liu, Yuntian Deng, Yizhe Zhang, and Junxian He. Learn to reason efficiently with adaptive length-based reward shaping, 2025b. URL <https://arxiv.org/abs/2505.15612>.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective, 2025c. URL <https://arxiv.org/abs/2503.20783>.
- Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*, 2025a.

- Rui Wang, Hongru Wang, Boyang Xue, Jianhui Pang, Shudong Liu, Yi Chen, Jiahao Qiu, Derek Fai Wong, Heng Ji, and Kam-Fai Wong. Harnessing the reasoning economy: A survey of efficient reasoning for large language models, 2025a. URL <https://arxiv.org/abs/2503.24377>.
- Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning, 2025b. URL <https://arxiv.org/abs/2506.01939>.
- Yibo Wang, Li Shen, Huanjin Yao, Tiansheng Huang, Rui Liu, Naiqiang Tan, Jiaying Huang, Kai Zhang, and Dacheng Tao. R1-compress: Long chain-of-thought compression via chunk compression and search, 2025c. URL <https://arxiv.org/abs/2505.16838>.
- Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Huggingface’s transformers: State-of-the-art natural language processing, 2020. URL <https://arxiv.org/abs/1910.03771>.
- Yuyang Wu, Yifei Wang, Tianqi Du, Stefanie Jegelka, and Yisen Wang. When more is less: Understanding chain-of-thought length in llms. *arXiv preprint arXiv:2502.07266*, 2025.
- Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*, 2025.
- Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, James Xu Zhao, Min-Yen Kan, Junxian He, and Michael Xie. Self-evaluation guided beam search for reasoning. *Advances in Neural Information Processing Systems*, 36:41618–41650, 2023.
- Fengli Xu, Qianye Hao, Zefang Zong, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, Tianjian Ouyang, Fanjin Meng, Chenyang Shao, Yuwei Yan, Qinglong Yang, Yiwen Song, Sijian Ren, Xinyuan Hu, Yu Li, Jie Feng, Chen Gao, and Yong Li. Towards large reasoning models: A survey of reinforced reasoning with large language models, 2025a. URL <https://arxiv.org/abs/2501.09686>.
- Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025b.
- Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less, 2025c. URL <https://arxiv.org/abs/2502.18600>.
- Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. Softcot: Soft chain-of-thought for efficient reasoning with llms. *arXiv preprint arXiv:2502.12134*, 2025d.
- Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning, 2025. URL <https://arxiv.org/abs/2502.03387>.
- Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying long chain-of-thought reasoning in llms. *arXiv preprint arXiv:2502.03373*, 2025.
- Bin Yu, Hang Yuan, Haotian Li, Xueyin Xu, Yuliang Wei, Bailing Wang, Weizhen Qi, and Kai Chen. Long-short chain-of-thought mixture supervised fine-tuning eliciting efficient reasoning in large language models, 2025. URL <https://arxiv.org/abs/2505.03469>.

- Yijia Luo, Yulin Song, Xingyao Zhang, Jiaheng Liu, Weixun Wang, GengRu Chen, Wenbo Su, and Bo Zheng. Deconstructing long chain-of-thought: A structured reasoning optimization framework for long cot distillation, 2025b. URL <https://arxiv.org/abs/2503.16385>.
- Wenjie Ma, Jingxuan He, Charlie Snell, Tyler Griggs, Sewon Min, and Matei Zaharia. Reasoning models can be effective without thinking. *arXiv preprint arXiv:2504.09858*, 2025a.
- Xinyin Ma, Guangnian Wan, Rungeng Yu, Gongfan Fang, and Xinchao Wang. Cot-valve: Length-compressible chain-of-thought tuning. *arXiv preprint arXiv:2502.09601*, 2025b.
- Rohin Manvi, Anikait Singh, and Stefano Ermon. Adaptive inference-time compute: Llms can predict if they can do better, even mid-generation. *arXiv preprint arXiv:2410.02725*, 2024.
- MAA Committees. Aime problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hamaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. sl: Simple test-time scaling, 2025. URL <https://arxiv.org/abs/2501.19393>.
- Tergel Munkhbat, Namgyu Ho, Seo Hyun Kim, Yongjin Yang, Yujin Kim, and Se-Young Yun. Self-training elicits concise reasoning in large language models. *arXiv preprint arXiv:2502.20122*, 2025.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data, 2024. URL <https://arxiv.org/abs/2406.18665>.
- OpenAI. Learning to reason with llms. <https://openai.com/research/learning-to-reason-with-llms>, 2025. Accessed: 15 March 2025.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ziqing Qiao, Yongheng Deng, Jiali Zeng, Dong Wang, Lai Wei, Guanbo Wang, Fandong Meng, Jie Zhou, Ju Ren, and Yaoxue Zhang. Concise: Confidence-guided compression in step-by-step efficient reasoning, 2025. URL <https://arxiv.org/abs/2505.04881>.

- Ping Yu, Jing Xu, Jason Weston, and Ilia Kulikov. Distilling system 2 into system 1. *arXiv preprint arXiv:2407.06023*, 2024.
- Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Reasoning models know when they're right: Probing hidden states for self-verification. *arXiv preprint arXiv:2504.05419*, 2025a.
- Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. Adapthink: Reasoning models can learn when to think, 2025b. URL <https://arxiv.org/abs/2505.13417>.
- Wenyuan Zhang, Shuaiyi Nie, Xinghua Zhang, Zefeng Zhang, and Tingwen Liu. S1-bench: A simple benchmark for evaluating system 1 thinking capability of large reasoning models. *ArXiv*, abs/2504.10368, 2025c. URL <https://api.semanticscholar.org/CorpusID:277781494>.
- Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, Shuohang Wang, Yelong Shen, and Xin Eric Wang. Soft thinking: Unlocking the reasoning potential of llms in continuous concept space, 2025d. URL <https://arxiv.org/abs/2505.15778>.
- Rongzhi Zhu, Yi Liu, Zequn Sun, Yiwei Wang, and Wei Hu. When can large reasoning models save thinking? mechanistic analysis of behavioral divergence in reasoning, 2025. URL <https://arxiv.org/abs/2505.15276>.
- Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

- Yuxiao Qu, Matthew YR Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. Optimizing test-time compute via meta reinforcement fine-tuning. *arXiv preprint arXiv:2503.07572*, 2025.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Shyam Sundhar Ramesh, Yifan Hu, Iason Chaimalas, Viraj Mehta, Pier Giuseppe Sessa, Haitham Bou Ammar, and Ilija Bogunovic. Group robust preference optimization in reward-free RLHF. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024. URL <https://openreview.net/forum?id=PRAsjrmXXX>.
- Carl Rasmussen and Zoubin Ghahramani. Occam's razor. In T. Leen, T. Dietterich, and V. Tresp (eds.), *Advances in Neural Information Processing Systems*, volume 13. MIT Press, 2000. URL https://proceedings.neurips.cc/paper_files/paper/2000/file/0950ca92a4dcf426067cfd2246bb5ff3-Paper.pdf.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL <https://arxiv.org/abs/2311.12022>.
- Matthew Renze and Erhan Guven. The benefits of a concise chain of thought on problem-solving in large language models. In 2024 2nd International Conference on Foundation and Large Language Models (FLLM), pp. 476–483. IEEE, 2024.
- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J Reddi. Reasoning with latent thoughts: On the power of looped transformers. *arXiv preprint arXiv:2502.17416*, 2025.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Xuan Shen, Yizhou Wang, Xiangxi Shi, Yanzhi Wang, Pu Zhao, and Jiuxiang Gu. Efficient reasoning with hidden thinking. *arXiv preprint arXiv:2501.19201*, 2025a.
- Yi Shen, Jian Zhang, Jieyun Huang, Shuming Shi, Wenjing Zhang, Jiangze Yan, Ning Wang, Kai Wang, and Shiguo

A PILOT EXPERIMENT SETUP

We selected AIME2024 (MAA Committees) as the test set for exploratory experiments to perform a qualitative analysis and further conducted a quantitative analysis through experiments on MATH-500 (Hendrycks et al., 2021), GPQA-Diamond (Rein et al., 2023). All experiments were conducted on DeepSeek-R1-Distill-Qwen-14B (DeepSeek-AI et al., 2025). In our experiments, we first enabled the LRM to perform a complete inference on the test set (including both the slow thinking and conclusion contents). Then, we preserved the thinking content and divided it into thinking chunks based on the action transition points. Samples with more than five thinking chunks were retained. For these samples, we retained varying proportions (20%-90%) of their thinking chunks and appended an end-of-thinking token delimiter to each truncated reasoning sequence to forcibly terminate the slow-thinking process. The model then generated its final conclusion based on the partial reasoning contents. For the conclusions obtained with varying thinking contents, we evaluated their correctness and presented the results of each sample in Figure 1. Furthermore, we investigated the number of samples that remained correct after early exiting when they were originally correct, as well as the number of samples that became correct after early exiting when they were originally incorrect, across three datasets in Figure 2.

B PROOF OF DEER-PRO’S EFFECTIVENESS AGAINST NOISE.

B.1 NOISE INDEPENDENCE OF THE MAD-CALIBRATED STRATEGY

Let us define the true confidence μ as the model’s actual probability of deriving the correct answer given the current reasoning content, corresponding to the real probability of pearl reasoning existing at this location. The early-exit decision threshold is denoted as λ , and the model executes early exit when $\mu > \lambda$. Since answer inducing prompts may introduce ε , the model’s output confidence fluctuates around the true confidence, yielding an observed confidence of $C_i = \mu + \varepsilon_i$. In practical testing environments, we compare C_i with λ to determine whether to perform early exit. Without loss of generality, we assume the noise terms ε_i to be independently and identically distributed (i.i.d.), a Gaussian distribution with mean 0 and standard deviation σ , i.e., $\varepsilon_i \sim N(0, \sigma^2)$. Here, σ represents the model’s sensitivity to prompt phrasing. A larger σ indicates higher model sensitivity, resulting in greater confidence fluctuations.

Next, we demonstrate DEER-PRO’s effectiveness against noise interference by comparing the decision error rates of DEER-PRO (C_{cali}), DEER (C_i), an averaging approach (C_{avg}) in critical risk scenarios. Suppose C_{cali} and C_{avg} each conduct N times answer inducing in parallel, with N identical to that in Equation (5). Given that preserving accuracy takes precedence over early-exit speedup gains in reasoning scenarios, we designate risk scenarios as those where true confidence $\mu < \lambda$. We then compute the false positive probability of observing $C > \lambda$ due to noise interference.

B.1.1 PROBABILITY OF ERROR FOR A SINGLE PROMPT:

$$P_{FP}(\text{Single}) = P(C_i > \lambda) = P(\mu + \varepsilon_i > \lambda) = P(\varepsilon_i > \lambda - \mu) \quad (6)$$

Since $\varepsilon_i \sim N(0, \sigma^2)$, we can standardize it as:

$$P_{FP}(\text{Single}) = P\left(\frac{\varepsilon_i}{\sigma} > \frac{\lambda - \mu}{\sigma}\right) \quad (7)$$

Let $Z = \frac{\varepsilon_i}{\sigma}$, then $Z \sim N(0, 1)$, then:

$$P_{FP}(\text{Single}) = P\left(Z > \frac{\lambda - \mu}{\sigma}\right) = 1 - \Phi\left(\frac{\lambda - \mu}{\sigma}\right) \quad (8)$$

where Φ is the cumulative distribution function (CDF) of the standard normal distribution.

B.1.2 PROBABILITY OF ERROR FOR AVERAGED CONFIDENCE:

For the averaged confidence C_{avg} , the noise term is still Gaussian, $\varepsilon_{avg} \sim N(0, \sigma^2/N)$. The probability of error for averaged confidence is:

$$P_{FP}(\text{Avg}) = P(C_{avg} > \lambda) = 1 - \Phi\left(\sqrt{N} \frac{\lambda - \mu}{\sigma}\right) \quad (9)$$

Lian. Dast: Difficulty-adaptive slow-thinking for large reasoning models. arXiv preprint arXiv:2503.04472, 2025b.

Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. Codi: Compressing chain-of-thought into continuous space via self-distillation. arXiv preprint arXiv:2502.21074, 2025c.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.

Jiwon Song, Dongwon Jo, Yulhwa Kim, and Jae-Joon Kim. Reasoning path compression: Compressing generation trajectories for efficient llm reasoning, 2025. URL <https://arxiv.org/abs/2505.13866>.

Dijia Su, Hanlin Zhu, Yingchen Xu, Jiantao Jiao, Yuandong Tian, and Qingqing Zheng. Token assorted: Mixing latent and text tokens for improved language model reasoning. arXiv preprint arXiv:2502.03275, 2025.

Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models, 2025. URL <https://arxiv.org/abs/2503.16419>.

Wenhui Tan, Jiaye Li, Jianzhong Ju, Zhenbo Luo, Jian Luan, and Ruihua Song. Think silently, think fast: Dynamic latent compression of llm reasoning chains, 2025. URL <https://arxiv.org/abs/2505.16552>.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, Haiqing Guo, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haotian Yao, Haotian Zhao, Haoyu Lu, Haoze Li, Haozhen Yu, Hongcheng Gao, Huabin Zheng, Huan Yuan, Jia Chen, Jianhang Guo, Jianlin Su, Jianzhou Wang, Jie Zhao, Jin Zhang, Jingyuan Liu, Junjie Yan, Junyan Wu, Lidong Shi, Ling Ye, Longhui Yu, Mengnan Dong, Neo Zhang, Ningchen Ma, Qiwei Pan, Qucheng Gong, Shaowei Liu, Shengling Ma, Shupeng Wei, Sihan Cao, Siying Huang, Tao Jiang, Weihao Gao, Weimin Xiong, Weiran He, Weixiao Huang, Wenhao Wu, Wenyang He, Xianghui Wei, Xianqing Jia, Xingzhe Wu, Xinran Xu, Xinxing Zu, Xinyu Zhou, Xuehai Pan, Y. Charles, Yang Li, Yangyang Hu, Yangyang Liu, Yanru Chen, Yejie Wang, Yibo Liu, Yidao Qin, Yifeng Liu, Ying Yang, Yiping Bao, Yulun Du, Yuxin Wu, Yuzhi Wang, Zaida Zhou, Zhaoji Wang, Zhaowei Li, Zhen Zhu, Zheng Zhang, Zhexu Wang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Ziyao Xu, and Zonghan Yang. Kimi k1.5: Scaling reinforcement learning with llms, 2025a. URL <https://arxiv.org/abs/2501.12599>.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1.5: Scaling reinforcement learning with llms. arXiv preprint arXiv:2501.12599, 2025b.

Since $\frac{(\lambda-\mu)\sqrt{N}}{\sigma} > \frac{\lambda-\mu}{\sigma}$ for $N > 1$, it follows that $P_{\text{FP}}(\text{Avg}) < P_{\text{FP}}(\text{Single})$. It indicates that simply averaging multiple observed confidence values can mitigate noise interference. Nevertheless, confidence averaging fails to address the fundamental problem, as the error rate P_{FP} remains dependent on the noise standard deviation σ , increasing monotonically as σ grows. For models with substantial intrinsic noise (large σ), the parameter inside the Φ function converges to zero, driving P_{FP} toward 0.5. This indicates that high-noise models reduce to random guessing, regardless of the decision threshold λ . Therefore, the reliability of traditional approaches is severely constrained by the model's inherent noise level σ , a factor beyond our control.

B.1.3 PROBABILITY OF ERROR FOR MAD-CALIBRATED CONFIDENCE (DEER-PRO):

$$P_{\text{FP}}(\text{calibration}) = P(\mathcal{C}_{\text{cali}} > \lambda) = P(\mathcal{C}_{\text{avg}} - \alpha \cdot \mathcal{C}_{\text{MAD}} > \lambda) \quad (10)$$

Substituting $\mathcal{C}_{\text{avg}} = \mu + \varepsilon_{\text{avg}}$, we obtain:

$$P_{\text{FP}}(\text{calibration}) = P(\mu + \varepsilon_{\text{avg}} - \alpha \cdot \mathcal{C}_{\text{MAD}} > \lambda) \quad (11)$$

Rearranging the terms in the equation yields:

$$P_{\text{FP}}(\text{calibration}) = P(\varepsilon_{\text{avg}} > \alpha \cdot \mathcal{C}_{\text{MAD}} + \lambda - \mu) \quad (12)$$

Next, we will discuss the robustness of DEER-PRO under two distinct scenarios.

Scenario 1: Approximate estimation of $\mathcal{C}_{\text{MAD}}$

Under our assumption where $\varepsilon_i \sim N(0, \sigma^2)$, we have $\varepsilon_{\text{avg}} \sim N(0, \sigma^2/N)$ and $E[\mathcal{C}_{\text{MAD}}] \approx 0.8\sigma$ (we will provide the proof in Section B.3). For large N , the law of large numbers allows us to approximate $\mathcal{C}_{\text{MAD}}$ as 0.8σ . Given this assumption, we have:

$$P_{\text{FP}}(\text{calibration}) = P(\mathcal{C}_{\text{avg}} > \lambda + 0.8\sigma\alpha) \quad (13)$$

The above equation reveals that DEER-PRO fundamentally differs by employing an adaptive threshold that scales with noise:

$$\lambda_{\text{effective}} = \lambda + 0.8\sigma\alpha \quad (14)$$

We proceed to reformulate the equation by transforming $P_{\text{FP}}(\text{calibration})$ into the cumulative distribution function (CDF) of the standard normal distribution Φ . For Equation (12), we substitute $\mathcal{C}_{\text{MAD}} = 0.8\sigma$ and obtain:

$$P_{\text{FP}}(\text{calibration}) = P(\varepsilon_{\text{avg}} > 0.8\sigma\alpha + \lambda - \mu) \quad (15)$$

Since $\varepsilon_{\text{avg}} \sim N(0, \sigma^2/N)$:

$$P_{\text{FP}}(\text{calibration}) = 1 - \Phi\left(\frac{(\lambda - \mu + 0.8\sigma\alpha)\sqrt{N}}{\sigma}\right) \quad (16)$$

Simplifying:

$$P_{\text{FP}}(\text{calibration}) = 1 - \Phi\left(\sqrt{N}\left(\frac{\lambda - \mu}{\sigma} + 0.8\alpha\right)\right) \quad (17)$$

When noise is minimal:

$$\lambda_{\text{effective}} \rightarrow \lambda \quad (18)$$

The calibrated method behaves like standard thresholding, maintaining high efficiency.

When noise dominates, $\frac{\lambda - \mu}{\sigma} \rightarrow 0$. The false positive rate becomes:

$$P_{\text{FP}} = 1 - \Phi(0.8\alpha\sqrt{N}) \quad (19)$$

which is independent of σ . It indicates that DEER-PRO effectively prevent early exit from reducing to random guessing ($P_{\text{FP}} \rightarrow 0.5$).

Scenario 2: Exact computation of $\mathcal{C}_{\text{MAD}}$ without approximation.

From $\mathcal{C}_{\text{MAD}} = \frac{1}{N} \sum_{i=1}^N |\mathcal{C}_i - \mathcal{C}_{\text{avg}}|$, we know that $\mathcal{C}_{\text{MAD}}$ is positive, therefore dividing both sides of the equation by this term, we obtain:

$$P(\varepsilon_{\text{avg}}/\mathcal{C}_{\text{MAD}} > \alpha + (\lambda - \mu)/\mathcal{C}_{\text{MAD}}) \quad (20)$$

Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025. URL <https://qwenlm.github.io/blog/qwq-32b/>.

Songjun Tu, Jiahao Lin, Qichao Zhang, Xiangyu Tian, Linjing Li, Xiangyuan Lan, and Dongbin Zhao. Learning when to think: Shaping adaptive reasoning in rl-style models via multi-stage rl, 2025. URL <https://arxiv.org/abs/2505.10832>.

Hongru WANG, Deng Cai, Wanjun Zhong, Shijue Huang, Jeff Z. Pan, Zeming Liu, and Kam-Fai Wong. Self-reasoning language models: Unfold hidden reasoning chains with few reasoning catalyst. In Workshop on Reasoning and Planning for Large Language Models, 2025. URL <https://openreview.net/forum?id=p4wXiD8FX1>.

Rui Wang, Hongru Wang, Boyang Xue, Jianhui Pang, Shudong Liu, Yi Chen, Jiahao Qiu, Derek Fai Wong, Heng Ji, and Kam-Fai Wong. Harnessing the reasoning economy: A survey of efficient reasoning for large language models, 2025a. URL <https://arxiv.org/abs/2503.24377>.

Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning, 2025b. URL <https://arxiv.org/abs/2506.01939>.

Yibo Wang, Li Shen, Huanjin Yao, Tiansheng Huang, Rui Liu, Naiqiang Tan, Jiaying Huang, Kai Zhang, and Dacheng Tao. R1-compress: Long chain-of-thought compression via chunk compression and search, 2025c. URL <https://arxiv.org/abs/2505.16838>.

Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. arXiv preprint arXiv:2109.01652, 2021.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Huggingface's transformers: State-of-the-art natural language processing, 2020. URL <https://arxiv.org/abs/1910.03771>.

Yuyang Wu, Yifei Wang, Tianqi Du, Stefanie Jegelka, and Yisen Wang. When more is less: Understanding chain-of-thought length in llms. arXiv preprint arXiv:2502.07266, 2025.

Since $\mu < \lambda$, then $\alpha + (\lambda - \mu)/C_{MAD} > \alpha$, so we can obtain:

$$P(\varepsilon_{avg}/C_{MAD} > \alpha + (\lambda - \mu)/C_{MAD}) < P(\varepsilon_{avg}/C_{MAD} > \alpha) \quad (21)$$

Therefore, $P(\varepsilon_{avg}/C_{MAD} > \alpha)$ is an upper bound for $P_{FP}(\text{calibration})$. For the ratio $\varepsilon_{avg}/C_{MAD}$, ε_{avg} represents the signal of the noise, while C_{MAD} represents the internal disorder of the noise. Therefore, we can define $\varepsilon_{avg}/C_{MAD}$ as the Signal-to-Noise Ratio (SNR).

Next, let us analyze the properties of SNR. Under our assumption where $\varepsilon_i \sim N(0, \sigma^2)$, we have $\varepsilon_{avg} \sim N(0, \sigma^2/N)$ and $E[C_{MAD}] \approx 0.8\sigma$. Since both ε_{avg} and C_{MAD} are proportional to σ , we can write SNR as:

$$\text{SNR} = (\sigma * Z_{avg}) / (\sigma * Z_{MAD}) = Z_{avg}/Z_{MAD} \quad (22)$$

where $Z_{avg} \sim N(0, 1/N)$ is a standardized noise mean, and Z_{mad} is a random variable related to MAD/σ whose distribution does not depend on σ . Hence, the probability distribution of the SNR is independent of the model noise standard deviation σ .

$$P_{FP}(\text{calibration}) < P(Z_{avg}/Z_{MAD} > \alpha) \quad (23)$$

Therefore, $P_{FP}(\text{calibration})$ is influenced only by the number of prompts N and the signal-to-noise ratio threshold α , where larger values of N and α lead to lower error rates.

Through the transformation from an absolute threshold test to a self-normalized SNR test, our MAD strategy effectively decouples decision-making from the model's intrinsic and uncontrollable noise level σ . In contrast to traditional approaches that break down under high-noise conditions, our method delivers consistent, robust performance independent of model noise levels.

B.2 ANALYSIS OF MAD-CALIBRATED STRATEGY'S SUPERIOR PERFORMANCE

In this section, we formally prove based on the event space that the false positive probability of the MAD strategy, $P_{FP}(\text{MAD})$ ($P_{FP}(\text{calibration})$), is significantly superior to that of the simple averaging strategy, $P_{FP}(\text{Avg})$, and consequently also outperforms $P_{FP}(\text{Single})$.

Theorem. The false positive probability of the MAD-calibrated strategy satisfies $P_{FP}(\text{MAD}) \leq \rho \cdot P_{FP}(\text{Avg})$, where $\rho = O(\exp(-\Theta(N)))$ is an exponentially decaying factor in the number of prompts N .

Proof. The false positive events are defined as:

$$E_{Avg} = \{\varepsilon : \varepsilon_{avg} > c\} \quad (24)$$

$$E_{MAD} = \{\varepsilon : \varepsilon_{avg} - \alpha \cdot \text{MAD} > c\} \quad (25)$$

where $c = \lambda - \mu > 0$ is the threshold gap, $\text{MAD} = \frac{1}{N} \sum_{i=1}^N |\varepsilon_i - \varepsilon_{avg}|$ is the mean absolute deviation, $\varepsilon_{avg} = \frac{1}{N} \sum_{i=1}^N \varepsilon_i$ is the average noise, and there are N independent noise terms $\varepsilon_i \sim N(0, \sigma^2)$ for $i = 1, \dots, N$.

Since $\alpha > 0$ and $\text{MAD} \geq 0$ by definition, we have:

$$\varepsilon_{avg} - \alpha \cdot \text{MAD} \leq \varepsilon_{avg} \quad (26)$$

Therefore, if $\varepsilon_{avg} - \alpha \cdot \text{MAD} > c$, then necessarily $\varepsilon_{avg} > c$. This establishes:

$$E_{MAD} \subseteq E_{Avg} \quad (27)$$

Consequently:

$$P_{FP}(\text{MAD}) = P(E_{MAD}) \leq P(E_{Avg}) = P_{FP}(\text{Avg}) \quad (28)$$

To quantify the improvement beyond this basic inequality, we decompose the probability using conditional probability:

$$P_{FP}(\text{MAD}) = P(E_{MAD} \cap E_{Avg}) = P(E_{MAD}|E_{Avg}) \cdot P(E_{Avg}) \quad (29)$$

Since $E_{MAD} \subseteq E_{Avg}$, we have:

$$P(E_{MAD}|E_{Avg}) = P(\varepsilon_{avg} - \alpha \cdot \text{MAD} > c | \varepsilon_{avg} > c) \quad (30)$$

Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. arXiv preprint arXiv:2502.12067, 2025.

Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, James Xu Zhao, Min-Yen Kan, Junxian He, and Michael Xie. Self-evaluation guided beam search for reasoning. Advances in Neural Information Processing Systems, 36: 41618–41650, 2023.

Fengli Xu, Qianye Hao, Zefang Zong, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, Tianjian Ouyang, Fanjin Meng, Chenyang Shao, Yuwei Yan, Qinglong Yang, Yiwen Song, Sijian Ren, Xinyuan Hu, Yu Li, Jie Feng, Chen Gao, and Yong Li. Towards large reasoning models: A survey of reinforced reasoning with large language models, 2025a. URL <https://arxiv.org/abs/2501.09686>.

Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. arXiv preprint arXiv:2502.18600, 2025b.

Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less, 2025c. URL <https://arxiv.org/abs/2502.18600>.

Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. Softcot: Soft chain-of-thought for efficient reasoning with llms. arXiv preprint arXiv:2502.12134, 2025d.

Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning, 2025. URL <https://arxiv.org/abs/2502.03387>.

Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying long chain-of-thought reasoning in llms. arXiv preprint arXiv:2502.03373, 2025.

Bin Yu, Hang Yuan, Haotian Li, Xueyin Xu, Yuliang Wei, Bailing Wang, Weizhen Qi, and Kai Chen. Long-short chain-of-thought mixture supervised fine-tuning eliciting efficient reasoning in large language models, 2025. URL <https://arxiv.org/abs/2505.03469>.

Ping Yu, Jing Xu, Jason Weston, and Ilia Kulikov. Distilling system 2 into system 1. arXiv preprint arXiv:2407.06023, 2024.

Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Reasoning models know when they're right: Probing hidden states for self-verification. arXiv preprint arXiv:2504.05419, 2025a.

This can be rewritten as:

$$P(E_{\text{MAD}}|E_{\text{Avg}}) = P\left(\text{MAD} < \frac{\varepsilon_{\text{avg}} - c}{\alpha} \middle| \varepsilon_{\text{avg}} > c\right) \quad (31)$$

Define the improvement factor:

$$\rho = P\left(\text{MAD} < \frac{\varepsilon_{\text{avg}} - c}{\alpha} \middle| \varepsilon_{\text{avg}} > c\right) \quad (32)$$

Then:

$$P_{\text{FP}}(\text{MAD}) = \rho \cdot P_{\text{FP}}(\text{Avg}) \quad (33)$$

To evaluate ρ , we analyze the structure of noise vectors that satisfy $\varepsilon_{\text{avg}} > c$. There are two primary patterns:

Pattern A (Coherent Pattern): All noise terms are close to c . Formally, for some small $\delta > 0$:

$$\text{Pattern A} = \{\varepsilon : |\varepsilon_i - c| < \delta \text{ for all } i = 1, \dots, N\} \quad (34)$$

Under Pattern A:

- $\varepsilon_{\text{avg}} \approx c + O(\delta/\sqrt{N})$ (by the central limit theorem)
- $\text{MAD} \leq 2\delta$ (since all values are within 2δ of each other)

Pattern B (Outlier Pattern): A few large outliers with remaining values near zero. For example:

- k values with $\varepsilon_i \approx Nc/k$ (large outliers)
- $N - k$ values with $\varepsilon_i \approx 0$

Under Pattern B:

- $\varepsilon_{\text{avg}} \approx c$
- $\text{MAD} \approx c(1 - 1/N)$ (large due to outliers)

Probability of Pattern A:

For a single noise term to fall in $(c - \delta, c + \delta)$:

$$P(|\varepsilon_i - c| < \delta) = \Phi\left(\frac{c + \delta}{\sigma}\right) - \Phi\left(\frac{c - \delta}{\sigma}\right) \quad (35)$$

Using Taylor expansion for small δ :

$$P(|\varepsilon_i - c| < \delta) \approx \phi\left(\frac{c}{\sigma}\right) \cdot \frac{2\delta}{\sigma} = \frac{2\delta}{\sigma\sqrt{2\pi}} \exp\left(-\frac{c^2}{2\sigma^2}\right) \quad (36)$$

For all N noise terms to satisfy this condition independently:

$$P(\text{Pattern A}) = \left[\frac{2\delta}{\sigma\sqrt{2\pi}} \exp\left(-\frac{c^2}{2\sigma^2}\right)\right]^N \quad (37)$$

Probability of $\varepsilon_{\text{avg}} > c$:

Since $\varepsilon_{\text{avg}} \sim N(0, \sigma^2/N)$:

$$P(\varepsilon_{\text{avg}} > c) = 1 - \Phi\left(\frac{c\sqrt{N}}{\sigma}\right) \quad (38)$$

Using Mill's ratio approximation for large arguments:

$$P(\varepsilon_{\text{avg}} > c) \approx \frac{\sigma}{c\sqrt{2\pi N}} \exp\left(-\frac{c^2 N}{2\sigma^2}\right) \quad (39)$$

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li.

Adaptthink: Reasoning models can learn when to think, 2025b. URL <https://arxiv.org/abs/2505.13417>.

Wenyuan Zhang, Shuaiyi Nie, Xinghua Zhang, Zefeng Zhang,

and Tingwen Liu. S1-bench: A simple benchmark for evaluating system 1 thinking capability of large reasoning models. ArXiv, abs/2504.10368, 2025c. URL <https://api.semanticscholar.org/CorpusID:277781494>.

Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang

Zhao, Shuohang Wang, Yelong Shen, and Xin Eric Wang.

Soft thinking: Unlocking the reasoning potential of llms in continuous concept space, 2025d. URL <https://arxiv.org/abs/2505.15778>.

Rongzhi Zhu, Yi Liu, Zequn Sun, Yiwei Wang, and Wei Hu.

When can large reasoning models save thinking? mechanistic analysis of behavioral divergence in reasoning, 2025. URL <https://arxiv.org/abs/2505.15276>.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu,

Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf,

Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. arXiv preprint arXiv:2406.15877, 2024.

The key insight is that Pattern A is the only pattern where MAD remains small enough to satisfy $\text{MAD} < (\varepsilon_{\text{avg}} - c)/\alpha$.

For Pattern B and other outlier-driven patterns, $\text{MAD} = O(c)$, while $\varepsilon_{\text{avg}} - c = O(1/\sqrt{N})$ when conditioned on $\varepsilon_{\text{avg}} \approx c$. Thus:

$$\text{MAD} \gg \frac{\varepsilon_{\text{avg}} - c}{\alpha} \quad (40)$$

Therefore, the improvement factor is dominated by Pattern A:

$$\rho \lesssim \frac{P(\text{Pattern A})}{P(\varepsilon_{\text{avg}} > c)} \quad (41)$$

Substituting the expressions:

$$\rho \lesssim \frac{\left[\frac{2\delta}{\sigma\sqrt{2\pi}} \exp\left(-\frac{c^2}{2\sigma^2}\right) \right]^N}{\frac{\sigma}{c\sqrt{2\pi N}} \exp\left(-\frac{c^2 N}{2\sigma^2}\right)} \quad (42)$$

Simplifying:

$$\rho \lesssim \frac{c\sqrt{N}}{\sigma} \left(\frac{2\delta}{\sigma\sqrt{2\pi}} \right)^N \exp\left(-\frac{c^2 N}{2\sigma^2} + \frac{c^2 N}{2\sigma^2}\right) \quad (43)$$

$$\rho \lesssim c\sqrt{N} \left(\frac{2\delta}{\sigma^2\sqrt{2\pi}} \right)^N \quad (44)$$

For $\delta = O(\sigma)$, let $\delta = k\sigma$ where k is a constant. Then:

$$\rho \lesssim c\sqrt{N} \left(\frac{2k}{\sqrt{2\pi}} \right)^N \quad (45)$$

When $k < \sqrt{\pi}/2$, the term $(2k/\sqrt{2\pi})^N$ decays exponentially. The polynomial factor $\sqrt{N}$ is dominated by the exponential decay, yielding:

$$\rho = O(\sqrt{N} \cdot \exp(-\beta N)) = O(\exp(-\Theta(N))) \quad (46)$$

for some positive constant β .

We have established that:

$$P_{\text{FP}}(\text{MAD}) = \rho \cdot P_{\text{FP}}(\text{Avg}) \quad (47)$$

where $\rho = O(\exp(-\Theta(N)))$ decays exponentially with the number of prompts N .

This demonstrates that the MAD-calibrated strategy provides an exponential improvement over the simple averaging approach. $\square$

Conclusion: The MAD penalty term effectively **filters out the more probable outlier patterns while only allowing the exponentially rare coherent patterns to trigger false positives**, thus achieving superior robustness against prompt-induced noise.

B.3 PROOF OF THE EXPECTED VALUE OF MAD

B.3.1 THEOREM 1

For a random variable $X \sim \mathcal{N}(\mu, \sigma^2)$, the expected value of the C_{MAD} is:

$$\mathbb{E}[C_{\text{MAD}}] = \sigma \sqrt{\frac{2}{\pi}} \approx 0.8\sigma \quad (48)$$

A Pilot Experiment Setup

我们选择 AIME2024 (MAA Committees) 作为探索性实验的测试集进行定性分析, 并进一步通过在 MATH-500 (Hendrycks et al., 2021)、GPQA-Diamond (Rein et al., 2023) 上的实验进行定量分析。所有实验均在 DeepSeek-R1-Distill-Qwen-14B (DeepSeek-AI et al., 2025) 上进行。在我们的实验中, 我们首先让 LRM 在测试集上执行完整的推理 (包括慢思考内容和结论内容)。然后, 我们保留思考内容, 并根据动作转换点将其划分为思考块。保留思考块数量超过五个的样本。对于这些样本, 我们保留了不同比例 (20%-90%) 的思考块, 并在每个截断的推理序列后附加一个思考结束标记分隔符, 以强制终止慢思考过程。模型随后基于部分推理内容生成其最终结论。对于在不同思考内容下获得的结论, 我们评估了其正确性, 并在图 1 中展示了每个样本的结果。此外, 我们在图 2 中研究了在三个数据集上, 原本正确的样本在提前退出后仍保持正确的数量, 以及原本不正确的样本在提前退出后变为正确的数量。

(7)

A.1 Noise Independence of the MAD-Calibrated Strategy

让我们将真实置信度 μ 定义为模型在给定当前推理内容下得出正确答案的实际概率, 对应于珍珠推理在此处存在的真实概率。提前退出决策阈值记为 λ , 当 $\mu > \lambda$ 时模型执行提前退出。由于答案诱导提示可能引入 ε , 模型的输出置信度围绕真实置信度波动, 产生一个观测置信度 $C_i = \mu + \varepsilon_i$ 。在实际测试环境中, 我们比较 C_i 与 λ 来决定是否执行提前退出。不失一般性, 我们假设噪声项 ε_i 是独立同分布的, 服从均值为 0、标准差为 σ 的高斯分布, 即 $\varepsilon_i \sim N(0, \sigma^2)$ 。这里, σ 表示模型对提示措辞的敏感性。更大的 σ 意味着更高的模型敏感性, 从而导致更大的置信度波动。

接下来, 我们通过比较 DEER-PRO (C_{cali})、DEER (C_i) 和一种平均方法 (C_{avg}) 在关键风险场景下的决策错误率, 来展示 DEER-PRO 对抗噪声干扰的有效性。假设 C_{cali} 和 C_{avg} 各自并行进行 N 次答案诱导, 且 N 与方程 (5) 中的相同。鉴于在推理场景中保持准确性优先于提前退出的加速收益, 我们将风险场景指定为真实置信度 $\mu < \lambda$ 的情况。然后我们计算由于噪声干扰而观测到 $C > \lambda$ 的假阳性概率。

A.1.1 Probability of Error for a Single Prompt:

$$P_{\text{FP}}(\text{Single}) = P(C_i > \lambda) = P(\mu + \varepsilon_i > \lambda) = P(\varepsilon_i > \lambda - \mu) \quad (8)$$

由于 $\varepsilon_i \sim N(0, \sigma^2)$, 我们可以将其标准化为:

$$P_{\text{FP}}(\text{Single}) = P\left(\frac{\varepsilon_i}{\sigma} > \frac{\lambda - \mu}{\sigma}\right) \quad (9)$$

令 $Z = \frac{\varepsilon_i}{\sigma}$, 则 $Z \sim N(0, 1)$, 于是:

$$P_{\text{FP}}(\text{Single}) = P\left(Z > \frac{\lambda - \mu}{\sigma}\right) = 1 - \Phi\left(\frac{\lambda - \mu}{\sigma}\right) \quad (10)$$

其中 Φ 是标准正态分布的累积分布函数。

B.3.2 PROOF

Definition. The Mean Absolute Deviation of a random variable X with mean μ is defined as:

$$C_{\text{MAD}} = \mathbb{E}[|X - \mu|] \quad (49)$$

Let $X \sim \mathcal{N}(\mu, \sigma^2)$. Consider the standardized random variable:

$$Z = \frac{X - \mu}{\sigma} \sim \mathcal{N}(0, 1) \quad (50)$$

Therefore:

$$|X - \mu| = \sigma|Z| \quad (51)$$

Taking expectations on both sides:

$$\mathbb{E}[|X - \mu|] = \sigma \cdot \mathbb{E}[|Z|] \quad (52)$$

For $Z \sim \mathcal{N}(0, 1)$ with probability density function $\phi(z) = \frac{1}{\sqrt{2\pi}}e^{-z^2/2}$:

$$\mathbb{E}[|Z|] = \int_{-\infty}^{\infty} |z| \cdot \phi(z) dz \quad (53)$$

Due to the symmetry of the standard normal distribution about zero and the even nature of $|z|$:

$$\mathbb{E}[|Z|] = 2 \int_0^{\infty} z \cdot \phi(z) dz = \frac{2}{\sqrt{2\pi}} \int_0^{\infty} z \cdot e^{-z^2/2} dz \quad (54)$$

Let $u = z^2/2$, then $du = z dz$. When $z = 0$, $u = 0$; when $z \rightarrow \infty$, $u \rightarrow \infty$.

$$\int_0^{\infty} z \cdot e^{-z^2/2} dz = \int_0^{\infty} e^{-u} du = [-e^{-u}]_0^{\infty} = 1 \quad (55)$$

Substituting back:

$$\mathbb{E}[|Z|] = \frac{2}{\sqrt{2\pi}} \cdot 1 = \sqrt{\frac{2}{\pi}} \quad (56)$$

Therefore:

$$C_{\text{MAD}} = \mathbb{E}[|X - \mu|] = \sigma \cdot \mathbb{E}[|Z|] = \sigma \sqrt{\frac{2}{\pi}} \approx 0.8\sigma \quad (57)$$

C MORE EXPERIMENT SETUP

Metrics. The goal of DEER is to maintain the correctness performance of LRMs while avoiding the redundant token overhead caused by overthinking. To this end, we selected *Accuracy* (ACC) and *Generation Length* (LEN) as the evaluation metrics. *Accuracy* (ACC) is calculated as follows: $\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}\{\mathcal{M}(\mathcal{LRM}(x_i)) = y_i\}$, where x_i is the question and y_i is the ground-truth answer from the dataset. $\mathcal{M}(\cdot)$ extracts the answer from the LRM's response. $\mathbb{I}\{\cdot\}$ is an indicator function that determines whether the inside given condition is valid. The accuracy evaluation is based on the evaluation framework publicly released by Ye et al. (2025) (LIMO). Intuitively, the longer the generated text, the greater the inference cost for LRMs. Therefore, we calculate the average generation tokens per sample to evaluate the cost as follows: $\text{Generation Length}(\text{LEN}) = \frac{1}{N} \sum_{i=1}^N |\mathcal{LRM}(x_i)|$, where $|\cdot|$ measures the number of generated tokens. For the two programming benchmarks, we use the Pass@1 metric to measure generated code correctness.

Implementation details. All evaluations are conducted in a Zero-shot Chain-of-Thought (CoT) setting with the following prompt: *"Please reason step by step, and put your final answer within \boxed{\cdot}."* For the decoding strategy, we employ greedy decoding with a single sample for the correctness evaluation. The ground-truth answers to the evaluation problems in our experiments are all well-structured numerical values or options. Therefore, we apply rule-based evaluations directly to verify mathematical equivalence. We set the maximum generation length at 16,384 to ensure that the evaluation captures complete problem-solving attempts. For DEER, the answer-inducing prompt employed is: *"\n\n Final Answer\n\n\boxed{\cdot}"*. For DEER-Pro, we additionally incorporated the following three prompts: *"\n\n Final Answer\n\n Based on the analysis above, the answer is \boxed{\cdot}"*, *"\n\n Final Answer\n\n The correct final answer is \boxed{\cdot}"*, *"\n\n Based on the previous thinking, I believe I already know the answer.\n\n Final Answer\n\n \boxed{\cdot}"*.

A.1.2 Probability of Error for Averaged Confidence:

对于平均置信度 C_{avg} , 噪声项仍然是高斯的, $\varepsilon_{\text{avg}} \sim N(0, \sigma^2/N)$ 。平均置信度的错误概率为:

$$P_{\text{FP}}(\text{Avg}) = P(C_{\text{avg}} > \lambda) = 1 - \Phi\left(\sqrt{N} \frac{(\lambda - \mu)}{\sigma}\right) \quad (11)$$

由于对于 $N > 1$ 有 $\frac{(\lambda - \mu)\sqrt{N}}{\sigma} > \frac{\lambda - \mu}{\sigma}$, 因此可得 $P_{\text{FP}}(\text{Avg}) < P_{\text{FP}}(\text{Single})$ 。这表明, 简单地平均多个观测到的置信度值可以减轻噪声干扰。然而, 置信度平均未能解决根本问题, 因为错误率 P_{FP} 仍然依赖于噪声标准差 σ , 并随着 σ 增大而单调增加。对于具有显著固有噪声 (大的 σ) 的模型, Φ 函数内部的参数趋近于零, 驱使 P_{FP} 趋向于 0.5。这表明, 无论决策阈值 λ 如何, 高噪声模型都会退化为随机猜测。因此, 传统方法的可靠性严重受限于模型固有的噪声水平 σ , 这是一个我们无法控制的因素。

A.1.3 Probability

of Error for MAD-Calibrated Confidence (DEER-Pro):

$$P_{\text{FP}}(\text{calibration}) = P(C_{\text{cali}} > \lambda) = P(C_{\text{avg}} - \alpha \cdot C_{\text{MAD}} > \lambda) \quad (12)$$

代入 $C_{\text{avg}} = \mu + \varepsilon_{\text{avg}}$, 我们得到:

$$P_{\text{FP}}(\text{calibration}) = P(\mu + \varepsilon_{\text{avg}} - \alpha \cdot C_{\text{MAD}} > \lambda) \quad (13)$$

重新排列方程中的项, 得到:

$$P_{\text{FP}}(\text{calibration}) = P(\varepsilon_{\text{avg}} > \alpha \cdot C_{\text{MAD}} + \lambda - \mu) \quad (14)$$

接下来, 我们将讨论 DEER-Pro 在两种不同场景下的鲁棒性。

Scenario 1: Approximate estimation of C_{MAD}

在我们的假设下, 其中 $\varepsilon_i \sim N(0, \sigma^2)$, 我们有 $\varepsilon_{\text{avg}} \sim N(0, \sigma^2/N)$ 和 $E[C_{\text{MAD}}] \approx 0.8\sigma$ (我们将在第 A.2 节提供证明)。对于大的 N , 大数定律允许我们将 C_{MAD} 近似为 0.8σ 。基于此假设, 我们有:

$$P_{\text{FP}}(\text{calibration}) = P(C_{\text{avg}} > \lambda + 0.8\sigma\alpha) \quad (15)$$

上述方程揭示了 DEER-Pro 的根本区别在于采用了随噪声缩放的自适应阈值:

$$\lambda_{\text{effective}} = \lambda + 0.8\sigma\alpha \quad (16)$$

我们继续通过将 $P_{\text{FP}}(\text{calibration})$ 转换为标准正态分布的累积分布函数 (CDF) Φ 来重新表述方程。对于方程 (14), 我们代入 $C_{\text{MAD}} = 0.8\sigma$ 并得到:

$$P_{\text{FP}}(\text{calibration}) = P(\varepsilon_{\text{avg}} > 0.8\sigma\alpha + \lambda - \mu) \quad (17)$$

由于 $\varepsilon_{\text{avg}} \sim N(0, \sigma^2/N)$:

$$P_{\text{FP}}(\text{calibration}) = 1 - \Phi\left(\frac{(\lambda - \mu + 0.8\sigma\alpha)\sqrt{N}}{\sigma}\right) \quad (18)$$

简化:

$$P_{\text{FP}}(\text{calibration}) = 1 - \Phi\left(\sqrt{N} \left(\frac{\lambda - \mu}{\sigma} + 0.8\alpha\right)\right) \quad (19)$$

Algorithm 1 Dynamic Early Exit in Reasoning (DEER)

```

1: Initialization: Large Reasoning Language Model  $\text{LRM}(\cdot)$ , zero-shot-CoT  $zs\_cot$ , question,
   answer inducer prompt  $I$ , set of action transition points  $\mathbb{P}$ , end-of-thinking delimiter  $\langle \text{think} \rangle$ ,
   maximum length  $max\_len$ , and confidence threshold  $\lambda$ .
2:  $x \leftarrow zs\_cot + \text{question}$ ,  $r \leftarrow []$ 
3: while  $len(x) < max\_len$  do
4:    $y \leftarrow \text{LRM}(x)$ 
5:   if  $y \in \mathbb{P}$  then                                ▷ Generate thoughts until meets action transition points
6:      $A \leftarrow \text{LRM}(x + I)$                         ▷ Prompt LRM to generate trial answer tokens
7:     Get  $C$  according to Equation 4                  ▷ Calculate the confidence of the trial answer
8:     if  $C > \lambda$  then
9:        $x \leftarrow x + \langle \text{think} \rangle$ ,  $r \leftarrow r + \langle \text{think} \rangle$   ▷ Exit when thinking is sufficient
10:    end if
11:  else
12:     $x \leftarrow x + y$ ,  $r \leftarrow r + y$ 
13:  end if
14: end while
15: return  $r$ 

```

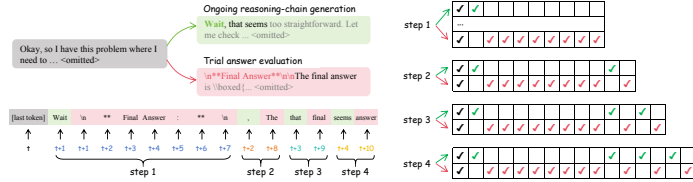


Figure 9: Branch-parallel decoding and dynamic KV cache management.

D MORE METHOD DETAILS

Fig. 9 illustrates the workflow of the proposed Branch-Parallel Decoding Acceleration. Algorithm 1 presents the pseudocode of DEER.

E MORE BENCHMARK DESCRIPTIONS.

Benchmarks. To thoroughly evaluate the models’ performance across various reasoning capabilities, we have chosen 6 math reasoning benchmarks, 1 science benchmarks, and 3 coding benchmarks as follows:

MATH BENCHMARKS:

- **GSM8K** is a well-curated collection of 1,319 problems in elementary mathematics. This benchmark is specifically designed to evaluate multi-step reasoning in foundational math tasks. Problems typically involve two to eight sequential operations, relying primarily on basic arithmetic performed over multiple intermediate steps.
- **MATH-500** is a challenging benchmark comprising competition-level problems drawn from diverse high school mathematics domains, including Prealgebra, Algebra, and Number Theory. For consistency with previous research, we adopt the same 500-problem subset originally curated by OpenAI for evaluation.
- **AMC 2023** contains 40 mathematical problems, covering algebra, geometry, number theory, and combinatorics. The American Mathematics Competitions (AMC), organized by the Mathematical Association of America (MAA), are prestigious contests designed to develop problem-solving skills and identify mathematical talent. For evaluation, we used 40 questions from AMC 23 in LIMO.

当噪声最小时:

$$\lambda_{\text{effective}} \rightarrow \lambda \quad (20)$$

校准方法的行为类似于标准阈值法, 保持高效率。

当噪声占主导时, $\frac{\lambda - \mu}{\sigma} \rightarrow 0$. 误报率变为:

$$P_{\text{FP}} = 1 - \Phi(0.8\alpha\sqrt{N}) \quad (21)$$

其独立于 σ 。这表明 DEER-PRO 有效地防止了提前退出退化为随机猜测 ($P_{\text{FP}} \rightarrow 0.5$)。

Scenario 2: Exact computation of C_{MAD} 无需近似。

从 $C_{\text{MAD}} = \frac{1}{N} \sum_{i=1}^N |C_i - C_{\text{avg}}|$, 我们知道 C_{MAD} 为正, 因此将此方程两边除以该项, 我们得到:

$$P(\varepsilon_{\text{avg}}/C_{\text{MAD}} > \alpha + (\lambda - \mu)/C_{\text{MAD}}) \quad (22)$$

由于 $\mu < \lambda$, 则 $\alpha + (\lambda - \mu)/C_{\text{MAD}} > \alpha$, 因此我们可以得到:

$$P(\varepsilon_{\text{avg}}/C_{\text{MAD}} > \alpha + (\lambda - \mu)/C_{\text{MAD}}) < P(\varepsilon_{\text{avg}}/C_{\text{MAD}} > \alpha) \quad (23)$$

因此, $P(\varepsilon_{\text{avg}}/C_{\text{MAD}} > \alpha)$ 是 $P_{\text{FP}}(\text{calibration})$ 的一个上界。对于比率 $\varepsilon_{\text{avg}}/C_{\text{MAD}}$, ε_{avg} 代表噪声的信号, 而 C_{MAD} 代表噪声的内部无序度。因此, 我们可以将 $\varepsilon_{\text{avg}}/C_{\text{MAD}}$ 定义为信噪比 (SNR)。接下来, 让我们分析信噪比 (SNR) 的性质。在我们的假设下, 当 $\varepsilon_i \sim N(0, \sigma^2)$ 时, 我们有 $\varepsilon_{\text{avg}} \sim N(0, \sigma^2/N)$ 和 $E[C_{\text{MAD}}] \approx 0.8\sigma$ 。由于 ε_{avg} 和 C_{MAD} 均与 σ 成比例, 我们可以将 SNR 写为:

$$\text{SNR} = (\sigma * Z_{\text{avg}})/(\sigma * Z_{\text{MAD}}) = Z_{\text{avg}}/Z_{\text{MAD}} \quad (24)$$

其中 $Z_{\text{avg}} \sim N(0, 1/N)$ 是一个标准化的噪声均值, 而 Z_{mad} 是一个与 MAD/σ 相关的随机变量, 其分布不依赖于 σ 。因此, SNR 的概率分布独立于模型噪声标准差 σ 。

$$P_{\text{FP}}(\text{calibration}) < P(Z_{\text{avg}}/Z_{\text{MAD}} > \alpha) \quad (25)$$

因此, $P_{\text{FP}}(\text{calibration})$ 仅受提示数量 N 和信噪比阈值 α 的影响, 其中更大的 N 和 α 值会导致更低的错误率。

通过从绝对阈值测试转变为自归一化的 SNR 测试, 我们的

MAD 策略有效地将决策过程与模型固有的、不可控的噪声

水平 σ 解耦。与传统方法在高噪声条件下失效不同, 我们的方法提供了与模型噪声水平无关的一致、稳健的性能。

$$E[|z_i|] = \sqrt{\frac{2}{\pi}} \approx 0.798, \quad \text{Var}(|z_i|) = 1 - \frac{2}{\pi} \approx 0.363 \quad (26)$$

$$\text{Var}(Z_{\text{mad}}) \approx \frac{0.363}{N} \quad (27)$$

$$\text{Var}(\text{SNR}) \approx \left(\frac{\partial f}{\partial X} \right)_E^2 \text{Var}(X) + \left(\frac{\partial f}{\partial Y} \right)_E^2 \text{Var}(Y) \quad (28)$$

$$\left. \frac{\partial f}{\partial X} \right|_E = \frac{1}{0.798} \approx 1.25, \quad \left. \frac{\partial f}{\partial Y} \right|_E = 0 \text{Var}(\text{SNR}) \approx (1.25)^2 \cdot \frac{1}{N} = \frac{1.5625}{N} \quad (29)$$

$$P(\text{SNR} > 1) \approx P\left(Z > \frac{\sqrt{N}}{1.25}\right) \quad (30)$$

- **AIME 2024** comprises 30 challenge problems selected from the 2024 American Invitational Mathematics Examination (AIME). This prestigious contest evaluates participants' mathematical reasoning abilities across diverse domains, including arithmetic, algebra, counting, geometry, number theory, probability, and other secondary school math topics. A distinctive feature of the AIME is its answer format: all solutions must be integers between 000 and 999 (inclusive). Each problem is categorized by difficulty level (1–5) according to the Art of Problem Solving (AoPS) scale. Beyond these three math problems, we also conducted evaluations on scientific questions.
- **AIME 2025** comprises 30 challenge problems selected from the 2025 American Invitational Mathematics Examination (AIME).
- **OlympiadBench** OlympiadBench is an Olympiad-level bilingual multimodal scientific benchmark dataset that aims to challenge and evaluate the advanced capabilities of Large Language Models and Large Multimodal Models. It features 8,476 problems sourced from mathematics and physics competitions at the Olympiad level, including those from the Chinese college entrance exam. Our experimental evaluation selects the same subset of 675 samples as used in LIMO, allowing for direct rule-based evaluation of the generated answers.

SCIENCE BENCHMARKS:

- **GPQA** is a PhD-level benchmark consisting of high-quality questions spanning physics, chemistry, and biology subdomains. Notably, domain experts with PhDs in these fields achieved only 69.7% accuracy on this dataset. For our experiments, we specifically select the highest quality subset, known as **GPQA Diamond** (composed of 198 questions).

PROGRAMMING BENCHMARKS:

- **HumanEval** is proposed by OpenAI, containing 164 hand-crafted (to avoid data leakage) Python programming tasks focusing on basic algorithms, each with function signatures, docstrings, canonical solutions, and unit tests.
- **BigCodeBench** is designed as a real-world-oriented benchmark, which includes 1,140 tasks requiring interactions with 139 libraries and diverse function calls.
- **LiveCodeBench** is a newly proposed benchmark dataset designed to evaluate the capabilities of large language models in code generation and related tasks. It aims to mitigate issues such as test set contamination found in existing benchmarks by emphasizing scenarios beyond code generation, ensuring high-quality problem sources, adequate test cases, and balanced difficulty levels. The dataset comprises problems sourced from well-known competitive programming platforms like AtCoder, LeetCode, and CodeForces, collected from specific time windows. Our evaluation is based on **LiveCodeBench-v5**, which contains 880 programming problems collected from May 2023 to January 2025.

F COMPUTATION SOURCE

In our experiments, $8 \times 80\text{g}$ memory H100 was used to perform evaluations.

G MORE LRM DESCRIPTIONS.

In this work, we validate the effectiveness of DEER across 12 reasoning models. The evaluated models include: Qwen3-1.7B, Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-32B, DeepSeek-R1-Distill-Qwen-1.5B, DeepSeek-R1-Distill-Qwen-7B, DeepSeek-R1-Distill-Qwen-14B, DeepSeek-R1-Distill-Qwen-32B, QwQ-32B, DeepSeek-R1-671B, and Llama-3.1-Nemotron-Nano-8B-v1. All models in the DeepSeek-R1-Distill-Series were supervised fine-tuned using reasoning data generated by the DeepSeek-R1 model. The Qwen3-1.7B, Qwen3-4B, Qwen3-8B, and Qwen3-14B models were trained using a method known as Strong-to-Weak Distillation. Trained via reinforcement learning, the non-distilled models QwQ-32B and Qwen3-32B demonstrate competitive performance on reasoning benchmarks, matching that of DeepSeek-R1-671B. Due to computational constraints, we implemented a quantized version of Deepseek-R1 based on KTransformers (kvcache ai, 2025).

在本节中, 我们基于事件空间正式证明, MAD 策略的误报概率 $P_{\text{FP}}(\text{MAD})$ ($P_{\text{FP}}(\text{calibration})$) 显著优于简单平均策略 $P_{\text{FP}}(\text{Avg})$ 的误报概率, 因此也优于 $P_{\text{FP}}(\text{Single})$ 。

Theorem. MAD 校准策略的误报概率满足 $P_{\text{FP}}(\text{MAD}) \leq \rho \cdot P_{\text{FP}}(\text{Avg})$, 其中 $\rho = O(\exp(-\Theta(N)))$ 是关于提示数量 N 的指数衰减因子。

Proof. 误报事件定义如下:

$$E_{\text{Avg}} = \{\varepsilon : \varepsilon_{\text{avg}} > c\} \quad (31)$$

$$E_{\text{MAD}} = \{\varepsilon : \varepsilon_{\text{avg}} - \alpha \cdot \text{MAD} > c\} \quad (32)$$

其中 $c = \lambda - \mu > 0$ 是阈值间隙, $\text{MAD} = \frac{1}{N} \sum_{i=1}^N |\varepsilon_i - \varepsilon_{\text{avg}}|$ 是平均绝对偏差, $\varepsilon_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N \varepsilon_i$ 是平均噪声, 并且存在 N 个独立的噪声项 $\varepsilon_i \sim N(0, \sigma^2)$, 其中 $i = 1, \dots, N$ 。

由于根据定义有 $\alpha > 0$ 和 $\text{MAD} \geq 0$, 我们得到:

$$\varepsilon_{\text{avg}} - \alpha \cdot \text{MAD} \leq \varepsilon_{\text{avg}} \quad (33)$$

因此, 如果 $\varepsilon_{\text{avg}} - \alpha \cdot \text{MAD} > c$, 则必然有 $\varepsilon_{\text{avg}} > c$ 。这确立了:

$$E_{\text{MAD}} \subseteq E_{\text{Avg}} \quad (34)$$

因此:

$$P_{\text{FP}}(\text{MAD}) = P(E_{\text{MAD}}) \leq P(E_{\text{Avg}}) = P_{\text{FP}}(\text{Avg}) \quad (35)$$

为了量化超越这一基本不等式的改进程度, 我们使用条件概率分解该概率:

$$P_{\text{FP}}(\text{MAD}) = P(E_{\text{MAD}} \cap E_{\text{Avg}}) = P(E_{\text{MAD}} | E_{\text{Avg}}) \cdot P(E_{\text{Avg}}) \quad (36)$$

由于 $E_{\text{MAD}} \subseteq E_{\text{Avg}}$, 我们有:

$$P(E_{\text{MAD}} | E_{\text{Avg}}) = P(\varepsilon_{\text{avg}} - \alpha \cdot \text{MAD} > c | \varepsilon_{\text{avg}} > c) \quad (37)$$

这可以重写为:

$$P(E_{\text{MAD}} | E_{\text{Avg}}) = P\left(\text{MAD} < \frac{\varepsilon_{\text{avg}} - c}{\alpha} \mid \varepsilon_{\text{avg}} > c\right) \quad (38)$$

定义改进因子:

$$\rho = P\left(\text{MAD} < \frac{\varepsilon_{\text{avg}} - c}{\alpha} \mid \varepsilon_{\text{avg}} > c\right) \quad (39)$$

则有:

$$P_{\text{FP}}(\text{MAD}) = \rho \cdot P_{\text{FP}}(\text{Avg}) \quad (40)$$

为了评估 ρ , 我们分析满足 $\varepsilon_{\text{avg}} > c$ 的噪声向量结构。主要有两种模式:

Pattern A (Coherent Pattern): 所有噪声项都接近 c 。形式化地, 对于某个小的 $\delta > 0$:

$$\text{Pattern A} = \{\varepsilon : |\varepsilon_i - c| < \delta \text{ for all } i = 1, \dots, N\} \quad (41)$$

在模式 A 下:

- $\varepsilon_{\text{avg}} \approx c + O(\delta/\sqrt{N})$ (根据中心极限定理)
- $\text{MAD} \leq 2\delta$ (因为所有值都在彼此 2δ 范围内)

Pattern B (Outlier Pattern): 少数几个大的离群值, 其余值接近零。例如:

H COMPUTATIONAL COST ANALYSIS

In this section, we provide a theoretical analysis to demonstrate that DEER effectively reduces computational costs. Let L denote the total length generated by the original CoT method, and α represent DEER's compression ratio relative to L , such that DEER generates a sequence of length αL . Then, we define k as the number of answer induction triggers within these αL tokens of reasoning and m as the average length generated per answer induction, which is typically a small constant. During transformer inference, the primary computational overhead stems from attention calculations, which constitutes our main focus. Assuming the generation process employs key-value caching technology, each new token only needs to compute attention with the cached key-value pairs.

H.1 COMPUTATIONAL COST ANALYSIS ON TIME

For the original CoT method, the computational cost is:

$$T = O(1) + O(2) + \dots + O(L) = O(L^2) \quad (58)$$

For our DEER, The computational cost comprises two components: αL forward passes in the main reasoning chain and km forward passes for answer inducing.

First, we calculate the cost of the main reasoning chain:

$$T_{\text{main}} = \sum_{t=1}^{\alpha L} t = \frac{\alpha L(\alpha L + 1)}{2} = O(\alpha^2 L^2) \quad (59)$$

Next, for the computational overhead during answer inducing, we first calculate the time cost of a single inducing. Suppose the j -th answer inducing is triggered at position p_j , yielding a cost of:

$$C_{\text{single}} = \sum_{i=1}^m (p_j + i - 1) = m \cdot p_j + \sum_{i=1}^m (i - 1) = m \cdot p_j + \frac{m(m-1)}{2} \quad (60)$$

Assuming the inducing positions p_j are uniformly distributed over the interval $[0, \alpha L]$, the average inducing position is $E[p_j] \approx \frac{\alpha L}{2}$. Hence, the total cost of answer inducing is:

$$C_{\text{induce}} \approx k \cdot m \cdot \frac{\alpha L}{2} + k \cdot \frac{m(m-1)}{2} = O(k \cdot m \cdot \alpha L) + O(k \cdot m^2) \quad (61)$$

Even in the worst-case scenario where most trigger points p_j cluster near the end of reasoning, the average inducing position is $E[p_j] \approx \alpha L$. The total cost of answer inducing is:

$$C_{\text{induce}} \approx k \cdot m \cdot \alpha L + k \cdot \frac{m(m-1)}{2} = O(k \cdot m \cdot \alpha L) + O(k \cdot m^2) \quad (62)$$

As the length of each answer inducing m is negligible compared to the reasoning length L , we have:

$$C_{\text{induce}} \approx O(k \cdot m \cdot \alpha L) \quad (63)$$

Finally, the total cost of DEER is:

$$C_{\text{DEER}} = C_{\text{main}} + C_{\text{induce}} = O(\alpha^2 L^2) + O(k \cdot m \cdot \alpha L) \quad (64)$$

DEER reduces the quadratic term from $O(L^2)$ to $O(\alpha^2 L^2)$ while only introducing a linear term $O(k \cdot m \cdot \alpha L)$. Since $k, m \ll L$ in long chain-of-thought reasoning, the savings from the quadratic term reduction far exceed the overhead of the additional linear term. This analysis effectively explains the superlinear speedup phenomenon observed in Section 4.4.

H.2 COMPUTATIONAL COST ANALYSIS ON MEMORY

The memory overhead analysis can be decomposed into two components: primary memory consumption from the KV cache and additional overhead from parallel decoding operations.

- k 个值满足 $\varepsilon_i \approx Nc/k$ (大的离群值)
- $N - k$ 个值满足 $\varepsilon_i \approx 0$

在模式 B 下:

- $\varepsilon_{\text{avg}} \approx c$
- $\text{MAD} \approx c(1 - 1/N)$ (由于离群值而很大)

Probability of Pattern A: 对于一个单独的噪声项落入 $(c - \delta, c + \delta)$ 的情况:

$$P(|\varepsilon_i - c| < \delta) = \Phi\left(\frac{c + \delta}{\sigma}\right) - \Phi\left(\frac{c - \delta}{\sigma}\right) \quad (42)$$

对小 δ 使用泰勒展开:

$$P(|\varepsilon_i - c| < \delta) \approx \phi\left(\frac{c}{\sigma}\right) \cdot \frac{2\delta}{\sigma} = \frac{2\delta}{\sigma\sqrt{2\pi}} \exp\left(-\frac{c^2}{2\sigma^2}\right) \quad (43)$$

对于所有 N 个噪声项独立满足此条件:

$$P(\text{Pattern A}) = \left[\frac{2\delta}{\sigma\sqrt{2\pi}} \exp\left(-\frac{c^2}{2\sigma^2}\right) \right]^N \quad (44)$$

Probability of $\varepsilon_{\text{avg}} > c$:

由于 $\varepsilon_{\text{avg}} \sim N(0, \sigma^2/N)$:

$$P(\varepsilon_{\text{avg}} > c) = 1 - \Phi\left(\frac{c\sqrt{N}}{\sigma}\right) \quad (45)$$

对大参数使用米尔比率近似:

$$P(\varepsilon_{\text{avg}} > c) \approx \frac{\sigma}{c\sqrt{2\pi N}} \exp\left(-\frac{c^2 N}{2\sigma^2}\right) \quad (46)$$

关键见解在于, 模式 A 是唯一能使 MAD 保持足够小以满足 $\text{MAD} < (\varepsilon_{\text{avg}} - c)/\alpha$ 的模式。

对于模式 B 和其他由离群值驱动的模式, $\text{MAD} = O(c)$, 而在给定 $\varepsilon_{\text{avg}} \approx c$ 的条件下 $\varepsilon_{\text{avg}} - c = O(1/\sqrt{N})$ 。因此:

$$\text{MAD} \gg \frac{\varepsilon_{\text{avg}} - c}{\alpha} \quad (47)$$

因此, 改进因子主要由模式 A 主导:

$$\rho \lesssim \frac{P(\text{Pattern A})}{P(\varepsilon_{\text{avg}} > c)} \quad (48)$$

代入表达式:

$$\rho \lesssim \frac{\left[\frac{2\delta}{\sigma\sqrt{2\pi}} \exp\left(-\frac{c^2}{2\sigma^2}\right) \right]^N}{\frac{\sigma}{c\sqrt{2\pi N}} \exp\left(-\frac{c^2 N}{2\sigma^2}\right)} \quad (49)$$

简化:

$$\rho \lesssim \frac{c\sqrt{N}}{\sigma} \left(\frac{2\delta}{\sigma\sqrt{2\pi}} \right)^N \exp\left(-\frac{c^2 N}{2\sigma^2} + \frac{c^2 N}{2\sigma^2}\right) \quad (50)$$

$$\rho \lesssim c\sqrt{N} \left(\frac{2\delta}{\sigma^2\sqrt{2\pi}} \right)^N \quad (51)$$

Peak Memory Reduction. The dominant memory overhead in modern LLM inference stems from the storage of attention keys and values in the KV cache, whose size scales linearly with the processed sequence length. Standard Chain-of-Thought (CoT) approaches necessitate maintaining KV cache for all L tokens, resulting in memory complexity of $O(L)$. Through early termination at position αL where $\alpha < 1$, DEER effectively reduces the peak sequence length during inference from L to αL . Consequently, the peak KV cache memory consumption is reduced from $O(L)$ to $O(\alpha L)$.

This memory reduction proves particularly valuable when processing long-context reasoning tasks. Beyond reducing peak memory requirements for individual requests, this approach enables systems to accommodate increased concurrent requests under identical memory constraints during batch processing, thereby enhancing overall throughput.

Additional Overhead for Parallel Decoding. The proposed parallel decoding variant, which performs answer induction forward passes concurrently with main reasoning, introduces minimal additional memory overhead. This efficiency is achieved through prefix caching and sharing mechanisms implemented in modern inference frameworks such as vLLM. When multiple reasoning branches share a common prefix sequence, the corresponding portions of their KV caches require only single storage in physical memory through technologies such as vLLM’s PagedAttention.

During parallel decoding, the answer induction branch incurs virtually no additional KV cache overhead, as it fully leverages the KV cache already computed by the main reasoning branch. The only marginal additional memory requirement arises from storing a limited number of tokens representing the answer induction branch’s decoding state.

For code generation tasks, our implementation incorporates specific optimizations whereby only the initial 50 tokens are generated for confidence estimation. This design choice represents an implementation detail rather than a core methodological contribution. Experimental validation confirms that utilizing partial answer tokens for early-exit confidence calculation remains effective for coding tasks.

I INVESTIGATION OF REASONING TRANSITION MONITORS

In Section 4.3 of the main text, our experiments reveal that the choice of Reasoning Transition Monitor exerts subtle effects on DEER, primarily manifested in how the number of potential early-exit opportunities affects the final generation length. In this section, we investigate the underlying connections between linguistic marker-based and entropy-based monitoring approaches.

Table 10 presents a comparative analysis of average token entropy between linguistic markers and other tokens across multiple datasets and models. Our findings reveal that linguistic markers exhibit significantly higher entropy compared to other tokens, suggesting that the linguistic marker-based approach inherently targets high-entropy positions where multiple candidate actions exist.

Additionally, we compute cosine similarity scores between consecutive tokens in the final layer’s hidden state representations, comparing linguistic markers with their adjacent tokens against regular token pairs. The similarity metric serves as an indicator of the model’s reasoning coherence: high similarity reflects continuous, coherent reasoning processes, whereas low similarity signals the occurrence of reasoning transitions. The results presented in Table 11 demonstrate that linguistic markers exhibit substantially lower similarity scores, indicating disruptions in representational continuity.

Collectively, these experiments provide compelling evidence that large reasoning language models do not undergo uncertain states silently; instead, they explicitly express uncertainty through language. The external linguistic markers leveraged by DEER constitute direct manifestations of internal state transitions, thereby providing strong empirical support for the theoretical foundations of our approach.

对于 $\delta = O(\sigma)$, 令 $\delta = k\sigma$, 其中 k 为常数。则:

$$\rho \lesssim c\sqrt{N} \left(\frac{2k}{\sqrt{2\pi}} \right)^N \quad (52)$$

当 $k < \sqrt{\pi/2}$ 时, 项 $(2k/\sqrt{2\pi})^N$ 呈指数衰减。多项式因子 $\sqrt{N}$ 被指数衰减所主导, 从而得到:

$$\rho = O(\sqrt{N} \cdot \exp(-\beta N)) = O(\exp(-\Theta(N))) \quad (53)$$

对于某个正常数 β 。

我们已经建立:

$$P_{\text{FP}}(\text{MAD}) = \rho \cdot P_{\text{FP}}(\text{Avg}) \quad (54)$$

其中 $\rho = O(\exp(-\Theta(N)))$ 随提示数量 N 呈指数衰减。

这证明了 MAD 校准策略相对于简单平均方法提供了指数级的改进。□

结论: MAD 惩罚项有效地 filters out the more probable outlier patterns while only allowing the exponentially rare coherent patterns to trigger false positives, 从而实现了提示诱导噪声的卓越鲁棒性。

A.2 Proof of the Expected Value of MAD

A.2.1 Theorem 1

对于一个随机变量 $X \sim \mathcal{N}(\mu, \sigma^2)$, 其 C_{MAD} 的期望值为:

$$\mathbb{E}[C_{\text{MAD}}] = \sigma \sqrt{\frac{2}{\pi}} \approx 0.8\sigma \quad (55)$$

A.2.2 Proof

Definition. 随机变量 X 的均值绝对偏差, 其均值为 μ , 定义为:

$$C_{\text{MAD}} = \mathbb{E}[|X - \mu|] \quad (56)$$

令 $X \sim \mathcal{N}(\mu, \sigma^2)$ 。考虑标准化随机变量:

$$Z = \frac{X - \mu}{\sigma} \sim \mathcal{N}(0, 1) \quad (57)$$

因此:

$$|X - \mu| = \sigma|Z| \quad (58)$$

对两边取期望:

$$\mathbb{E}[|X - \mu|] = \sigma \cdot \mathbb{E}[|Z|] \quad (59)$$

对于具有概率密度函数 $\phi(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}$ 的 $Z \sim \mathcal{N}(0, 1)$:

$$\mathbb{E}[|Z|] = \int_{-\infty}^{\infty} |z| \cdot \phi(z) dz \quad (60)$$

由于标准正态分布关于零对称以及 $|z|$ 的偶函数性质:

$$\mathbb{E}[|Z|] = 2 \int_0^{\infty} z \cdot \phi(z) dz = \frac{2}{\sqrt{2\pi}} \int_0^{\infty} z \cdot e^{-z^2/2} dz \quad (61)$$

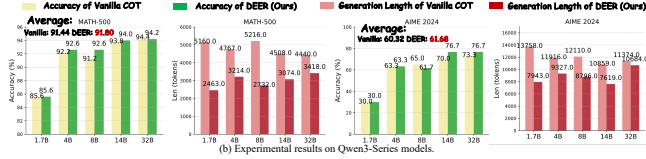


Figure 10: Experimental results of DEER compared to Vanilla CoT across Qwen3-Series models of varying sizes on MATH-500 and AIME 2024.

	AMC	OlympiadBench	AIME24
Qwen3-32B	5 34 0	30 441 27	4 19 3
Qwen3-14B	1 37 1	54 401 21	4 19 2
Qwen3-8B	2 36 0	52 369 31	2 17 2
RL-Distill-7B	5 28 2	69 286 33	3 10 1

Figure 11: More detailed experimental results of DEER compared to Vanilla CoT. $\checkmark$ denotes a correct answer, and $\times$ denotes an incorrect answer.

J INVESTIGATION INTO THE REASONS BEHIND DEER’S THRESHOLD ROBUSTNESS

In Section 4.3 of the main text, we demonstrate DEER’s robustness to the threshold λ through experiments across various models and datasets. In this section, we investigate the underlying source of this robustness. We analyze the confidence scores of induced answers at all potential exit positions across three models on three mathematical reasoning datasets, calculating the proportion of scores falling within three distinct intervals. Specifically, 0–0.9 represents the low-confidence interval, 0.97–1.0 represents the high-confidence interval, and 0.9–0.97 constitutes the error-prone gray zone.

The results presented in Table 12 reveal that the model’s confidence distribution exhibits a pronounced polarization phenomenon. The vast majority of cases concentrate at either the highly confident or insufficiently confident extremes, with minimal presence in the intermediate range (the error-prone gray zone). When our method induces the model to generate final answers, the confidence scores follow a distinctive U-shaped distribution, with remarkably low probability mass between 0.9 and 0.97. This phenomenon indicates that when the model possesses sufficient certainty about an answer based on its preceding reasoning chain, it generates the answer with exceptionally high probability (typically exceeding 0.99). Conversely, when uncertainty exists, the assigned probability drops substantially.

Furthermore, we observe that all three models exhibit higher proportions of high-confidence scores compared to low-confidence scores on simpler problems (GSM8K). While on more challenging problems (AIME24), the proportion of low-confidence scores exceeds that of high-confidence scores. This observation further validates the rationality of the DEER method: the confidence assigned to trial answers accurately reflects whether the existing reasoning is sufficient to solve the problem. Consequently, confidence scores are generally lower on difficult problems, leading to many failed early-exit attempts in the initial stages. This pattern also explains DEER’s varying performance across different problem difficulties. On simpler problems, the model demonstrates sufficient confidence, resulting in better compression effects. On challenging problems, the model becomes more cautious, yielding weaker compression but maintaining satisfactory accuracy. This adaptive behavior shows that DEER naturally balances computational efficiency and solution quality based on problem complexity.

令 $u = z^2/2$, 则 $du = z dz$ 。当 $z = 0$ 时, $u = 0$; 当 $z \rightarrow \infty$ 时, $u \rightarrow \infty$ 。

$$\int_0^\infty z \cdot e^{-z^2/2} dz = \int_0^\infty e^{-u} du = [-e^{-u}]_0^\infty = 1 \quad (62)$$

代回:

$$\mathbb{E}[|Z|] = \frac{2}{\sqrt{2\pi}} \cdot 1 = \sqrt{\frac{2}{\pi}} \quad (63)$$

因此:

$$C_{\text{MAD}} = \mathbb{E}[|X - \mu|] = \sigma \cdot \mathbb{E}[|Z|] = \sigma \sqrt{\frac{2}{\pi}} \approx 0.8\sigma \quad (64)$$

Algorithm 1 Dynamic Early Exit in Reasoning (DEER)

```

1: Initialization: Large Reasoning Language Model LRM( $\cdot$ ), zero-shot-CoT  $zs\_cot$ ,
   question, answer inducer prompt  $I$ , set of action transition points  $\mathbb{P}$ , end-of-thinking
   delimiter  $\langle /think \rangle$ , maximum length  $max\_len$ , and confidence threshold  $\lambda$ .
2:  $\mathbf{x} \leftarrow zs\_cot + \text{question}$ ,  $\mathbf{r} \leftarrow []$ 
3: while  $len(\mathbf{x}) < max\_len$  do
4:    $\mathbf{y} \leftarrow \text{LRM}(\mathbf{x})$ 
5:   if  $\mathbf{y} \in \mathbb{P}$  then ▷ Generate thoughts until meets action transition points
6:      $\mathbf{A} \leftarrow \text{LRM}(\mathbf{x} + I)$  ▷ Prompt LRM to generate trial answer tokens
7:     Get  $\mathcal{C}$  according to Equation 5 ▷ Calculate the confidence of the trial answer
8:     if  $\mathcal{C} > \lambda$  then
9:        $\mathbf{x} \leftarrow \mathbf{x} + \langle /think \rangle$ ,  $\mathbf{r} \leftarrow \mathbf{r} + \langle /think \rangle$  ▷ Exit when thinking is sufficient
10:    end if
11:  else
12:     $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{y}$ ,  $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{y}$ 
13:  end if
14: end while
15: return  $\mathbf{r}$ 

```

B More Experiment Setup

Metrics. DEER 的目标是在保持 LRM 正确性能的同时, 避免因过度思考 (overthinking) 导致的冗余令牌开销。为此, 我们选择 Accuracy (ACC) 和 Generation Length (LEN) 作为评估指标。Accuracy (ACC) 的计算方式如下: $\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}\{\mathcal{M}(\text{LRM}(x_i)) = y_i\}$, 其中 x_i 是问题, y_i 是数据集中的真实答案。 $\mathcal{M}(\cdot)$ 从 LRM 的响应中提取答案。 $\mathbb{I}\{\cdot\}$ 是一个指示函数, 用于判断内部给定条件是否有效。准确性评估基于 Ye et al. (2025) (LIMO) 公开发布的评估框架。直观上, 生成的文本越长, LRM 的推理成本就越高。因此, 我们计算每个样本的平均生成令牌数来评估成本, 如下所示: $\text{Generation Length}(\text{LEN}) = \frac{1}{N} \sum_{i=1}^N |\text{LRM}(x_i)|$, 其中 $|\cdot|$ 测量生成的令牌数量。对于两个编程基准测试, 我们使用 Pass@1 指标来衡量生成代码的正确性。

Implementation details. 所有评估均在零样本思维链 (CoT) 设置下进行, 使用以下提示词: "Please reason step by step, and put your final answer within `\boxed{\cdot}`." 对于解码策略, 我们在正确性评估中采用贪婪解码和单一样本。我们实验中评估问题的真实答案都是结构良好的数值或选项。因此, 我们直接应用基于规则的评估来验证数学等价性。我们将最大生成长度设置为 16,384,

Table 2: Comparison of **Vanilla**, **DEER**, and **DEER-Pro** across multiple models and datasets. Acc = accuracy (%), Len = average tokens, CR = compression ratio.

Method	GSM8K			MATH			AMC			AIME			GPQA			Overall	
	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	CR
DeepSeek-R1-Distill-Qwen-1.5B																	
Vanilla	76.1	1,617	100%	69.0	6,018	100%	52.5	8,819	100%	23.3	13,702	100%	7.1	13,029	100%	45.6	100%
DEER	74.7	984	60.9%	67.8	2,497	41.5%	60.0	5,496	62.3%	23.3	9,557	69.7%	12.1	5,762	44.2%	47.6	55.7%
DEER-Pro	77.3	1,062	65.7%	70.0	2,891	48.0%	62.5	5,701	64.6%	26.7	10,390	75.8%	14.5	6,820	52.3%	50.2	61.3%
Qwen3-4B																	
Vanilla	94.1	2,175	100%	92.2	4,767	100%	87.5	7,443	100%	63.3	11,916	100%	46.5	9,294	100%	76.7	100%
DEER	94.5	1,250	57.5%	92.6	3,214	67.4%	87.5	4,906	65.9%	63.3	9,327	78.3%	47.5	3,275	35.2%	77.1	60.9%
DEER-Pro	94.5	1,301	59.8%	93.0	3,517	73.8%	92.5	5,153	69.2%	65.0	9,651	81.0%	49.2	3,750	40.3%	78.8	64.8%
Qwen3-1.7B																	
Vanilla	90.1	2,045	100%	85.6	5,160	100%	70.0	8,637	100%	30.0	13,758	100%	35.9	9,271	100%	62.3	100%
DEER	90.3	1,066	52.1%	85.6	2,463	47.7%	70.0	4,673	54.1%	30.0	7,943	57.7%	43.4	3,549	38.3%	63.9	50.0%
DEER-Pro	90.7	1,261	61.7%	87.2	2,702	52.4%	75.0	5,143	59.5%	35.0	8,644	62.8%	44.5	3,960	42.7%	66.5	55.8%

Table 3: Experimental results on programming tasks. Acc = accuracy (%), Tok. = average tokens, CR = compression ratio.

Model	Method	HumanEval			BigCodeBench			LiveCodeBench			Overall	
		Acc	Tok.	CR	Acc	Tok.	CR	Acc	Tok.	CR	Acc	CR
R1-Distill-Qwen Series												
32B	Vanilla	91.5	3,861	100%	44.5	5,459	100%	56.0	9,109	100%	64.0	100%
	DEER	93.9	1,254	32.5%	46.1	1,929	35.3%	56.6	3,677	40.4%	65.5	36.1%
14B	Vanilla	89.0	4,039	100%	40.9	4,806	100%	52.7	9,259	100%	60.9	100%
	DEER	90.9	1,000	24.8%	40.1	1,583	32.9%	52.1	4,000	43.2%	61.2	33.6%
7B	Vanilla	78.6	5,666	100%	26.1	8,516	100%	38.4	10,482	100%	47.7	100%
	DEER	78.6	913	16.1%	25.2	1,605	18.8%	40.3	2,582	24.6%	48.0	19.9%
Qwen3 Series												
14B	Vanilla	93.3	3,277	100%	44.6	5,072	100%	73.4	8,203	100%	70.4	100%
	DEER	93.9	1,118	34.1%	44.3	792	15.6%	74.1	5,437	66.3%	70.8	38.7%
8B	Vanilla	85.4	3,904	100%	37.9	6,994	100%	64.7	8,871	100%	62.7	100%
	DEER	87.8	793	20.3%	41.1	608	8.7%	65.1	4,257	48.0%	64.7	25.7%
4B	Vanilla	91.5	3,768	100%	36.1	7,804	100%	64.7	8,789	100%	64.1	100%
	DEER	92.7	1,050	27.9%	37.1	826	10.6%	63.5	5,626	64.0%	64.4	34.2%
1.7B	Vanilla	83.5	3,580	100%	25.7	6,151	100%	51.7	9,447	100%	53.6	100%
	DEER	84.1	2,236	62.5%	28.3	2,104	34.2%	51.4	8,425	89.2%	54.6	62.0%

K MORE EXPERIMENTAL RESULTS

More experiments across Model Sizes on Qwen3. The performance of the Qwen3-series models across model sizes and reasoning difficulty in Fig. 10 is consistent with the findings presented in Section 4.2.

Performance on SoTA Reasoning Models.

We evaluated DEER’s effectiveness on two state-of-the-art reasoning models: Qwen3-32B (representing dense models) and Deepseek-R1 671B (representing MoE models). To fully leverage their reasoning capabilities, we set their maximum sequence lengths to the officially recommended 32k and 16k, respectively. The impact of max length will be further discussed in the next section. Due to computational constraints, we implemented a quantized version of Deepseek-R1 based on KTransformers (kvcache ai, 2025). Fig. 12 provides a close look at DEER’s performance on two challenging datasets, AIME and MATH.

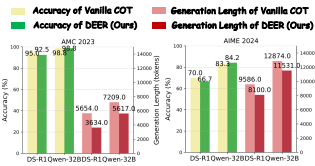


Figure 12: Performance on SoTA models.

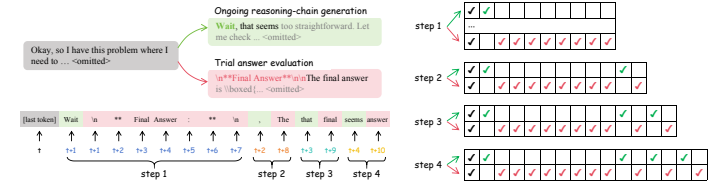


图 9: 分支并行解码与动态 KV 缓存管理。

以确保评估能捕捉到完整的问题解决尝试。对于 DEER，使用的答案诱导提示词是：'\n\n Final Answer\nboxed' 对于 DEER-Pro，我们额外加入了以下三个提示词：'\n\n Final Answer\n\n Based on the analysis above, the answer is \boxed'、'\n\n Final Answer\n\n The correct final answer is \boxed'、'\n\n Based on the previous thinking, I believe I already know the answer.\n Final Answer\n \boxed'。

C More Method Details

图 9 展示了所提出的分支并行解码加速的工作流程。算法 1 展示了 DEER 的伪代码。

D More Benchmark Descriptions.

Benchmarks. 为了全面评估模型在各种推理能力上的表现，我们选择了 6 个数学推理基准、1 个科学基准和 3 个编码基准，具体如下：

Math Benchmarks:

- GSM8K 是一个精心策划的集合，包含 1,319 个初等数学问题。该基准专门用于评估基础数学任务中的多步推理能力。问题通常涉及两到八个连续操作，主要依赖于在多个中间步骤中执行的基本算术。
- MATH-500 是一个具有挑战性的基准，包含来自不同高中数学领域（包括预代数、代数和数论）的竞赛级问题。为了与先前研究保持一致，我们采用由 OpenAI 最初策划用于评估的相同 500 个问题子集。
- AMC 2023 包含 40 个数学问题，涵盖代数、几何、数论和组合数学。美国数学竞赛（AMC）由美国数学协会（MAA）组织，是一项旨在培养解决问题能力和识别数学人才的著名竞赛。在评估中，我们使用了 LIMO 中来自 AMC 23 的 40 个问题。
- AIME 2024 包含从 2024 年美国数学邀请赛（AIME）中选出的 30 个挑战性问题。这项著名的竞赛评估参与者在多个领域的数学推理能力，包括算术、代数、计数、几何、数论、概率以及其他中学数学主题。AIME 的一个显著特点是其答案格式：所有解必须是 000 到 999（含）之间的整数。每个问题根据 Art of Problem Solving（AoPS）的难度等级（1-5）进行分类。除了这三个数学问题，我们还对科学问题进行了评估。

Table 4: Additional threshold sensitivity experiments across more models and tasks.

Qwen3-14B	GSM8K	MATH	AMC
Vanilla	95.1	93.8	95.0
0.80	96.0	93.8	93.8
0.85	95.7	94.4	93.8
0.90	96.1	93.8	95.0
0.95	96.0	94.0	95.6
0.97	95.7	93.8	94.4
Qwen3-8B	GSM8K	MATH	AMC
Vanilla	94.9	91.2	87.5
0.80	95.2	91.4	88.8
0.85	94.9	92.0	90.0
0.90	95.5	92.8	91.3
0.95	95.3	93.2	92.5
0.97	95.3	93.0	92.5

The results show that DEER maintains competitive accuracy (with R1 making only one additional error on each dataset) while significantly reducing sequence length by 10.4% - 35.7%.

Performance Trends across More Model Sizes and Benchmarks. To provide a more comprehensive demonstration of DEER’s effectiveness and facilitate comparison for researchers, we present experimental results on seven reasoning benchmarks and eleven large reasoning language models. Fig. 13 compares the experimental results between DEER and vanilla CoT, demonstrating that the conclusions drawn in the main text of the paper hold consistently across more benchmarks and additional models.

In addition to popular reasoning models, we also evaluate DEER on less commonly used models. As mentioned in the Limitations section, Llama-3.1-Nemotron-Nano-8B-v1 consistently exhibits low confidence in generating intermediate answers, resulting in a significantly lower early stopping rate during evaluation compared to mainstream models (Qwen3-8B: 80%, R1-Distill-Qwen-7B: 85%, Llama-3.1-Nemotron-Nano-8B-v1: 55%). Consequently, as shown in Table 5, the improvement in reasoning efficiency for Llama-3.1-Nemotron-Nano-8B-v1 is limited. Nevertheless, DEER still effectively mitigates overthinking in this model, as evidenced by its ability to prevent subsequent reasoning steps from altering correct answers into incorrect ones through early stopping.

Performance with Different Decoding Configurations. As the configuration of DeepSeek-R1-Distill-Series models recommends a maximum length of 16k (16,384), we evaluate Qwen3-14B under the same setting in the main experiments to maintain setup consistency. In practice, this length is sufficient for most real-world applications. In addition, to ensure experimental stability and reproducibility, we employ greedy decoding in the main experiments. Nevertheless, to provide a more comprehensive assessment of DEER’s performance, we further conduct experiments on larger variants of the Qwen3-series models (8B, 14B, 32B) using the officially recommended decoding strategy (max_len = 32768, top_p = 0.95, temperature = 0.6). Fig. 8 shows that DEER remains significantly effective under these configurations, demonstrating the robustness of our approach.

Performance with Different Multi-Token Confidence Averaging Methods. In the main text, we mentioned adopting the geometric mean strategy for calculating multi-token answer confidence scores, as it better aligns with the multiplicative nature of joint probability computation in language models and exhibits higher sensitivity to low probability values. In this section, we supplement our analysis with comparative experiments using arithmetic mean calculation (DEER-AM), employing the same early-exit threshold of 0.95. The results in Table 7 demonstrate that DEER-AM exhibits a significant decrease in accuracy compared to DEER-GM, while achieving marginal improvements in compression ratio. This indicates that the arithmetic mean dilutes the contribution of low-valued tokens, resulting in overall inflated confidence scores and consequently leading to premature incorrect exits. Therefore, we recommend using the geometric mean for estimating true confidence scores.

- AIME 2025 包含从 2025 年美国数学邀请赛（AIME）中选出的 30 个挑战性问题。
- OlympiadBench OlympiadBench 是一个奥林匹克级别的双语多模态科学基准数据集，旨在挑战和评估大型语言模型和大型多模态模型的高级能力。它包含 8,476 个源自奥林匹克级别数学和物理竞赛的问题，包括中国高考中的问题。我们的实验评估选择了与 LIMO 中使用的相同的 675 个样本子集，允许对生成的答案进行直接的基于规则的评估。

Science Benchmarks:

- GPQA 是一个博士级别的基准，包含跨越物理、化学和生物学子领域的高质量问题。值得注意的是，在这些领域拥有博士学位的领域专家在该数据集上仅达到 69.7% 的准确率。在我们的实验中，我们特别选择了最高质量的子集，即 GPQA Diamond（由 198 个问题组成）。

Programming Benchmarks:

- HumanEval 由 OpenAI 提出，包含 164 个手工制作的（为避免数据泄露）专注于基础算法的 Python 编程任务，每个任务都包含函数签名、文档字符串、规范解决方案和单元测试。
- BigCodeBench 被设计为一个面向现实世界的基准测试，包含 1,140 个任务，需要与 139 个库和多样化的函数调用进行交互。
- LiveCodeBench 是一个新提出的基准数据集，旨在评估大语言模型在代码生成及相关任务上的能力。它旨在缓解现有基准中存在的测试集污染等问题，强调代码生成之外的场景，确保高质量的问题来源、充足的测试用例以及平衡的难度级别。该数据集包含从 AtCoder、LeetCode 和 CodeForces 等知名竞赛编程平台收集的问题，这些问题的收集时间窗口是特定的。我们的评估基于 LiveCodeBench-v5，它包含从 2023 年 5 月到 2025 年 1 月收集的 880 个编程问题。

在我们的实验中， $8 \times 80\text{g}$ 内存的 H100 被用于执行评估。

E More LRM Descriptions.

在这项工作中，我们验证了 DEER 在 12 个推理模型上的有效性。评估的模型包括：Qwen3-1.7B、Qwen3-4B、Qwen3-8B、Qwen3-14B、Qwen3-32B、DeepSeek-R1-Distill-Qwen-1.5B、DeepSeek-R1-Distill-Qwen-7B、DeepSeek-R1-Distill-Qwen-14B、DeepSeek-R1-Distill-Qwen-32B、QwQ-32B、DeepSeek-R1-671B 以及 Llama-3.1-Nemotron-Nano-8B-v1。DeepSeek-R1-Distill 系列中的所有模型均使用 DeepSeek-R1 模型生成的推理数据进行监督微调。Qwen3-1.7B、Qwen3-4B、Qwen3-8B 和 Qwen3-14B 模型采用一种称为“强到弱蒸馏”的方法进行训练。通过强化学习训练的非蒸馏模型 QwQ-32B 和 Qwen3-32B 在推理基准测试中表现出具有竞争力的性能，与 DeepSeek-R1-671B 相当。由于计算资源限制，我们基于 KTransformers (kvcache ai, 2025) 实现了 Deepseek-R1 的量化版本。

Table 5: Results on **MATH-500** (DeepSeek-R1-Distill-Qwen-14B) with different reasoning transition monitors. DEER(W) denotes transition via *Wait*, DEER(A) via *Alternatively*, and DEER(Ent) via entropy threshold. Chunk Size denotes the length (token numbers) of one reasoning chunk (thought), and Chunk Num denotes the number of reasoning chunks.

Method	Accuracy	Tokens	Chunk Size	Chunk Num	Exit Ratio	Exit Acc.
Vanilla	88.6	3815	—	—	—	—
DEER(W)	89.6 ^{+1.0}	2572 ^{-32.6%}	259.5	14.7	87.6%	93.4%
DEER(A)	90.8 ^{+2.2}	2775 ^{-27.3%}	719.8	5.3	54.8%	91.2%
DEER(Ent)	90.2 ^{+1.6}	2339 ^{-38.7%}	183.7	20.8	90.2%	93.0%

Table 6: Comparison of **Vanilla**, **DEER-W**, and **DEER-Ent** across multiple models and datasets. Acc = accuracy (%), Len = average tokens, CR = compression ratio.

Method	GSM8K			MATH			AMC			AIME			GPQA			Overall	
	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	CR
DeepSeek-R1-Distill-Qwen-7B																	
Vanilla	89.6	1,484	100%	87.4	3,858	100%	78.8	6,792	100%	41.7	13,765	100%	23.7	10,247	100%	64.2	100%
DEER-W	90.6	917	61.8%	89.8	2,143	55.5%	85.0	4,451	65.5%	49.2	9,839	71.5%	31.3	5,469	53.4%	69.2	61.5%
DEER-Ent	90.8	876	59.0%	89.2	2,261	58.6%	85.0	4,072	60.0%	48.4	8,961	65.1%	29.6	5,037	49.2%	68.6	58.4%
Qwen3-14B																	
Vanilla	95.1	2,047	100%	93.8	4,508	100%	95.0	7,139	100%	70.0	10,859	100%	60.1	7,339	100%	82.8	100%
DEER-W	95.3	840	41.0%	94.0	3,074	68.2%	95.0	4,763	66.7%	76.7	7,619	70.2%	57.6	2,898	39.5%	83.7	57.1%
DEER-Ent	96.1	803	39.2%	93.8	2,979	66.1%	93.3	4,903	68.7%	73.3	7,128	65.6%	58.1	2,818	38.4%	82.9	55.6%
Qwen3-8B																	
Vanilla	94.9	2,245	100%	91.2	5,216	100%	87.5	7,986	100%	65.0	12,110	100%	51.5	9,145	100%	78.0	100%
DEER-W	95.2	1,071	47.7%	92.6	2,732	52.4%	92.5	4,392	55.0%	61.7	8,796	72.6%	52.5	3,111	34.0%	78.9	52.3%
DEER-Ent	95.8	1,037	46.2%	93.6	2,789	53.5%	91.3	4,003	50.1%	63.3	8,328	68.8%	51.5	3,248	35.5%	79.1	50.8%

Error Bars with 95% Confidence Intervals. To demonstrate the statistical significance of DEER’s accuracy gains, we conducted multiple experimental runs on two models and calculated error bars with 95% confidence intervals. Specifically, we performed four independent runs on GSM8K, MATH, and GPQA benchmarks. Given the limited sample sizes of AMC23 and AIME24, we increased the number of experimental repetitions to eight for these datasets. The results presented in Table 9 confirm that the accuracy improvements achieved by our method are statistically significant.

L CASE STUDY DETAILS

In Fig. 13, we provide examples of results on MATH-500 to visually demonstrate the effectiveness of DEER. The design of DEER ensures that it follows the same reasoning process as the vanilla CoT method before early exiting. Both methods arrive at the correct answer during the first reasoning step, as shown in the green box. The difference lies in the fact that our method exits early after evaluating the confidence of the trial answer as sufficiently high, thus producing the correct result. In contrast, the vanilla CoT method proceeds to the next reasoning action. After double-checking and switching reasoning approaches, the model becomes trapped in an endless cycle of verification due to inconsistent answers from the two approaches, ultimately failing to provide a final answer. In addition, Fig. 14, 15, 16 provides additional generated examples to more comprehensively demonstrate the effectiveness of DEER’s early-exit mechanism and illustrate the underlying mechanisms of the approach.

Figure 17 shows the detailed process of DEER applied on a mathematical example. It can be observed that, at each reasoning switch point (“*Wait*” token), DEER generates a trial answer and evaluates its confidence. The change in confidence is consistent with the reliability of the current reasoning chunks and trial answers. This shows that LRMs implicitly know when to leave early, and our method is simple and effective to realize such potential of the model itself.

F Computational Cost Analysis

在本节中，我们提供一个理论分析来证明 DEER 能有效降低计算成本。令 L 表示原始 CoT 方法生成的总长度， α 代表 DEER 相对于 L 的压缩比，使得 DEER 生成长度为 αL 的序列。然后，我们定义 k 为这 αL 个推理标记内的答案诱导触发次数， m 为每次答案诱导的平均生成长度，这通常是一个较小的常数。在 Transformer 推理过程中，主要的计算开销源于注意力计算，这是我们的主要关注点。假设生成过程采用了键值缓存技术，每个新标记只需与缓存的键值对计算注意力。

F.1 Computational Cost Analysis on Time

对于原始的 CoT 方法，其计算成本为：

$$T = O(1) + O(2) + \dots + O(L) = O(L^2) \quad (65)$$

对于我们的 DEER 方法，计算成本包含两个部分：主推理链中的 αL 次前向传递以及答案诱导所需的 km 次前向传递。

首先，我们计算主推理链的成本：

$$T_{\text{main}} = \sum_{t=1}^{\alpha L} t = \frac{\alpha L(\alpha L + 1)}{2} = O(\alpha^2 L^2) \quad (66)$$

接下来，对于答案诱导期间的计算开销，我们首先计算单次诱导的时间成本。假设第 j 次答案诱导在位置 p_j 处被触发，其成本为：

$$C_{\text{single}} = \sum_{i=1}^m (p_j + i - 1) = m \cdot p_j + \sum_{i=1}^m (i - 1) = m \cdot p_j + \frac{m(m-1)}{2} \quad (67)$$

假设诱导位置 p_j 在区间 $[0, \alpha L]$ 上均匀分布，则平均诱导位置为 $E[p_j] \approx \frac{\alpha L}{2}$ 。因此，答案诱导的总成本为：

$$C_{\text{induce}} \approx k \cdot m \cdot \frac{\alpha L}{2} + k \cdot \frac{m(m-1)}{2} = O(k \cdot m \cdot \alpha L) + O(k \cdot m^2) \quad (68)$$

即使在最坏情况下，即大多数触发点 p_j 聚集在推理过程末尾附近，平均诱导位置为 $E[p_j] \approx \alpha L$ 。此时答案诱导的总成本为：

$$C_{\text{induce}} \approx k \cdot m \cdot \alpha L + k \cdot \frac{m(m-1)}{2} = O(k \cdot m \cdot \alpha L) + O(k \cdot m^2) \quad (69)$$

由于每次答案诱导的长度 m 与推理长度 L 相比可以忽略不计，我们有：

$$C_{\text{induce}} \approx O(k \cdot m \cdot \alpha L) \quad (70)$$

最后，DEER 的总成本为：

$$C_{\text{DEER}} = C_{\text{main}} + C_{\text{induce}} = O(\alpha^2 L^2) + O(k \cdot m \cdot \alpha L) \quad (71)$$

DEER 将二次项从 $O(L^2)$ 减少到 $O(\alpha^2 L^2)$ ，同时仅引入了一个线性项 $O(k \cdot m \cdot \alpha L)$ 。由于在长链式推理中 $k, m \ll L$ ，二次项减少所带来的节省远超过额外线性项的开销。这一分析有效地解释了第 4.4 节中观察到的超线性加速现象。

F.2 Computational Cost Analysis on Memory

内存开销分析可以分解为两个组成部分：来自 KV 缓存的主要内存消耗，以及来自并行解码操作的额外开销。

Table 7: Comparison of **Vanilla**, **DEER-GM** (Geometric Mean), and **DEER-AM** (Arithmetic Mean) across multiple models and datasets. Acc = accuracy (%), Len = average tokens, CR = compression ratio.

	GSM8K			MATH			AMC			AIME			GPQA			Overall	
Method	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	CR
DeepSeek-R1-Distill-Qwen-7B																	
Vanilla	89.6	1,484	100%	87.4	3,858	100%	78.8	6,792	100%	41.7	13,765	100%	23.7	10,247	100%	64.2	100%
DEER-GM	90.6	917	61.8%	89.8	2,143	55.5%	85.0	4,451	65.5%	49.2	9,839	71.5%	31.3	5,469	53.4%	69.2	61.5%
DEER-AM	90.2	832	56.1%	88.2	1,879	48.7%	80.0	3,872	57.0%	43.3	8,095	58.8%	22.6	4,116	40.2%	64.9	52.2%
Qwen3-1.4B																	
Vanilla	95.1	2,047	100%	93.8	4,508	100%	95.0	7,139	100%	70.0	10,859	100%	60.1	7,339	100%	82.8	100%
DEER-GM	95.3	840	41.0%	94.0	3,074	68.2%	95.0	4,763	66.7%	76.7	7,619	70.2%	57.6	2,898	39.5%	83.7	57.1%
DEER-AM	95.3	811	39.6%	92.4	2,620	58.1%	90.0	4,513	63.2%	63.3	6,933	63.8%	53.6	2,508	34.2%	78.9	51.8%
Qwen3-8B																	
Vanilla	94.9	2,245	100%	91.2	5,216	100%	87.5	7,986	100%	65.0	12,110	100%	51.5	9,145	100%	78.0	100%
DEER-GM	95.2	1,071	47.7%	92.6	2,732	52.4%	92.5	4,392	55.0%	61.7	8,796	72.6%	52.5	3,111	34.0%	78.9	52.3%
DEER-AM	94.9	972	43.3%	92.0	2,522	48.4%	87.5	3,899	48.8%	56.7	7,697	63.6%	49.7	2,950	32.3%	76.2	47.3%

M RELATED WORK DETAILS

The advent of OpenAI o1 (OpenAI, 2025) established test-time scaling (Snell et al., 2024) as a pivotal research direction in the LLM community. This approach enhances LLMs’ slow thinking capabilities, enabling breakthroughs in complex problem solving. The recent open-sourcing of DeepSeek-R1 (DeepSeek-AI et al., 2025) has further intensified interest in locally deployed reasoning models. However, two critical challenges have emerged: 1) excessively long CoT generated significantly degrades inference efficiency, and 2) growing empirical evidence (Chen et al., 2025b; Team et al., 2025a) reveals their susceptibility to overthinking – a phenomenon where models continue reasoning beyond the point of optimal output. Zhang et al. (2025c) introduces a novel benchmark named S1-Bench to test the performance of LRMs on simple tasks, evaluating the overthinking issues of these LRMs. Following the taxonomy of efficient reasoning established in (Sui et al., 2025; Wang et al., 2025a), we categorize related work into three classes: post-training based, prompt-based, and output-based efficient reasoning methods.

Post-training based efficient reasoning methods use supervised fine-tuning (Yu et al., 2024; Kang et al., 2025; Xia et al., 2025; Ma et al., 2025b; Munkhbat et al., 2025; Zhu et al., 2025; Liu et al., 2024; Han et al., 2024; Qiao et al., 2025; Yu et al., 2025) with variable-length CoT data or incorporate length rewards (Team et al., 2025b; Luo et al., 2025a; Aggarwal & Welleck, 2025; Arora & Zanette, 2025; Yeo et al., 2025; Shen et al., 2025b; Qu et al., 2025; Cui et al., 2025; Dai et al., 2025; Liu et al., 2025a; Tu et al., 2025; Wang et al., 2025c; Dumitru et al., 2025; Li et al., 2025a; Jiang et al., 2025a; Zhang et al., 2025b) in reinforcement learning to enable the model to adaptively generate chains of thought of different lengths. However, these methods often require a large amount of computational resources and face challenges in dataset construction. Recently, some work (Hao et al., 2024; Shen et al., 2025c; Cheng & Van Durme, 2024; Dang et al., 2025; Shen et al., 2025a; Su et al., 2025; Tan et al., 2025; Saunshi et al., 2025; Zhang et al., 2025d) has shown that using latent representations to replace explicit textual reasoning steps allows reasoning models to be more efficient. However, such methods often require extensive-epoch SFT on carefully curated datasets (Hao et al., 2024; Xu et al., 2025d), leading to overfitting on the output format and consequently compromising the model’s inherent expressiveness and generalization ability.

Prompt-based efficient reasoning methods (Han et al., 2024; Xu et al., 2025b; Lee et al., 2025; Renze & Guven, 2024; Chen et al., 2024) use varying prompts to enforce reasoning models to generate concise CoT with less unnecessary reasoning steps. Especially, (Aytes et al., 2025; Chuang et al., 2024; 2025; Ong et al.) assign different prompts to queries based on their difficulty, thereby adjusting the length of the CoT generated by reasoning models. We also explored the performance of our method combined with prompt design in Tab. 1, demonstrating further reductions in the length of reasoning chains while maintaining considerable accuracy.

Most of the **Output-based efficient reasoning** methods focus on optimizing the best-of-N sampling for LLMs, such as pruning low-quality samples (Xie et al., 2023; Liao et al., 2025) and implementing early stopping (Li et al., 2024; Manvi et al., 2024; Aggarwal et al., 2023) when multiple samples

Peak Memory Reduction. 现代 LLM 推理中的主要内存开销源于注意力键和值在 KV 缓存中的存储，其大小与处理的序列长度成线性比例。标准的思维链方法需要为所有 L 个令牌维护 KV 缓存，导致内存复杂度为 $O(L)$ 。通过在位置 αL 处提前终止（其中 $\alpha < 1$ ），DEER 有效地将推理期间的峰值序列长度从 L 减少到 αL 。因此，峰值 KV 缓存内存消耗从 $O(L)$ 减少到 $O(\alpha L)$ 。

这种内存减少在处理长上下文推理任务时尤其有价值。除了降低单个请求的峰值内存需求外，这种方法使系统能够在批处理期间，在相同的内存约束下容纳更多的并发请求，从而提高整体吞吐量。

Additional Overhead for Parallel Decoding. 所提出的并行解码变体，在执行答案归纳前向传播的同时进行主要推理，引入了极少的额外内存开销。这种效率是通过在现代推理框架（如 vLLM）中实现的前缀缓存和共享机制实现的。当多个推理分支共享一个共同的前缀序列时，它们 KV 缓存的相应部分通过诸如 vLLM 的 PagedAttention 等技术，在物理内存中仅需单次存储。

在并行解码期间，答案归纳分支几乎不会产生额外的 KV 缓存开销，因为它完全利用了主推理分支已经计算好的 KV 缓存。唯一的边际额外内存需求来自于存储代表答案归纳分支解码状态的有限数量的令牌。

对于代码生成任务，我们的实现包含了特定的优化，即仅生成前 50 个令牌用于置信度估计。这一设计选择是一个实现细节，而非核心方法学贡献。实验验证证实，利用部分答案令牌进行提前退出的置信度计算对于编码任务仍然有效。

G Investigation of Reasoning Transition Monitors

在正文的 ?? 节中，我们的实验表明，推理转换监控器的选择对 DEER 产生微妙的影响，主要体现在潜在提前退出机会的数量如何影响最终生成长度。在本节中，我们研究基于语言标记和基于熵的监控方法之间的潜在联系。

表 10 展示了在多个数据集和模型上，语言标记与其他标记之间平均令牌熵的对比分析。我们的发现表明，语言标记的熵显著高于其他标记，这表明基于语言标记的方法本质上针对的是存在多个候选动作的高熵位置。

此外，我们计算了最终层隐藏状态表示中连续令牌之间的余弦相似度得分，将语言标记与其相邻令牌的相似度与常规令牌进行比较。该相似度指标作为模型推理连贯性的指示器：高相似度反映连续、连贯的推理过程，而低相似度则标志着推理转换的发生。表 11 中呈现的结果表明，语言标记的相似度得分显著更低，这表明表征连续性出现了中断。

总的来说，这些实验提供了有力的证据，表明大型推理语言模型并非在静默中经历不确定状态；相反，它们通过语言明确地表达不确定性。DEER 所利用的外部语言标记构成了内部状态转换的直接表现，从而为我们的方法理论基础提供了强有力的实证支持。

Table 8: Experimental results on the Qwen3-series models under the officially recommended settings (max_len = 32768, top_p = 0.95, temperature = 0.6).

Budget	Method	GSM8K		MATH-500		AMC23		AIME24		AIME25		OlympiadB		Overall	
		Acc	Tok.	Acc	Tok.	Acc	Tok.	Acc	Tok.	Acc	Tok.	Acc	Tok.	Acc	CR
Qwen3-8B	Vanilla	95.7	2246	93.8	5368	93.8	9424	70.0	16717	65.0	17880	67.9	11025	81.0	100%
	DEER	95.5	981	94.0	3227	95.0	5898	75.8	12465	63.3	15135	67.0	9075	81.8	68.0%
Qwen3-14B	Vanilla	95.7	1699	94.8	4800	95.0	6837	75.0	14347	76.7	16437	68.7	9992	84.3	100%
	DEER	95.8	933	95.0	3301	96.3	6299	74.2	10896	76.7	15014	68.9	8263	84.5	77.6%
Qwen3-32B	Vanilla	96.0	1714	95.8	4609	98.8	7209	83.3	12874	78.3	15292	69.3	9775	86.9	100%
	DEER	95.8	992	95.4	3325	98.8	5617	84.2	11531	78.3	13981	69.8	8671	87.1	79.6%

Table 9: Accuracy performance on reasoning benchmarks with 95% confidence intervals.

Model	GSM8K	MATH	AMC23	AIME24	GPQA
Vanilla (ds-7B)	0.897 [0.891, 0.902]	0.877 [0.869, 0.884]	0.794 [0.767, 0.821]	0.425 [0.400, 0.449]	0.247 [0.203, 0.291]
DEER (ds-7B)	0.904 [0.896, 0.912]	0.897 [0.883, 0.911]	0.856 [0.835, 0.878]	0.492 [0.463, 0.520]	0.299 [0.257, 0.341]
Vanilla (Qwen3-14B)	0.948 [0.942, 0.955]	0.938 [0.932, 0.943]	0.938 [0.918, 0.957]	0.708 [0.660, 0.757]	0.596 [0.571, 0.621]
DEER (Qwen3-14B)	0.955 [0.949, 0.962]	0.942 [0.936, 0.948]	0.953 [0.940, 0.967]	0.754 [0.718, 0.791]	0.587 [0.566, 0.608]

achieve self-consistency. However, following the introduction of advanced reasoning models like R1, there is less reliance on best-of-N sampling methods, as these models exhibit strong reasoning capabilities independently. Very recently, two concurrent works share similar motivations with ours. Zhang et al. (2025a) also proposes to terminate early based on trial answers, but requires an additional probe model to determine the correctness. They focus on enhancing the verification capabilities of the probe model, whereas our method explore how to enable the model to self-determine when to exit early and integrate seamlessly into existing reasoning logic. Ma et al. (2025a) prompts reasoning models to directly output final answers during decoding, but only achieves better performance in the low-budget regime or being adapted to best-of-N methods compared to baselines, which limits the applicability and generalization. Song et al. (2025) periodically compresses the KV cache by retaining KV cache that receive high importance score to accelerates inference by leveraging the semantic sparsity of reasoning paths. Jiang et al. (2025b) uses a teacher model to perform skill-aware step decomposition and content pruning, and then distills the pruned reasoning paths into a student model. Huang et al. (2025) projects the steering direction onto the low-dimensional activation manifold and intervenes the activations to reduce thinking tokens.

N USE OF LLMs

In the preparation of this manuscript, Large Language Models (LLMs) were employed as auxiliary tools for Language Polishing. During the final stages of manuscript preparation, LLMs were utilized to refine the language of selected passages, including grammar checking, sentence structure optimization, and expression standardization. This process was limited to linguistic improvements and did not involve the generation or modification of any substantive academic content, including research insights, data analysis, or conclusion derivation. It should be emphasized that all core arguments, research methodologies, experimental designs, data analyses, and conclusions presented in this paper were independently developed by the authors. LLMs served solely as language processing aids, and the authors assume full academic responsibility for all content.

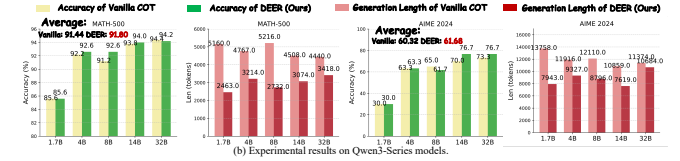


图 10: DEER 与 Vanilla CoT 在不同规模的 Qwen3 系列模型上, 在 MATH-500 和 AIME 2024 数据集上的实验结果对比。

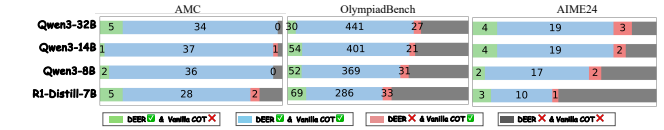


图 11: DEER 与 Vanilla CoT 更详细的实验结果对比。✓ 表示正确答案, ✗ 表示错误答案。

H Investigation into the

Reasons Behind DEER's Threshold Robustness

在正文第??节中, 我们通过在模型和数据集上的实验, 证明了 DEER 对阈值 λ 的鲁棒性。在本节中, 我们探究这种鲁棒性的根本来源。我们分析了三个模型在三个数学推理数据集上所有潜在退出位置处诱导答案的置信度分数, 计算了分数落在三个不同区间的比例。具体来说, 0-0.9 代表低置信度区间, 0.97-1.0 代表高置信度区间, 而 0.9-0.97 则构成了易出错的灰色地带。

表12中呈现的结果表明, 模型的置信度分布呈现出明显的极化现象。绝大多数情况集中在高度自信或信心不足的两个极端, 中间范围(易出错的灰色地带)的存在极少。当我们的方法诱导模型生成最终答案时, 置信度分数遵循独特的 U 形分布, 在 0.9 到 0.97 之间的概率质量极低。这一现象表明, 当模型基于其先前的推理链对某个答案拥有足够把握时, 它会以极高的概率(通常超过 0.99)生成该答案。反之, 当存在不确定性时, 分配的概率则会大幅下降。

此外, 我们观察到, 在较简单的问题(GSM8K)上, 所有三个模型的高置信度分数比例都高于低置信度分数比例; 而在更具挑战性的问题(AIME24)上, 低置信度分数的比例则超过了高置信度分数。这一观察进一步验证了 DEER 方法的合理性: 分配给试验答案的置信度准确地反映了现有推理是否足以解决问题。因此, 在困难问题上置信度通常较低, 导致在初始阶段出现许多失败的早期退出尝试。这种模式也解释了 DEER 在不同问题难度上表现各异的原因。在较简单的问题上, 模型表现出足够的信心, 从而获得更好的压缩效果。在具有挑战性的问题上, 模型变得更加谨慎, 导致压缩效果较弱, 但仍能保持令人满意的准确性。这种自适应行为表明, DEER 能够根据问题复杂度, 自然地平衡计算效率和求解质量。

More experiments across Model Sizes on Qwen3. 图 10 中 Qwen3 系列模型在不同模型规模和推理难度上的表现, 与第 4.2 节中提出的发现是一致的。

Table 10: Token Entropy for *Linguistic Markers* and *Other Tokens*.

Qwen3-8B	Linguistic Markers	Other Tokens
gsm8k	0.901	0.438
math	1.058	0.385
gpqa	1.269	0.500
DS-7B	Linguistic Markers	Other Tokens
gsm8k	1.550	0.658
math	1.753	0.565
gpqa	1.241	0.510

Table 11: Hidden States Cosine Similarity between *Linguistic Markers* and *Other Tokens*.

Qwen3-8B	Linguistic Markers	Other Tokens
gsm8k	0.262	0.543
math	0.237	0.493
gpqa	0.240	0.509
DS-7B	Linguistic Markers	Other Tokens
gsm8k	0.306	0.608
math	0.247	0.530
gpqa	0.231	0.505

Table 12: Confidence interval distribution (%) across tasks for different models.

Qwen3-14B	0-0.9	0.9-0.97	0.97-1.0
gsm8k	38.92	5.95	55.06
math	49.53	4.83	45.46
aime	77.20	2.45	20.35
Qwen3-8B	0-0.9	0.9-0.97	0.97-1.0
gsm8k	35.31	5.34	59.12
math	45.27	4.58	49.98
aime	78.61	2.39	19.00
DS-7B	0-0.9	0.9-0.97	0.97-1.0
gsm8k	26.54	5.29	68.17
math	34.09	5.46	60.45
aime	80.90	1.41	17.69

表 2: 比较多种模型和数据集上的 Vanilla、DEER 和 DEER-PRo。Acc = 准确率 (%), Len = 平均词元数, CR = 压缩比。

	GSM8K			MATH			AMC			AIME			GPQA			Overall	
Method	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	CR
DeepSeek-R1-Distill-Qwen-1.5B																	
Vanilla	76.1	1,617	100%	69.0	6,018	100%	52.5	8,819	100%	23.3	13,702	100%	7.1	13,029	100%	45.6	100%
DEER	74.7	984	60.9%	67.8	2,497	41.5%	60.0	5,496	62.3%	23.3	9,557	69.7%	12.1	5,762	44.2%	47.6	55.7%
DEER-PRo	77.3	1,062	65.7%	70.0	2,891	48.0%	62.5	5,701	64.6%	26.7	10,390	75.8%	14.5	6,820	52.3%	50.2	61.3%
Qwen3-4B																	
Vanilla	94.1	2,175	100%	92.2	4,767	100%	87.5	7,443	100%	63.3	11,916	100%	46.5	9,294	100%	76.7	100%
DEER	94.5	1,250	57.5%	92.6	3,214	67.4%	87.5	4,906	65.9%	63.3	9,327	78.3%	47.5	3,275	35.2%	77.1	60.9%
DEER-PRo	94.5	1,301	59.8%	93.0	3,517	73.8%	92.5	5,153	69.2%	65.0	9,651	81.0%	49.2	3,750	40.3%	78.8	64.8%
Qwen3-1.7B																	
Vanilla	90.1	2,045	100%	85.6	5,160	100%	70.0	8,637	100%	30.0	13,758	100%	35.9	9,271	100%	62.3	100%
DEER	90.3	1,066	52.1%	85.6	2,463	47.7%	70.0	4,673	54.1%	30.0	7,943	57.7%	43.4	3,549	38.3%	63.9	50.0%
DEER-PRo	90.7	1,261	61.7%	87.2	2,702	52.4%	75.0	5,143	59.5%	35.0	8,644	62.8%	44.5	3,960	42.7%	66.5	55.8%

表 3: 编程任务上的实验结果。Acc = 准确率 (%), Tok. = 平均令牌数, CR = 压缩率。

Model	Method	HumanEval			BigCodeBench			LiveCodeBench			Overall	
		Acc	Tok.	CR	Acc	Tok.	CR	Acc	Tok.	CR	Acc	CR
R1-Distill-Qwen Series												
32B	Vanilla	91.5	3,861	100%	44.5	5,459	100%	56.0	9,109	100%	64.0	100%
	DEER	93.9	1,254	32.5%	46.1	1,929	35.3%	56.6	3,677	40.4%	65.5	36.1%
14B	Vanilla	89.0	4,039	100%	40.9	4,806	100%	52.7	9,259	100%	60.9	100%
	DEER	90.9	1,000	24.8%	40.7	1,583	32.9%	52.1	4,000	43.2%	61.2	33.6%
7B	Vanilla	78.6	5,666	100%	26.1	8,516	100%	38.4	10,482	100%	47.7	100%
	DEER	78.6	913	16.1%	25.2	1,605	18.8%	40.3	2,582	24.6%	48.0	19.9%
Qwen3 Series												
14B	Vanilla	93.3	3,277	100%	44.6	5,072	100%	73.4	8,203	100%	70.4	100%
	DEER	93.9	1,118	34.1%	44.3	792	15.6%	74.1	5,437	66.3%	70.8	38.7%
8B	Vanilla	85.4	3,904	100%	37.9	6,994	100%	64.7	8,871	100%	62.7	100%
	DEER	87.8	793	20.3%	41.1	608	8.7%	65.1	4,257	48.0%	64.7	25.7%
4B	Vanilla	91.5	3,768	100%	36.1	7,804	100%	64.7	8,789	100%	64.1	100%
	DEER	92.7	1,050	27.9%	37.1	826	10.6%	63.5	5,626	64.0%	64.4	34.2%
1.7B	Vanilla	83.5	3,580	100%	25.7	6,151	100%	51.7	9,447	100%	53.6	100%
	DEER	84.1	2,236	62.5%	28.3	2,104	34.2%	51.4	8,425	89.2%	54.6	62.0%

Performance on SoTA Reasoning Models. 我们在两个最先进的推理模型上评估了 DEER 的有效性: Qwen3-32B (代表稠密模型) 和 Deepseek-R1 671B (代表 MoE 模型)。为了充分利用它们的推理能力, 我们将其最大序列长度分别设置为官方推荐的 32k 和 16k。最大长度的影响将在下一节进一步讨论。由于计算资源限制, 我们基于 KTransformers (kvcache

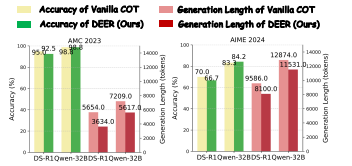


图 12: 在先进模型上的性能表现。

Table 13: Experimental results on more types of reasoning models and reasoning benchmarks. "Acc." denotes accuracy, "Tok." denotes token count, and "CR" denotes compression rate. ↑ indicates that higher values are better, while ↓ indicates that lower values are better. The best results are highlighted in **bold**.

Method	GSM8K			MATH-500			AMC23			MATH			AIME-24			AIME-25			OlympiadBench			SCIENCE			Overall	
	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	Acc $\uparrow$	CR $\downarrow$	
<i>DeepSeek-R1-Distill-Qwen-32B</i>	Vanilla	94.3	1202	100%	89.2	3736	100%	87.5	5354	100%	56.7	10293	100%	43.3	11075	100%	55.7	7334	100%	56.1	7181	100%	69.0	100%	69.0	100%
	DEER	95.1	819	68.1%	90.4	2425	64.9%	95	4252	79.4%	63.3	7424	72.1%	46.7	8913	80.5%	57.8	5351	73.0%	64.1	4943	68.8%	73.2	72.4%		
	<i>DeepSeek-R1-Distill-Qwen-14B</i>	Vanilla	93.9	1458	100%	88.6	3815	100%	82.5	6545	100%	51.7	11211	100%	36.7	12304	100%	52.6	7908	100%	52.0	6731	100%	65.4	100%	
<i>DeepSeek-R1-Distill-Qwen-7B</i>	DEER	93.3	1040	71.3%	89.8	2577	67.5%	85.0	4240	64.8%	68.4	8115	72.4%	36.7	10125	82.3%	56.7	5736	72.5%	56.6	4856	72.1%	69.3	71.9%		
	<i>DeepSeek-R1-Distill-Qwen-1.5B</i>	Vanilla	89.6	1484	100%	89.8	2438	100%	78.8	6792	100%	47.1	13765	100%	26.7%	12767	100%	47.3	8563	100%	37.3	10247	100%	56.5	100%	
	DEER	90.0	917	61.8%	89.8	2143	55.5%	85.0	4451	65.5%	49.2	9839	71.5%	36.7	7257	56.8%	52.6	5420	63.3%	31.3	5469	53.4%	62.2	61.1%		
<i>DeepSeek-R1-Distill-Qwen-1.5B</i>	Vanilla	76.1	1617	100%	69.0	6018	100%	52.5	8819	100%	23.3	13702	100%	13.3	14450	100%	28.0	11200	100%	7.1	13029	100%	38.5	100%		
	DEER	74.7	984	60.9%	67.8	2497	41.5%	60.0	5496	62.3%	23.3	9557	69.7%	10.0	9281	64.2%	32.0	5960	53.2%	12.1	5762	44.2%	40.0	56.6%		
	<i>Qwen3-32B</i>	Vanilla	96.3	1668	100%	94.4	4440	100%	95.0	7627	100%	73.3	11374	100%	65.0	12446	100%	63.4	6438	100%	65.2	6893	100%	78.9	100%	
<i>Qwen3-14B</i>	DEER	96.2	769	46.1%	94.2	3418	77.0%	97.5	5753	75.4%	76.7	8682	76.3%	66.7	10893	87.5%	67.9	5189	80.6%	64.7	4167	60.5%	80.6	71.9%		
	Vanilla	95.1	2047	100%	93.8	4508	100%	95.0	7139	100%	70.0	10859	100%	63.3	12286	100%	62.5	8692	100%	60.1	7339	100%	77.1	100%		
	DEER	95.3	840	41.0%	94.0	3074	68.2%	95.0	4763	66.7%	76.7	7619	70.2%	66.7	11135	90.6%	67.4	7060	81.2%	57.6	2898	39.5%	79.0	65.0%		
<i>Qwen3-8B</i>	Vanilla	94.9	2245	100%	91.2	5216	100%	87.5	7986	100%	65.0	12110	100%	54.2	12835	100%	59.3	9487	100%	51.5	9145	100%	71.9	100%		
	DEER	95.2	1071	47.7%	92.6	2732	52.4%	92.5	4392	55.0%	61.7	8796	72.6%	60.0	12229	95.3%	62.4	7479	78.8%	52.5	3111	34.0%	73.8	62.3%		
	<i>Qwen3-4B</i>	Vanilla	94.1	2175	100%	92.2	4767	100%	87.5	7443	100%	63.3	11916	100%	48.4	13112	100%	59.3	9098	100%	46.5	9294	100%	70.2	100%	
<i>Qwen3-1.7B</i>	DEER	94.5	1250	57.5%	92.6	3214	67.4%	87.5	4906	65.9%	63.3	9327	78.3%	55.0	12039	91.8%	64.7	7569	83.2%	47.5	3275	35.4%	72.2	68.5%		
	Vanilla	90.1	2045	100%	85.6	5160	100%	70.0	8637	100%	30.0	13758	100%	26.7	13943	100%	52.2	9437	100%	35.9	9271	100%	55.8	100%		
	DEER	90.3	1066	52.1%	85.6	2463	47.7%	70.0	4673	54.1%	30.0	7943	57.7%	36.7	11579	83.0%	52.6	7257	76.9%	43.4	3549	38.3%	58.4	58.6%		
<i>Qwen3-32B</i>	Vanilla	96.7	1427	100%	93.8	4508	100%	92.5	6792	100%	66.7	10821	100%	46.7	12300	100%	65.2	8546	100%	63.1	7320	100%	75.0	100%		
	DEER	96.3	977	68.5%	94.6	3316	73.6%	95.0	5782	85.1%	70.0	10097	93.3%	50.0	11598	94.3%	65.2	7639	89.4%	64.1	6163	84.2%	76.5	84.0%		
	<i>Llama-3.1-Nemotron-Nano-8B-v1</i>	Vanilla	89.2	1618	100%	91.2	3794	100%	90.0	6153	100%	56.7	10821	100%	35.0	13192	100%	54.3	7321	100%	41.9	8074	100%	65.5	100%	
DEER	89.8	1473	91.0%	91.4	2995	78.9%	90.0	5408	87.9%	66.7	9755	90.6%	36.7	11820	89.6%	60.7	6407	87.5%	47.5	7576	93.8%	69.0	88.5%			

表 4: 更多模型和任务上的额外阈值敏感性实验。

Qwen3-14B	GSM8K	MATH	AMC
Vanilla	95.1	93.8	95.0
0.80	96.0	93.8	93.8
0.85	95.7	94.4	93.8
0.90	96.1	93.8	95.0
0.95	96.0	94.0	95.6
0.97	95.7	93.8	94.4
Qwen3-8B	GSM8K	MATH	AMC
Vanilla	94.9	91.2	87.5
0.80	95.2	91.4	88.8
0.85	94.9	92.0	90.0
0.90	95.5	92.8	91.3
0.95	95.3	93.2	92.5
0.97	95.3	93.0	92.5

ai, 2025) 实现了 Deepseek-R1 的量化版本。图

12 详细展示了 DEER 在两个具有挑战性的数据集 AIME 和 MATH 上的表现。结果表明, DEER 在保持有竞争力的准确率 (R1 在每个数据集上仅多犯一个错误) 的同时, 显著地将序列长度减少了 10.4% - 35.7%。

Performance Trends across More Model Sizes and Benchmarks. 为了更全面地展示 DEER 的有效性, 并方便研究人员进行比较, 我们在七个推理基准测试和十一个大型推理语言模型上呈现了实验结果。图 K 比较了 DEER 与原始思维链的实验结果, 表明论文正文中得出的结论在更多基准测试和额外模型上保持一致。

除了流行的推理模型, 我们也评估了 DEER 在较少使用的模型上的表现。如局限性部分所述, Llama-3.1-Nemotron-Nano-8B-v1 在生成中间答案时始终表现出较低的信心度, 导致其在评估期间的提前停止率显著低于主流模型 (Qwen3-8B: 80%, R1-Distill-Qwen-7B: 85%, Llama-3.1-Nemotron-Nano-8B-v1: 55%)。因此, 如表 5 所示, Llama-3.1-Nemotron-Nano-8B-v1 的推理效率提升有限。尽管如此, DEER 仍然有效地缓解了该模型中的过度思考问题, 这体现在它能够通过提前停止来防止后续推理步骤将正确答案改为错误答案。

Performance with Different Decoding Configurations. 由于 DeepSeek-R1-Distill-Series 模型的配置推荐最大长度为 16k (16,384), 为了保持设置的一致性, 我们在主要实验中也以相同设置评估了 Qwen3-14B。实际上, 这个长度对于大多数现实世界的应用来说已经足够。此外, 为了确保实验的稳定性和可复现性, 我们在主要实验中采用了贪心解码。尽管如此, 为了更全面地评估 DEER 的性能, 我们进一步在 Qwen3 系列模型的更大变体 (8B、14B、32B) 上使用官方推荐的解码策略 ($\max_len = 32768$, $\top_p = 0.95$, $temperature = 0.6$) 进行了实验。图 8 显示, 在这些配置下, DEER 仍然显著有效, 证明了我们方法的鲁棒性。

Performance with Different Multi-Token Confidence Averaging Methods. 在正文中, 我们提到了采用几何平均策略来计算多标记答案的置信度分数, 因为它更符合语言模型中联合概率计算的乘法性质, 并且对低概率值表现出更高的敏感性。在本节中, 我们补充了使用算术

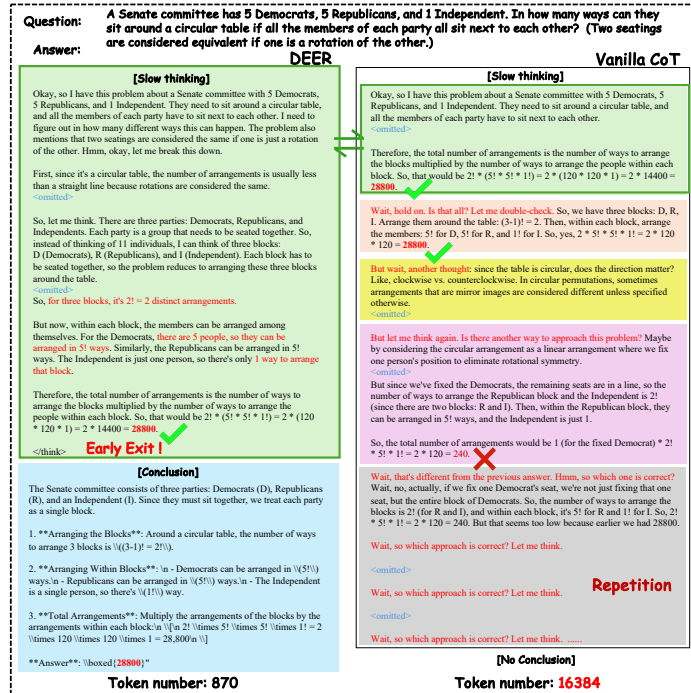


Figure 13: Comparison of generated content samples between DEER and Vanilla CoT on MATH-500. Both DEER and vanilla CoT arrive at the correct answer during the first reasoning step, as shown in the green box. The difference lies in the fact that DEER exits early after evaluating the confidence of the trial answer as sufficiently high, thus producing the correct result.

表 5: 在 MATH-500 (DeepSeek-R1-Distill-Qwen-14B) 上使用不同推理转换监控器的结果。DEER(W) 表示通过 Wait 进行转换, DEER(A) 表示通过 Alternatively 进行转换, DEER(Ent) 表示通过熵阈值进行转换。块大小表示一个推理块 (思考) 的长度 (标记数量), 块数量表示推理块的数量。

Method	Accuracy	Tokens	Chunk Size	Chunk Num	Exit Ratio	Exit Acc.
Vanilla	88.6	3815	—	—	—	—
DEER(W)	89.6 ^{+1.0}	2572 ^{-32.6%}	259.5	14.7	87.6%	93.4%
DEER(A)	90.8 ^{+2.2}	2775 ^{-27.3%}	719.8	5.3	54.8%	91.2%
DEER(Ent)	90.2 ^{+1.6}	2339 ^{-38.7%}	183.7	20.8	90.2%	93.0%

平均计算 (DEER-AM) 的对比实验分析, 采用相同的 0.95 提前退出阈值。表7中的结果表明, 与 DEER-GM 相比, DEER-AM 的准确率显著下降, 同时在压缩比方面仅实现了边际改进。这表明算术平均稀释了低值标记的贡献, 导致整体置信度分数虚高, 进而引发过早的错误退出。因此, 我们建议使用几何平均来估计真实的置信度分数。

Error Bars with 95% Confidence Intervals. 为了证明 DEER 准确率提升的统计显著性, 我们在两个模型上进行了多次实验运行, 并计算了 95% 置信区间的误差条。具体来说, 我们在 GSM8K、MATH 和 GPQA 基准上进行了四次独立运行。鉴于 AMC23 和 AIME24 的样本量有限, 我们将这些数据集的实验重复次数增加到八次。表9中呈现的结果证实, 我们的方法所实现的准确率提升具有统计显著性。

I Case Study Details

在图 13 中, 我们提供了在 MATH-500 数据集上的结果示例, 以直观展示 DEER 的有效性。DEER 的设计确保其在提前退出之前遵循与原始思维链方法相同的推理过程。如绿色框所示, 两种方法都在第一个推理步骤中得出了正确答案。区别在于, 我们的方法在评估试验答案的置信度足够高后提前退出, 从而产生正确结果。相比之下, 原始思维链方法继续进行下一个推理动作。经过双重检查并切换推理方法后, 由于两种方法得出的答案不一致, 模型陷入了无尽的验证循环, 最终未能提供最终答案。此外, 图 14、15、16 提供了额外的生成示例, 以更全面地展示 DEER 提前退出机制的有效性, 并说明该方法的底层机制。

图 17 展示了 DEER 应用于一个数学示例的详细过程。可以观察到, 在每个推理切换点 (“Wait” 标记处), DEER 会生成一个试验答案并评估其置信度。置信度的变化与当前推理块和试验答案的可靠性是一致的。这表明大型推理模型隐式地知道何时提前退出, 而我们的方法简单地实现了模型自身的这种潜力。

J Related Work Details

OpenAI o1 (OpenAI, 2025) 的出现, 确立了测试时缩放 (Snell et al., 2024) 作为 LLM 社区的一个关键研究方向。这种方法增强了 LLM 的慢思考能力, 使其在解决复杂问题方面取得突破。近期 DeepSeek-R1 (DeepSeek-AI et al., 2025) 的开源进一步激发了人们对本地部署推理模型的兴趣。然而, 两个关键挑战已经浮现: 1) 生成过长的思维链会显著降低推理效率, 2) 越来越多的实证证据 (Chen et al., 2025b; Team et al., 2025a) 揭示了它们容易陷入

Question: The coordinates of a parallelogram are (5, 3), (6, 8), (7, 4) and \$(x, y)\$ and \$x > 7\$\$. What is the value of \$x + y\$?

Answer: **DEER**

Okay, so I have this problem where I need to find the coordinates of the fourth vertex of a parallelogram. The given points are (5, 3), (6, 8), (7, 4), and (x, y) with the condition that $x > 7$. Then, I need to find the value of $x + y$. Hmm, let me think about how to approach this. First, I remember that in a parallelogram, opposite sides are equal and parallel. That means the midpoints of the diagonals should be the same. So, if I can find the midpoint of one diagonal, it should be equal to the midpoint of the other diagonal. Let me label the points to make it clearer. Let's say point A is (5, 3), point B is (6, 8), point C is (7, 4), and point D is (x, y). Now, I need to figure out which points are connected to each other. Since it's a parallelogram, the order of the points matters. I think the points should be connected in order: so A to B to C to D and back to A. But I'm not entirely sure, so maybe I should consider different possibilities.

<omitted>

Let me try another approach. Another property of a parallelogram is that the diagonals bisect each other. So, the midpoint of diagonal AC should be the same as the midpoint of diagonal BD. Let me calculate the midpoint of AC first. Point A is (5, 3) and point C is (7, 4). The midpoint formula is $(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2})$. So, midpoint of AC is $(\frac{5 + 7}{2}, \frac{3 + 4}{2}) = (12/2, 7/2) = (6, 3.5)$.

<omitted>

Midpoint is $(\frac{5 + x}{2}, \frac{3 + y}{2})$. Midpoint of BC: Point B is (6, 8) and point C is (7, 4). Midpoint is $(\frac{6 + 7}{2}, \frac{8 + 4}{2}) = (13/2, 12/2) = (6.5, 6)$. Setting midpoints equal: $\frac{5 + x}{2} = 6.5$ and $\frac{3 + y}{2} = 6$. Solving the first equation: $5 + x = 13 \Rightarrow x = 8$. Solving the second equation: $3 + y = 12 \Rightarrow y = 9$. Okay, this gives $x = 8$ and $y = 9$, which satisfies $x > 7$. So, the fourth point is (8, 9), and $x + y = 8 + 9 = 17$.

</think>

To find the coordinates of the fourth vertex of the parallelogram, we use the property that the diagonals of a parallelogram bisect each other. Let's calculate the midpoint of diagonal BC. Points B (6, 8) and C (7, 4). Midpoint formula: $(\frac{6 + 7}{2}, \frac{8 + 4}{2}) = (6.5, 6)$. Let D be (x, y). Midpoint of AD: $(\frac{5 + x}{2}, \frac{3 + y}{2})$. Equate to (6.5, 6): $\frac{5 + x}{2} = 6.5 \Rightarrow x = 8$. $\frac{3 + y}{2} = 6 \Rightarrow y = 9$. So, the fourth point is (8, 9), and $x + y = 17$.

**Calculate $(x + y)$: $8 + 9 = 17$.

17

Token number: ~800

Vanilla CoT

Okay, so I have this problem where I need to find the coordinates of the fourth vertex of a parallelogram. The given points are (5, 3), (6, 8), (7, 4), and (x, y) with the condition that $x > 7$. Then, I need to find the value of $x + y$. Hmm, let me think about how to approach this. First, I remember that in a parallelogram, opposite sides are equal and parallel. That means the midpoints of the diagonals should be the same. So, if I can find the midpoint of one diagonal, it should be equal to the midpoint of the other diagonal. Let me label the points to make it clearer. Let's say point A is (5, 3), point B is (6, 8), point C is (7, 4), and point D is (x, y). Now, I need to figure out which points are connected to each other. Since it's a parallelogram, the order of the points matters. I think the points should be connected in order: so A to B to C to D and back to A. But I'm not entirely sure, so maybe I should consider different possibilities.

Let me try another approach. Another property of a parallelogram is that the diagonals bisect each other. So, the midpoint of diagonal AC should be the same as the midpoint of diagonal BD. Let me calculate the midpoint of AC first. Point A is (5, 3) and point C is (7, 4). The midpoint formula is $(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2})$. So, midpoint of AC is $(\frac{5 + 7}{2}, \frac{3 + 4}{2}) = (6, 3.5)$.

Midpoint of BC: Point B is (6, 8) and point C is (7, 4). Midpoint is $(\frac{6 + 7}{2}, \frac{8 + 4}{2}) = (6.5, 6)$. Setting midpoints equal: $\frac{5 + x}{2} = 6.5$ and $\frac{3 + y}{2} = 6$. Solving the first equation: $5 + x = 13 \Rightarrow x = 8$. Solving the second equation: $3 + y = 12 \Rightarrow y = 9$. Okay, this gives $x = 8$ and $y = 9$, which satisfies $x > 7$. So, the fourth point is (8, 9), and $x + y = 8 + 9 = 17$.

Wait, let me verify this. If the points are A(5,3), B(6,8), C(7,4), and D(8,9), does this form a parallelogram? Let me check the vectors.

These aren't equal either. Hmm, that's confusing. Wait, maybe I made a mistake in assigning the points. Let me try another approach.

Hmm, these aren't equal. Wait, that's a problem. Wait, but according to the vector addition, D should be (8,9). So, why are vectors AD and BC not equal? Wait, maybe I need to check the other pair of sides. Vector AD and vector BC are not equal, but vector AB and vector DC are equal.

These aren't equal, but in a parallelogram, both pairs should be equal. So, that's a problem. Wait, but according to the midpoint method, it's a parallelogram. So, maybe I'm missing something.

No answer!

Token number: 16384

Figure 14: Comparison of generated content between DEER and Vanilla on MATH-500.

表 6: Vanilla、DEER-W 和 DEER-Ent 在多个模型和数据集上的比较。Acc = 准确率 (%), Len = 平均标记数, CR = 压缩比。

Method	GSM8K			MATH			AMC			AIME			GPQA			Overall	
	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	CR
DeepSeek-R1-Distill-Qwen-7B																	
Vanilla	89.6	1,484	100%	87.4	3,858	100%	78.8	6,792	100%	41.7	13,765	100%	23.7	10,247	100%	64.2	100%
DEER-W	90.6	917	61.8%	89.8	2,143	55.5%	85.0	4,451	65.5%	49.2	9,839	71.5%	31.3	5,469	53.4%	69.2	61.5%
DEER-Ent	90.8	876	59.0%	89.2	2,261	58.6%	85.0	4,072	60.0%	48.4	8,961	65.1%	29.6	5,037	49.2%	68.6	58.4%
Qwen3-14B																	
Vanilla	95.1	2,047	100%	93.8	4,508	100%	95.0	7,139	100%	70.0	10,859	100%	60.1	7,339	100%	82.8	100%
DEER-W	95.3	840	41.0%	94.0	3,074	68.2%	95.0	4,763	66.7%	76.7	7,619	70.2%	57.6	2,898	39.5%	83.7	57.1%
DEER-Ent	96.1	803	39.2%	93.8	2,979	66.1%	93.3	4,903	68.7%	73.3	7,128	65.6%	58.1	2,818	38.4%	82.9	55.6%
Qwen3-8B																	
Vanilla	94.9	2,245	100%	91.2	5,216	100%	87.5	7,986	100%	65.0	12,110	100%	51.5	9,145	100%	78.0	100%
DEER-W	95.2	1,071	47.7%	92.6	2,732	52.4%	92.5	4,392	55.0%	61.7	8,796	72.6%	52.5	3,111	34.0%	78.9	52.3%
DEER-Ent	95.8	1,037	46.2%	93.6	2,789	53.3%	91.3	4,003	50.1%	63.3	8,328	68.8%	51.5	3,248	35.5%	79.1	50.8%

表 7: 比较多个模型和数据集上的 Vanilla、DEER-GM（几何平均）和 DEER-AM（算术平均）。Acc = 准确率 (%), Len = 平均词元数, CR = 压缩比。

Method	GSM8K			MATH			AMC			AIME			GPQA			Overall	
	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	Len	CR	Acc	CR
DeepSeek-R1-Distill-Qwen-7B																	
Vanilla	89.6	1,484	100%	87.4	3,858	100%	78.8	6,792	100%	41.7	13,765	100%	23.7	10,247	100%	64.2	100%
DEER-GM	90.6	917	61.8%	89.8	2,143	55.5%	85.0	4,451	65.5%	49.2	9,839	71.5%	31.3	5,469	53.4%	69.2	61.5%
DEER-AM	90.2	832	56.1%	88.2	1,879	48.7%	80.0	3,872	57.0%	43.3	8,095	58.8%	22.6	4,116	40.2%	64.9	52.2%
Qwen3-14B																	
Vanilla	95.1	2,047	100%	93.8	4,508	100%	95.0	7,139	100%	70.0	10,859	100%	60.1	7,339	100%	82.8	100%
DEER-GM	95.3	840	41.0%	94.0	3,074	68.2%	95.0	4,763	66.7%	76.7	7,619	70.2%	57.6	2,898	39.5%	83.7	57.1%
DEER-AM	95.3	811	39.6%	92.4	2,620	58.1%	90.0	4,513	63.2%	63.3	6,933	63.8%	53.6	2,508	34.2%	78.9	51.8%
Qwen3-8B																	
Vanilla	94.9	2,245	100%	91.2	5,216	100%	87.5	7,986	100%	65.0	12,110	100%	51.5	9,145	100%	78.0	100%
DEER-GM	95.2	1,071	47.7%	92.6	2,732	52.4%	92.5	4,392	55.0%	61.7	8,796	72.6%	52.5	3,111	34.0%	78.9	52.3%
DEER-AM	94.9	972	43.3%	92.0	2,522	48.4%	87.5	3,899	48.8%	56.7	7,697	63.6%	49.7	2,950	32.3%	76.2	47.3%

过度思考——即模型在达到最优输出点后仍继续推理的现象。Zhang et al. (2025c) 引入了一个名为 S1-Bench 的新基准，用于测试 LRM 在简单任务上的性能，评估这些 LRM 的过度思考问题。遵循 (Sui et al., 2025; Wang et al., 2025a) 中建立的高效推理分类法，我们将相关工作分为三类：基于后训练、基于提示和基于输出的高效推理方法。Post-training based efficient reasoning 方法使用监督微调 (Yu et al., 2024; Kang et al., 2025; Xia et al., 2025; Ma et al., 2025b; Munkhbat et al., 2025; Zhu et al., 2025; Liu et al., 2024; Han et al., 2024; Qiao et al., 2025; Yu et al., 2025) 配合可变长度的思维链数据，或在强化学习中引入长度奖励 (Team et al., 2025b; Luo et al., 2025a; Aggarwal & Welleck, 2025; Arora & Zanette, 2025; Yeo et al., 2025; Shen et al., 2025b; Qu et al., 2025; Cui et al., 2025; Dai et al., 2025; Liu et al., 2025a; Tu et al., 2025; Wang et al., 2025c; Dumitru et al., 2025; Li et al., 2025a; Jiang et al., 2025a; Zhang et al., 2025b)，以使模型能够自适应地生成不同长度的思维链。然而，这些方法通常需要大量的计算资源，并且在数据集构建方面面临挑战。最近，一些工作 (Hao et al., 2024; Shen et al., 2025c; Cheng & Van Durme, 2024; Dang et al., 2025; Shen et al., 2025a; Su et al., 2025; Tan et al., 2025; Saunshi et al., 2025; Zhang et al., 2025d) 表

Question: Point A lies somewhere within or on the square which has opposite corners at $(0,0)$ and $(2,2)$. Point B lies somewhere within or on the square which has opposite corners at $(4,2)$ and $(5,3)$. What is the greatest possible value of the slope of the line containing points A and B ? Express your answer as a common fraction.

DEER

Okay, so I have this problem where I need to find the greatest possible slope of a line connecting two points, A and B. Point A is somewhere inside or on a square with opposite corners at $(0,0)$ and $(2,2)$. Point B is inside or on another square with opposite corners at $(4,2)$ and $(5,3)$. I need to figure out the maximum slope possible between any two points A and B in these squares. First, let me visualize the squares. The first square has corners at $(0,0)$ and $(2,2)$. Similarly, the second square has opposite corners at $(4,2)$ and $(5,3)$. Hmm, so that square is a bit smaller. Let me figure out its sides. The distance between $(4,2)$ and $(5,3)$ is $\sqrt{(5-4)^2 + (3-2)^2} = \sqrt{1+1} = \sqrt{2}$. So, it's a square with side length $\sqrt{2}$, rotated 45 degrees relative to the axes. Wait, actually, no. If the opposite corners are $(4,2)$ and $(5,3)$, then the sides are not aligned with the axes. <omitted> But wait, is that achievable? Because point A is in the first square, so if A is at $(2,0)$, which is a corner, and point B is at $(4,3)$, which is a corner of the second square. So, connecting $(2,0)$ to $(4,3)$ would give a slope of $(3-0)/(4-2) = 3/2$. But is that the maximum? </think>

To find the greatest possible slope of the line containing points A and B , we need to maximize the expression $\frac{y_B - y_A}{x_B - x_A}$. Point A lies within or on the square with opposite corners at $(0,0)$ and $(2,2)$. The minimum x -coordinate for A is 0 and the maximum x -coordinate is 2 . Point B lies within or on the square with opposite corners at $(4,2)$ and $(5,3)$. The maximum y -coordinate for B is 3 and the minimum x -coordinate is 4 . To maximize the slope, we should choose A at $(2,0)$ and B at $(4,3)$. The slope is then $\frac{3-0}{4-2} = \frac{3}{2}$. However, upon closer inspection, the square for B is actually a diamond with corners at $(4,2)$, $(5,3)$, $(5,2)$, and $(4,3)$. The point $(4,3)$ is indeed a corner of this square. Therefore, the maximum slope is achievable. The greatest possible slope is $\frac{3}{2}$. <gold_answer>: $\frac{3}{2}$

Token number: ~1000

Vanilla CoT

Okay, so I have this problem where I need to find the greatest possible slope of a line connecting two points, A and B. Point A is somewhere inside or on a square with opposite corners at $(0,0)$ and $(2,2)$. Point B is inside or on another square with opposite corners at $(4,2)$ and $(5,3)$. I need to figure out the maximum slope possible between any two points A and B in these squares. First, let me visualize the squares. The first square has corners at $(0,0)$ and $(2,2)$. Similarly, the second square has opposite corners at $(4,2)$ and $(5,3)$. Hmm, so that square is a bit smaller. Let me figure out its sides. The distance between $(4,2)$ and $(5,3)$ is $\sqrt{(5-4)^2 + (3-2)^2} = \sqrt{1+1} = \sqrt{2}$. So, it's a square with side length $\sqrt{2}$, rotated 45 degrees relative to the axes. Wait, actually, no. If the opposite corners are $(4,2)$ and $(5,3)$, then the sides are not aligned with the axes. <omitted> But wait, is that achievable? Because point A is in the first square, so if A is at $(2,0)$, which is a corner, and point B is at $(4,3)$, which is a corner of the second square. So, connecting $(2,0)$ to $(4,3)$ would give a slope of $(3-0)/(4-2) = 3/2$. But is that the maximum? </think>

Wait, but maybe we can get a steeper slope by choosing points that are not at the corners. Because if we can have a larger $y_B - y_A$ with a smaller $x_B - x_A$, that would give a steeper slope. <omitted> Wait, but hold on. Let me think again. Maybe if we don't take A at $(2,0)$ and B at $(4,3)$, but somewhere else, we can get a steeper slope. <omitted> But wait, is there a way to have a larger slope? For example, if A is at $(2,2)$ and B is at $(4,3)$. Then, the slope is $(3-2)/(4-2) = 1/2$, which is less than $3/2$. Alternatively, if A is at $(0,0)$ and B is at $(5,3)$, then the slope is $(3-0)/(5-0) = 3/5$, which is less than $3/2$. <omitted> Alternatively, if point A is at (x, y_A) where $x < 2$, then point B would have to be at $(x+2, y_B)$. But since the second square starts at $x=4$, $x+2$ must be at least 4, so x must be at least 2. <omitted>

No answer !

Token number: 16384

Figure 15: Comparison of generated content between DEER and Vanilla on MATH-500.

表 8: 在官方推荐设置（最大 _ 长度 = 32768, top_p = 0.95, 温度 = 0.6）下, Qwen3 系列模型的实验结果。

Budget	Method	GSM8K Acc	Tok.	MATH-500 Acc	Tok.	AMC23 Acc	Tok.	AIME24 Acc	Tok.	AIME25 Acc	Tok.	OlympiadB Acc	Tok.	Overall Acc	CR
Qwen3-8B															
32k	Vanilla	95.7	2246	93.8	5368	93.8	9424	70.0	16717	65.0	17880	67.9	11025	81.0	100%
	DEER	95.5	981	94.0	3227	95.0	5898	75.8	12465	63.3	15135	67.0	9075	81.8	68.0%
Qwen3-14B															
32k	Vanilla	95.7	1699	94.8	4800	95.0	6837	75.0	14347	76.7	16437	68.7	9992	84.3	100%
	DEER	95.8	933	95.0	3301	96.3	6299	74.2	10896	76.7	15014	68.9	8263	84.5	77.6%
Qwen3-32B															
32k	Vanilla	96.0	1714	95.8	4609	98.8	7209	83.3	12874	78.3	15292	69.3	9775	86.9	100%
	DEER	95.8	992	95.4	3325	98.8	5617	84.2	11531	78.3	13981	69.8	8671	87.1	79.6%

表 9: 在推理基准测试上的准确率表现, 包含 95% 置信区间。

Model	GSM8K	MATH	AMC23	AIME24	GPQA
Vanilla (ds-7B)	0.897 [0.891, 0.902]	0.877 [0.869, 0.884]	0.794 [0.767, 0.821]	0.425 [0.400, 0.449]	0.247 [0.203, 0.291]
DEER (ds-7B)	0.904 [0.896, 0.912]	0.897 [0.883, 0.911]	0.856 [0.835, 0.878]	0.492 [0.463, 0.520]	0.299 [0.257, 0.341]
Vanilla (Qwen3-14B)	0.948 [0.942, 0.955]	0.938 [0.932, 0.943]	0.938 [0.918, 0.957]	0.708 [0.660, 0.757]	0.596 [0.571, 0.621]
DEER (Qwen3-14B)	0.955 [0.949, 0.962]	0.942 [0.936, 0.948]	0.953 [0.940, 0.967]	0.754 [0.718, 0.791]	0.587 [0.566, 0.608]

明, 使用潜在表征来替代显式的文本推理步骤, 可以使推理模型更加高效。然而, 此类方法通常需要在精心策划的数据集上进行多轮次的监督微调 (Hao et al., 2024; Xu et al., 2025d), 这会导致模型对输出格式的过拟合, 从而损害其固有的表达能力和泛化能力。

Prompt-based efficient reasoning 方法 (Han et al., 2024; Xu et al., 2025b; Lee et al., 2025; Renze & Guven, 2024; Chen et al., 2024) 使用不同的提示词来强制推理模型生成更简洁的思维链, 减少不必要的推理步骤。特别是, (Aytes et al., 2025; Chuang et al., 2024; 2025; Ong et al.) 根据查询的难度为其分配不同的提示词, 从而调整推理模型生成的思维链长度。我们也在表 1 中探索了我们的方法与提示设计相结合的性能, 证明了在保持相当准确性的同时, 可以进一步减少推理链的长度。大多数 Output-based efficient reasoning 方法侧重于优化大型语言模型的最佳 N 采样, 例如剪枝低质量样本 (Xie et al., 2023; Liao et al., 2025) 以及在多个样本达到自洽时实施提前停止 (Li et al., 2024; Manvi et al., 2024; Aggarwal et al., 2023)。然而, 随着如 R1 等先进推理模型的引入, 对最佳 N 采样方法的依赖有所减少, 因为这些模型本身就展现出强大的独立推理能力。最近, 两项同时进行的工作与我们的动机相似。Zhang et al. (2025a) 也提出基于试验答案提前终止, 但需要一个额外的探针模型来判断正确性。他们侧重于增强探针模型的验证能力, 而我们的方法探索如何使模型能够自我决定何时提前退出并无缝集成到现有的推理逻辑中。Ma et al. (2025a) 提示推理模型在解码过程中直接输出最终答案, 但仅在与基线相比的低预算场景下或适配到最佳 N 方法时取得更好性能, 这限制了其适用性和泛化能力。Song et al. (2025) 通过保留获得高重要性分数的 KV 缓存来周期性压缩 KV 缓存, 利用推理路径的语义稀疏性来加速推理。Jiang et al. (2025b) 使用教师模型执行技能感知的步骤分解和内容剪枝, 然后将剪枝后的推理路径蒸馏到学生模型中。Huang et al. (2025) 将引导方向投影到低维激活流形上, 并通过干预激活来减少思考令牌。

The expression $\$1 + 2 + 3 - 4 + 5 + 6$ is equivalent to 13. If we add parentheses in various locations, we can change this value. For example, $\$1 + 2 + 3 - (4 + 5) + 6 = 1 + 2 + 3 - 9 + 6 = 6 - 9 + 6 = 3$. If we only add parentheses (and not rearrange the order of the terms), what is the minimum possible value this expression can obtain? (For the purposes of this problem, we cannot change addition or subtraction into multiplication. For example, $\$(1 + 2 + 3) - (4) + 5 + 6$ is not a valid placement of parentheses.) Enter your answer as a single value.

Answer:

DEER

[illegible]

</think>

To find the minimum possible value of the expression $1 + 2 + 3 - 4 + 5 + 6$ by adding parentheses, we can strategically group terms to maximize the impact of the subtraction. By grouping the terms after the subtraction, we can create a larger number to subtract, significantly reducing the overall value. For example, grouping $(4 + 5 + 6)$ together: $1 + 2 + 3 - (4 + 5 + 6) = 6 - 15 = -9$. Thus, the minimum possible value is -9 .

Token number: ~1500

Vanilla CoT

Okay, so I have this math problem here: the expression is $1 + 2 + 3 + 4 + 5 + 6$, and it's equal to 13. The task is to add parentheses in various places to change the value, but I can't change the numbers or the operators.

First, I need to find the minimum possible value this expression can obtain by adding parentheses. Hmm, interesting. I must make sure I understand the problem correctly. I can only add parentheses, which will change the order in which operations are performed, but I can't move the numbers around or change the plus and minus signs into multiplication or division. So, for example, I can do something like $(1 + 2) + 3 + 4 + 5 + 6$, but I can't do $1 * 2 + 3 + 4 + 5 + 6$ because that would involve multiplying, which isn't allowed.

Committed:

Yep, that's correct. So, 13 is the original value. Now, let's try some alternatives. Grouping $(1 + 2)$ gives $3 + 3 + 4 + 5 + 6 = 21$. Grouping $(3 + 4)$ gives $1 + 7 + 5 + 6 = 19$. Grouping $(5 + 6)$ gives $1 + 2 + 3 + 11 = 17$. Grouping $(1 + 2 + 3)$ gives $6 + 4 + 5 + 6 = 21$. Grouping $(4 + 5)$ gives $1 + 2 + 9 + 6 = 18$. Grouping $(2 + 3 + 4)$ gives $1 + 9 + 5 + 6 = 21$. Grouping $(3 + 4 + 5)$ gives $1 + 2 + 12 + 6 = 21$. Grouping $(4 + 5 + 6)$ gives $1 + 2 + 3 + 15 = 21$. Grouping $(1 + 2 + 3 + 4)$ gives $10 + 5 + 6 = 21$. Grouping $(2 + 3 + 4 + 5)$ gives $1 + 14 + 6 = 21$. Grouping $(3 + 4 + 5 + 6)$ gives $1 + 2 + 18 = 21$.

Wait, all these groupings result in values greater than 13. But the problem says there are other ways to get different results. Let me think again. Maybe I should consider more complex groupings. For example, $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + (4 + 5) + 6 = 21$. Or $1 + ((2 + 3) + 4) + (5 + 6) = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + (5 + 6) = 21$. Or $(1 + (2 + 3)) + (4 + 5) + 6 = 21$. Or $1 + (2 + (3 + 4)) + (5 + 6) = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + (5 + 6) = 21$. Or $1 + ((2 + 3) + 4) + 5 + (6) = 21$. Or $1 + 2 + ((3 + 4) + 5) + (6) = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + (6) = 21$. Or $(1 + (2 + 3)) + 4 + (5 + 6) = 21$. Or $1 + (2 + (3 + 4)) + (5 + 6) = 21$. Or $1 + 2 + (3 + (4 + 5)) + (6) = 21$. Or $((1 + 2) + 3) + (4 + 5) + (6) = 21$. Or $1 + ((2 + 3) + 4) + (5 + 6) = 21$. Or $1 + 2 + ((3 + 4) + 5) + (6) = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + (5 + 6) = 21$. Or $(1 + (2 + 3)) + (4 + 5) + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2 + (3 + 4)) + 5 + 6 = 21$. Or $1 + 2 + (3 + (4 + 5)) + 6 = 21$. Or $((1 + 2) + 3) + 4 + 5 + 6 = 21$. Or $1 + ((2 + 3) + 4) + 5 + 6 = 21$. Or $1 + 2 + ((3 + 4) + 5) + 6 = 21$. Or $1 + 2 + 3 + ((4 + 5) + 6) = 21$. Or $((1 + 2) + (3 + 4)) + 5 + 6 = 21$. Or $(1 + (2 + 3)) + 4 + 5 + 6 = 21$. Or $1 + (2$

Wait, hold on. The original expression is $1 + 2 + 3 - 4 + 5 + 6$. If I group $4 + 5 + 6$, that would be subtracting 15, so $6 - 15 = -9$.

<omitted>

Wait, but in the original expression, after the subtraction, it's $+5 + 6$. So, if I group $4 + 5 + 6$, that would be changing the operations after the subtraction, but I can't rearrange terms. So, is that allowed?

<omitted>

Alternatively, group 1 + (2 + 3 - 4 + 5 + 6). As before, 1 + 12 = 13. \n\nAlternatively, group (1 + 2 + 3 - 4 + 5) + 6. ...

Endless exhaustive search!

Token number: 16384

Figure 16: Comparison of generated content between DEER and Vanilla on MATH-500.

表 10: Linguistic Markers 与 Other Tokens 的标记熵。

Qwen3-8B	Linguistic Markers	Other Tokens
gsm8k	0.901	0.438
math	1.058	0.385
gpqa	1.269	0.500

DS-7B	Linguistic Markers	Other Tokens
gsm8k	1.550	0.658
math	1.753	0.565
gpqa	1.241	0.510

表 11: Linguistic Markers 与 Other Tokens 之间的隐藏状态余弦相似度。

Qwen3-8B	Linguistic Markers	Other Tokens
gsm8k	0.262	0.543
math	0.237	0.493
gpqa	0.240	0.509

DS-7B	Linguistic Markers	Other Tokens
gsm8k	0.306	0.608
math	0.247	0.530
gpqa	0.231	0.505

K Use of LLMs

在准备本稿件的过程中，大型语言模型（LLMs）被用作语言润色的辅助工具。在稿件准备的最佳阶段，我们利用 LLMs 对选定的段落进行了语言上的精炼，包括语法检查、句子结构优化和表达标准化。此过程仅限于语言层面的改进，不涉及任何实质性学术内容的生成或修改，包括研究见解、数据分析或结论推导。需要强调的是，本文中提出的所有核心论点、研究方法、实验设计、数据分析和结论均由作者独立完成的学术责任。LLMs 仅作为语言处理辅助工具，作者对所有内容承担完全的学术责任。

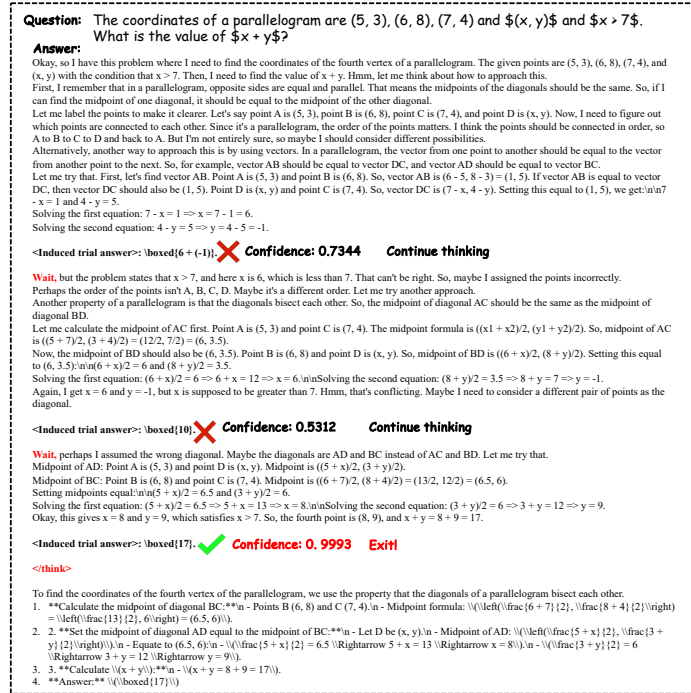


Figure 17: An example demonstrating LRM's implicit ability to sense the emergence of pearl reasoning. When the model's reasoning content is insufficient to support the elicited answer, the confidence remains at a low level, as demonstrated by the first two confidence values in the figure. Conversely, when the model has provided adequate and sound reasoning as justification, the confidence reaches high levels, potentially approaching 1, as shown by the final confidence value in the figure.

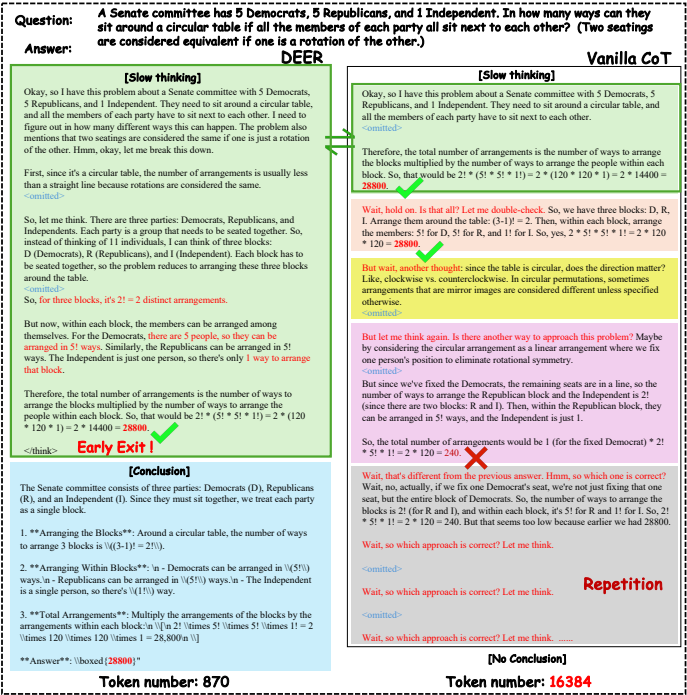
表 12: 不同模型在各任务上的置信区间分布 (%)。

Qwen3-14B	0-0.9	0.9-0.97	0.97-1.0
gsm8k	38.92	5.95	55.06
math	49.53	4.83	45.46
aime	77.20	2.45	20.35
Qwen3-8B	0-0.9	0.9-0.97	0.97-1.0
gsm8k	35.31	5.34	59.12
math	45.27	4.58	49.98
aime	78.61	2.39	19.00
DS-7B	0-0.9	0.9-0.97	0.97-1.0
gsm8k	26.54	5.29	68.17
math	34.09	5.46	60.45
aime	80.90	1.41	17.69

<think>
 In $n!$ to express the double sum $\sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!}$ in terms of $(p|p)$ and $(q|q)$, we start by changing variables. Let $V(n+j+k)$. For each $(n \nabla q \nabla 2)$, the number of pairs $(j|k)$ such that $(j+k+n) = (n-1)$. This allows us to rewrite the double sum as $n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!} = n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!}$. In the next, we split this into two separate sums: $n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!} = n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!} + n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!}$. The first sum is the series for $(p|p)$ and the second sum is the series for $(q|q)$. We have thus written the double sum as $n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!} = n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!} + n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!}$. Substituting these into the expression gives $n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!} = n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!} + n! \sum_{j=1}^{n-1} \sum_{k=1}^{n-j} \frac{1}{j!k!} \frac{1}{(n-j-k)!}$. Thus, the double sum simplifies to $(p|p) + (q|q)$.

表 13: 在更多类型推理模型和推理基准上的实验结果。“Acc”表示准确率,“ToK”表示标记数量,“CR”表示压缩率,“↑”表示数值越高越好,而“↓”表示数值越低越好。最佳结果以 **bold** 高亮显示。

Method	GSMK			MATH-500			AMC23			AIME24			AIME25			OlympiadBench			Science			Overall		
	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
DeepSeek-R1-Distill-Qwen-72B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	94.3	1,202	100%	89.2	3,736	100%	87.5	5,354	100%	56.7	10,293	100%	43.3	11,075	100%	55.7	7,334	100%	56.1	7,181	100%	69.0	100%
	DeepSeek	95.1	891	68.1%	90.4	2,425	64.9%	95	4,252	79.4%	63.3	7,424	72.1%	46.7	8,913	80.5%	57.8	5,351	73.0%	64.1	4,943	68.8%	73.2	72.4%
DeepSeek-R1-Distill-Qwen-14B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	93.9	1,458	100%	88.6	3,815	100%	82.5	6,345	100%	51.7	11,211	100%	36.7	12,304	100%	52.6	7,908	100%	52.0	6,731	100%	65.4	100%
	DeepSeek	93.3	1,040	71.3%	89.8	2,577	67.5%	85.0	4,240	64.8%	68.4	8,115	72.4%	36.7	10,125	82.3%	55.0	5,736	72.5%	56.6	4,856	72.1%	69.3	71.9%
DeepSeek-R1-Distill-Qwen-7B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	89.6	1,484	100%	87.4	3,858	100%	78.8	6,792	100%	41.7	13,705	100%	26.7%	12,767	100%	47.3	8,563	100%	23.7	10,217	100%	56.5	100%
	DeepSeek	90.6	917	61.8%	89.8	2,143	55.5%	85.0	4,451	65.5%	49.2	9,839	71.5%	36.7	7,257	56.8%	52.6	5,420	63.3%	31.3	5,469	53.4%	62.2	61.1%
DeepSeek-R1-Distill-Qwen-1.5B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	76.1	1,617	100%	69.0	6,018	100%	52.5	8,819	100%	23.3	13,702	100%	13.3	14,450	100%	28.0	11,200	100%	7.1	13,029	100%	38.5	100%
	DeepSeek	74.7	984	60.9%	67.8	2,497	41.5%	60.0	5,496	62.8%	23.3	9,557	69.7%	10.0	9,281	64.2%	32.0	5,960	58.2%	12.1	5,762	44.2%	40.0	56.6%
Qwen3-72B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	96.3	1,668	100%	94.4	4,440	100%	95.0	7,027	100%	73.3	11,374	100%	65.0	12,446	100%	63.4	6,438	100%	65.2	6,893	100%	78.9	100%
	DeepSeek	96.2	769	46.1%	94.2	3,418	77.0%	97.5	5,753	75.4%	76.7	8,682	76.3%	66.7	10,893	87.5%	67.9	5,189	80.6%	64.7	4,167	60.5%	80.6	71.9%
Qwen3-14B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	95.1	2,047	100%	93.8	4,508	100%	95.0	7,139	100%	70.0	10,859	100%	63.3	12,286	100%	62.5	8,692	100%	60.1	7,339	100%	77.1	100%
	DeepSeek	95.3	840	41.0%	94.0	3,074	68.2%	95.0	4,763	66.7%	76.7	7,619	70.2%	66.7	11,135	90.6%	67.4	7,060	81.2%	57.6	2,898	39.5%	79.0	65.0%
Qwen3-8B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	94.9	2,245	100%	91.2	5,216	100%	87.5	7,986	100%	65.0	12,110	100%	54.2	12,835	100%	59.3	9,487	100%	51.5	9,145	100%	71.9	100%
	DeepSeek	95.2	1,071	47.7%	92.6	2,782	52.4%	92.5	4,392	55.0%	61.7	8,796	72.6%	60.0	12,229	95.3%	62.4	7,479	78.8%	52.5	3,111	34.0%	73.8	62.3%
Qwen3-4B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	94.1	2,175	100%	92.2	4,767	100%	87.5	7,443	100%	63.3	11,916	100%	48.4	13,112	100%	59.3	9,098	100%	46.5	9,294	100%	70.2	100%
	DeepSeek	94.5	1,250	57.5%	92.6	3,214	67.4%	87.5	4,906	65.9%	63.3	9,327	78.3%	55.0	12,039	91.8%	64.7	7,569	83.2%	47.5	3,275	35.4%	72.2	68.5%
Qwen3-1.7B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	90.1	2,045	100%	85.6	5,160	100%	70.0	8,637	100%	30.0	13,758	100%	26.7	13,943	100%	52.2	9,437	100%	35.9	9,271	100%	55.8	100%
	DeepSeek	90.3	1,066	52.1%	85.6	2,463	47.7%	70.0	4,673	54.1%	30.0	7,943	57.7%	36.7	11,579	83.0%	52.6	7,257	76.9%	43.4	3,549	38.3%	58.4	58.6%
QwQ-32B	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	96.7	1,427	100%	93.8	4,508	100%	92.5	6,792	100%	66.7	10,821	100%	46.7	12,300	100%	65.2	8,546	100%	63.1	7,320	100%	75.0	100%
	DeepSeek	96.3	977	68.5%	94.6	3,316	73.6%	95.0	5,782	85.1%	70.0	10,097	93.3%	50.0	11,598	94.3%	65.2	7,639	89.4%	64.1	6,163	84.2%	76.5	84.0%
Llama-3.1-Nemotron-Nano-8B-v1	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	TokL	CRL	Acc $\uparrow$	CRL	
	Vanilla	89.2	1,618	100%	91.2	3,794	100%	90.0	6,153	100%	56.7	10,821	100%	35.0	13,192	100%	54.3	7,321	100%	41.9	8,074	100%	65.5	100%
	DeepSeek	89.8	1,473	91.0%	91.4	2,995	78.9%	90.0	5,408	87.6%	66.7	9,755	90.6%	36.7	11,820	89.6%	60.7	6,407	87.5%	47.5	7,576	93.8%	69.0	88.5%



Question: The coordinates of a parallelogram are (5, 3), (6, 8), (7, 4) and \$(x, y)\$ and \$x > 7\$. What is the value of \$x + y\$?

Answer: **DEER**

Okay, so I have this problem where I need to find the coordinates of the fourth vertex of a parallelogram. The given points are (5, 3), (6, 8), (7, 4), and (x, y) with the condition that $x > 7$. Then, I need to find the value of $x + y$. Hmm, let me think about how to approach this. First, I remember that in a parallelogram, opposite sides are equal and parallel. That means the midpoints of the diagonals should be the same. So, if I can find the midpoint of one diagonal, it should be equal to the midpoint of the other diagonal. Let me label the points to make it clearer. Let's say point A is (5, 3), point B is (6, 8), point C is (7, 4), and point D is (x, y). Now, I need to figure out which points are connected to each other. Since it's a parallelogram, the order of the points matters. I think the points should be connected in order, so A to B to C to D and back to A. But I'm not entirely sure, so maybe I should consider different possibilities.

<omitted>

Let me try another approach. Another property of a parallelogram is that the diagonals bisect each other. So, the midpoint of diagonal AC should be the same as the midpoint of diagonal BD. Let me calculate the midpoint of AC first. Point A is (5, 3) and point C is (7, 4). The midpoint formula is $(x_1 + x_2)/2, (y_1 + y_2)/2$. So, midpoint of AC is $((5 + 7)/2, (3 + 4)/2) = (12/2, 7/2) = (6, 3.5)$.

<omitted>

Midpoint is $((5 + x)/2, (3 + y)/2)$. Midpoint of BC: Point B is (6, 8) and point C is (7, 4). Midpoint is $((6 + 7)/2, (8 + 4)/2) = (13/2, 12/2) = (6.5, 6)$. Setting midpoints equal: $(5 + x)/2 = 6.5$ and $(3 + y)/2 = 6$. Solving the first equation: $(5 + x)/2 = 6.5 \Rightarrow 5 + x = 13 \Rightarrow x = 8$. Solving the second equation: $(3 + y)/2 = 6 \Rightarrow 3 + y = 12 \Rightarrow y = 9$. Okay, this gives $x = 8$ and $y = 9$, which satisfies $x > 7$. So, the fourth point is (8, 9), and $x + y = 8 + 9 = 17$.

</think>

To find the coordinates of the fourth vertex of the parallelogram, we use the property that the diagonals of a parallelogram bisect each other. Calculate the midpoint of diagonal BC: Points B (6, 8) and C (7, 4). Midpoint formula: $(\frac{6+7}{2}, \frac{8+4}{2}) = (\frac{13}{2}, 6) = (6.5, 6)$. Set the midpoint of diagonal AD equal to the midpoint of BC: Let D be (x, y). Midpoint of AD: $(\frac{5+x}{2}, \frac{3+y}{2})$. Equate to (6.5, 6): $(\frac{5+x}{2}, \frac{3+y}{2}) = (6.5, 6)$. Solve for x: $\frac{5+x}{2} = 6.5 \Rightarrow 5+x = 13 \Rightarrow x = 8$. Solve for y: $\frac{3+y}{2} = 6 \Rightarrow 3+y = 12 \Rightarrow y = 9$. Calculate $x + y = 8 + 9 = 17$.

Answer: 17

Token number: ~800

Vanilla CoT

Okay, so I have this problem where I need to find the coordinates of the fourth vertex of a parallelogram. The given points are (5, 3), (6, 8), (7, 4), and (x, y) with the condition that $x > 7$. Then, I need to find the value of $x + y$. Hmm, let me think about how to approach this. First, I remember that in a parallelogram, opposite sides are equal and parallel. That means the midpoints of the diagonals should be the same. So, if I can find the midpoint of one diagonal, it should be equal to the midpoint of the other diagonal. Let me label the points to make it clearer. Let's say point A is (5, 3), point B is (6, 8), point C is (7, 4), and point D is (x, y). Now, I need to figure out which points are connected to each other. Since it's a parallelogram, the order of the points matters. I think the points should be connected in order, so A to B to C to D and back to A. But I'm not entirely sure, so maybe I should consider different possibilities.

Let me try another approach. Another property of a parallelogram is that the diagonals bisect each other. So, the midpoint of diagonal AC should be the same as the midpoint of diagonal BD.

<omitted>

Midpoint is $((5 + x)/2, (3 + y)/2)$. Midpoint of BC: Point B is (6, 8) and point C is (7, 4). Midpoint is $((6 + 7)/2, (8 + 4)/2) = (13/2, 12/2) = (6.5, 6)$. Setting midpoints equal: $(5 + x)/2 = 6.5$ and $(3 + y)/2 = 6$. Solving the first equation: $(5 + x)/2 = 6.5 \Rightarrow 5 + x = 13 \Rightarrow x = 8$. Solving the second equation: $(3 + y)/2 = 6 \Rightarrow 3 + y = 12 \Rightarrow y = 9$. Okay, this gives $x = 8$ and $y = 9$, which satisfies $x > 7$. So, the fourth point is (8, 9), and $x + y = 8 + 9 = 17$.

Wait, let me verify this. If the points are A(5,3), B(6,8), C(7,4), and D(8,9), does this form a parallelogram? Let me check the vectors.

<omitted>

These aren't equal either. Hmm, that's confusing. Wait, maybe I made a mistake in assigning the points. Let me try another approach.

<omitted>

Hmm, these aren't equal. Wait, that's a problem. Wait, but according to the vector addition, D should be (8,9). So, why are vectors AD and BC not equal? Wait, maybe I need to check the other pair of sides. Vector AD and vector BC are not equal, but vector AB and vector DC are equal.

<omitted>

These aren't equal, but in a parallelogram, both pairs should be equal. So, that's a problem. Wait, but according to the midpoint method, it's a parallelogram. So, maybe I'm missing something.

No answer!

Token number: 16384

图 14: 在 MATH-500 上, DEER 与 Vanilla 生成内容的对比。

Question: Point A lies somewhere within or on the square which has opposite corners at $(0,0)$ and $(2,2)$. Point B lies somewhere within or on the square which has opposite corners at points $(4,2)$ and $(5,3)$. What is the greatest possible value of the slope of the line containing points A and B ? Express your answer as a common fraction.

DEER

Okay, so I have this problem where I need to find the greatest possible slope of a line connecting two points, A and B. Point A is somewhere inside or on a square with opposite corners at $(0,0)$ and $(2,2)$. Point B is inside or on another square with opposite corners at $(4,2)$ and $(5,3)$. I need to figure out the maximum slope possible between any two points A and B in these squares.

First, let me visualize the squares. The first square has corners at $(0,0)$ and $(2,2)$. Similarly, the second square has opposite corners at $(4,2)$ and $(5,3)$. Hmm, so that square is a bit smaller. Let me figure out its sides. The distance between $(4,2)$ and $(5,3)$ is $\sqrt{(5-4)^2 + (3-2)^2} = \sqrt{1+1} = \sqrt{2}$. So, it's a square with side length $\sqrt{2}$, rotated 45 degrees relative to the axes.

Wait, actually, no. If the opposite corners are $(4,2)$ and $(5,3)$, then the sides are not aligned with the axes.

But wait, is that achievable? Because point A is in the first square, so if A is at $(2,0)$, which is a corner, and point B is at $(4,3)$, which is a corner of the second square. So, connecting $(2,0)$ to $(4,3)$ would give a slope of $(3-0)/(4-2) = 3/2$. But is that the maximum?

To find the greatest possible slope of the line containing points A and B , we need to maximize the expression $\frac{y_B - y_A}{x_B - x_A}$.

Point A lies within or on the square with opposite corners at $(0,0)$ and $(2,2)$. The minimum x -coordinate for A is 0 and the maximum x -coordinate is 2 . Point B lies within or on the square with opposite corners at $(4,2)$ and $(5,3)$. The maximum y -coordinate for B is 3 and the minimum x -coordinate is 4 .

To maximize the slope, we should choose A at $(2,0)$ and B at $(4,3)$. The slope is then $\frac{3-0}{4-2} = \frac{3}{2}$.

However, upon closer inspection, the square for B is actually a diamond with corners at $(4,2)$, $(5,3)$, $(5,2)$, and $(4,3)$. The point $(4,3)$ is indeed a corner of this square. Therefore, the maximum slope is achievable.

The greatest possible slope is $\frac{3}{2}$.

Token number: ~1000

Vanilla CoT

Okay, so I have this problem where I need to find the greatest possible slope of a line connecting two points, A and B. Point A is somewhere inside or on a square with opposite corners at $(0,0)$ and $(2,2)$. Point B is inside or on another square with opposite corners at $(4,2)$ and $(5,3)$. I need to figure out the maximum slope possible between any two points A and B in these squares.

First, let me visualize the squares. The first square has corners at $(0,0)$ and $(2,2)$. Similarly, the second square has opposite corners at $(4,2)$ and $(5,3)$. Hmm, so that square is a bit smaller. Let me figure out its sides. The distance between $(4,2)$ and $(5,3)$ is $\sqrt{(5-4)^2 + (3-2)^2} = \sqrt{1+1} = \sqrt{2}$. So, it's a square with side length $\sqrt{2}$, rotated 45 degrees relative to the axes.

Wait, actually, no. If the opposite corners are $(4,2)$ and $(5,3)$, then the sides are not aligned with the axes.

But wait, is that achievable? Because point A is in the first square, so if A is at $(2,0)$, which is a corner, and point B is at $(4,3)$, which is a corner of the second square. So, connecting $(2,0)$ to $(4,3)$ would give a slope of $(3-0)/(4-2) = 3/2$. But is that the maximum?

Wait, but maybe we can get a steeper slope by choosing points that are not at the corners. Because if we can have a larger $y_B - y_A$ with a smaller $x_B - x_A$, that would give a steeper slope.

But wait, hold on. Let me think again. Maybe if we don't take A at $(2,0)$ and B at $(4,3)$, but somewhere else, we can get a steeper slope.

But wait, is there a way to have a larger slope? For example, if A is at $(2,2)$ and B is at $(4,3)$. Then, the slope is $(3-2)/(4-2) = 1/2$, which is less than $3/2$.

Alternatively, if A is at $(0,0)$ and B is at $(5,3)$, then the slope is $(3-0)/(5-0) = 3/5$, which is less than $3/2$.

Alternatively, if point A is at (x, y_A) where $x < 2$, then point B would have to be at $(x+2, y_B)$. But since the second square starts at $x=4$, $x+2$ must be at least 4, so x must be at least 2.

No answer !

Token number: 16384

图 15: 在 MATH-500 上, DEER 与 Vanilla 生成内容的对比。

The expression $1 + 2 + 3 - 4 + 5 + 6$ is equivalent to 13. If we add parentheses in various locations, we can change this value. For example, $1+2+3-(4+5)+6=1+2+3-9+6=6-9+6=3$. If we only add parentheses (and not rearrange the order of the terms), what is the minimum possible value this expression can obtain? (For the purposes of this problem, we cannot change addition or subtraction into multiplication. For example, $(1+2+3)(-4)+5+6$ is not a valid placement of parentheses.)
Enter your answer as a single value."

Answer:

DEER

Okay, so I have this math problem here: the expression is $1 + 2 + 3 - 4 + 5 + 6$, and it's equal to 13. The task is to add parentheses in various places to change the value, but I can't rearrange the terms or turn the operations into multiplication. I need to find the minimum possible value this expression can obtain by adding parentheses. Hmm, interesting.
First, let me make sure I understand the problem correctly. I can only add parentheses, which will change the order in which operations are performed, but I can't move the numbers around or change the plus and minus signs into multiplication or division. So, for example, I can do something like $(1 + 2) + 3 - 4 + 5 + 6$, but I can't do $1*(2+3) - 4 + 5 + 6$ because that would involve multiplication, which isn't allowed.
<omitted>
Yep, that's correct. So, 13 is the original value.
Alternatively, group $(1 + 2) + (3 - 4 + 5 + 6)$. So, $1 + 2 = 3$, $3 - 4 = -1$, $-1 + 5 = 4$, $4 + 6 = 10$. Then, $3 + 10 = 13$.
Alternatively, group $1 + (2 + 3 - 4 + 5) + 6$. So, $2 + 3 = 5$, $5 - 4 = 1$, $1 + 5 = 6$. Then, $1 + 6 + 6 = 13$.
Hmm, seems like 3 is the lowest I can get so far. But let me think if there's another way.
What if I group $1 + 2 + 3 - (4 + 5 + 6)$. Let's compute that: $4 + 5 + 6 = 15$. Then, $1 + 2 + 3 = 6$, $6 - 15 = -9$. So, that's -9. Wait, that's way lower than 3. Is that allowed?

</think>

To find the minimum possible value of the expression $1 + 2 + 3 - 4 + 5 + 6$ by adding parentheses, we can strategically group terms to maximize the impact of the subtraction.
By grouping the terms after the subtraction, we can create a larger number to subtract, significantly reducing the overall value. For example, grouping $(4 + 5 + 6)$ together:
 $1 + 2 + 3 - (4 + 5 + 6) = 6 - 15 = -9$.
Thus, the minimum possible value is $\boxed{-9}$.

Token number: ~1500

Vanilla CoT

Okay, so I have this math problem here: the expression is $1 + 2 + 3 - 4 + 5 + 6$, and it's equal to 13. The task is to add parentheses in various places to change the value, but I can't rearrange the terms or turn the operations into multiplication. I need to find the minimum possible value this expression can obtain by adding parentheses. Hmm, interesting.
First, let me make sure I understand the problem correctly. I can only add parentheses, which will change the order in which operations are performed, but I can't move the numbers around or change the plus and minus signs into multiplication or division. So, for example, I can do something like $(1 + 2) + 3 - 4 + 5 + 6$, but I can't do $1*(2+3) - 4 + 5 + 6$ because that would involve multiplication, which isn't allowed.
<omitted>
Yep, that's correct. So, 13 is the original value.
Alternatively, group $(1 + 2) + (3 - 4 + 5 + 6)$. So, $1 + 2 = 3$, $3 - 4 = -1$, $-1 + 5 = 4$, $4 + 6 = 10$. Then, $3 + 10 = 13$.
Alternatively, group $1 + (2 + 3 - 4 + 5) + 6$. So, $2 + 3 = 5$, $5 - 4 = 1$, $1 + 5 = 6$. Then, $1 + 6 + 6 = 13$.
Hmm, seems like 3 is the lowest I can get so far. But let me think if there's another way.
What if I group $1 + 2 + 3 - (4 + 5 + 6)$. Let's compute that: $4 + 5 + 6 = 15$. Then, $1 + 2 + 3 = 6$, $6 - 15 = -9$. So, that's -9. Wait, that's way lower than 3. Is that allowed?

Wait, hold on. The original expression is $1 + 2 + 3 - 4 + 5 + 6$. If I group $4 + 5 + 6$, that would be subtracting 15, so $6 - 15 = -9$.
<omitted>
Wait, but in the original expression, after the subtraction, it's $+5 + 6$. So, if I group $4 + 5 + 6$, that would be changing the operations after the subtraction, but I can't rearrange terms. So, is that allowed?
<omitted>
Alternatively, group $1 + (2 + 3 - 4 + 5 + 6)$. As before, $1 + 12 = 13$.
Alternatively, group $(1 + 2 + 3 - 4 + 5) + 6$...

Endless exhaustive search!

Token number: 16384

图 16: 在 MATH-500 上, DEER 与 Vanilla 生成内容的对比。

Question: The coordinates of a parallelogram are (5, 3), (6, 8), (7, 4) and \$(x, y)\$ and \$x > 7\$. What is the value of \$x + y\$?

Answer: Okay, so I have this problem where I need to find the coordinates of the fourth vertex of a parallelogram. The given points are (5, 3), (6, 8), (7, 4), and (x, y) with the condition that x > 7. Then, I need to find the value of x + y. Hmm, let me think about how to approach this. First, I remember that in a parallelogram, opposite sides are equal and parallel. That means the midpoints of the diagonals should be the same. So, if I can find the midpoint of one diagonal, it should be equal to the midpoint of the other diagonal. Let me label the points to make it clearer. Let's say point A is (5, 3), point B is (6, 8), point C is (7, 4), and point D is (x, y). Now, I need to figure out which points are connected to each other. Since it's a parallelogram, the order of the points matters. I think the points should be connected in order, so A to B to C to D and back to A. But I'm not entirely sure, so maybe I should consider different possibilities. Alternatively, another way to approach this is by using vectors. In a parallelogram, the vector from one point to another should be equal to the vector from another point to the next. So, for example, vector AB should be equal to vector DC, and vector AD should be equal to vector BC. Let me try that. First, let's find vector AB. Point A is (5, 3) and point B is (6, 8). So, vector AB is (6 - 5, 8 - 3) = (1, 5). If vector AB is equal to vector DC, then vector DC should also be (1, 5). Point D is (x, y) and point C is (7, 4). So, vector DC is (7 - x, 4 - y). Setting this equal to (1, 5), we get: 7 - x = 1 and 4 - y = 5. Solving the first equation: 7 - x = 1 => x = 7 - 1 = 6. Solving the second equation: 4 - y = 5 => y = 4 - 5 = -1. <Induced trial answer>: boxed[6 + (-1)]. Confidence: 0.7344 Continue thinking Wait, but the problem states that x > 7, and here x is 6, which is less than 7. That can't be right. So, maybe I assigned the points incorrectly. Perhaps the order of the points isn't A, B, C, D. Maybe it's a different order. Let me try another approach. Another property of a parallelogram is that the diagonals bisect each other. So, the midpoint of diagonal AC should be the same as the midpoint of diagonal BD. Let me calculate the midpoint of AC first. Point A is (5, 3) and point C is (7, 4). The midpoint formula is ((x1 + x2)/2, (y1 + y2)/2). So, midpoint of AC is ((5 + 7)/2, (3 + 4)/2) = (12/2, 7/2) = (6, 3.5). Now, the midpoint of BD should also be (6, 3.5). Point B is (6, 8) and point D is (x, y). So, midpoint of BD is ((6 + x)/2, (8 + y)/2). Setting this equal to (6, 3.5): (6 + x)/2 = 6 and (8 + y)/2 = 3.5. Solving the first equation: (6 + x)/2 = 6 => 6 + x = 12 => x = 6. Solving the second equation: (8 + y)/2 = 3.5 => 8 + y = 7 => y = -1. Again, I get x = 6 and y = -1, but x is supposed to be greater than 7. Hmm, that's conflicting. Maybe I need to consider a different pair of points as the diagonal. <Induced trial answer>: boxed[10]. Confidence: 0.5312 Continue thinking Wait, perhaps I assumed the wrong diagonal. Maybe the diagonals are AD and BC instead of AC and BD. Let me try that. Midpoint of AD: Point A is (5, 3) and point D is (x, y). Midpoint is ((5 + x)/2, (3 + y)/2). Midpoint of BC: Point B is (6, 8) and point C is (7, 4). Midpoint is ((6 + 7)/2, (8 + 4)/2) = (13/2, 12/2) = (6.5, 6). Setting midpoints equal: (5 + x)/2 = 6.5 and (3 + y)/2 = 6. Solving the first equation: (5 + x)/2 = 6.5 => 5 + x = 13 => x = 8. Solving the second equation: (3 + y)/2 = 6 => 3 + y = 12 => y = 9. Okay, this gives x = 8 and y = 9, which satisfies x > 7. So, the fourth point is (8, 9), and x + y = 8 + 9 = 17. <Induced trial answer>: boxed[17]. Confidence: 0.9993 Exit! <think> To find the coordinates of the fourth vertex of the parallelogram, we use the property that the diagonals of a parallelogram bisect each other. 1. **Calculate the midpoint of diagonal BC:** Points B (6, 8) and C (7, 4). Midpoint formula: $(\frac{6+7}{2}, \frac{8+4}{2}) = (\frac{13}{2}, 6)$. 2. **Set the midpoint of diagonal AD equal to the midpoint of BC:** Let D be (x, y). Midpoint of AD: $(\frac{5+x}{2}, \frac{3+y}{2})$. Equate to (6.5, 6): $(\frac{5+x}{2}, \frac{3+y}{2}) = (6.5, 6) \Rightarrow 5+x=13 \Rightarrow x=8$ and $3+y=12 \Rightarrow y=9$. 3. **Calculate $x+y$:** $8+9=17$. 4. **Answer:** boxed[17]

图 17: 一个展示 LRM 感知珍珠推理涌现的隐含能力的示例。当模型的推理内容不足以支持引出的答案时，置信度保持在较低水平，如图中前两个置信度值所示。相反，当模型提供了充分且合理的推理作为依据时，置信度达到高水平，可能接近 1，如图中最后一个置信度值所示。

46

Question: Define $p = \sum_{k=1}^{\infty} \frac{1}{k^2} \quad \text{and} \quad q = \sum_{k=1}^{\infty} \frac{1}{k^3}.$ Find a way to write $\sum_{k=1}^{\infty} \frac{1}{(j+k)^3}$ in terms of p and q.	
Answer:	408
DeepSeek-R1-Distill-Qwen-32B Okay, so I have this problem where I need to express the double sum $\sum_{j=1}^{\infty} \sum_{k=1}^{\infty} \frac{1}{(j+k)^3}$ in terms of $\sum_{k=1}^{\infty} \frac{1}{k^2}$ and $\sum_{k=1}^{\infty} \frac{1}{k^3}$. Hmm, let me think about how to approach this. First, I know that $\sum_{k=1}^{\infty} \frac{1}{k^2}$ is the famous Basel problem, which equals $\frac{\pi^2}{6}$, and $\sum_{k=1}^{\infty} \frac{1}{k^3}$ is the Riemann zeta function at 3, which is known as Apéry's constant, approximately 1.202... But I don't think I need the exact values; instead, I need to relate the double sum to these series. Looking at the double sum, it's over all positive integers j and k, and the term is $\frac{1}{(j+k)^3}$. Maybe I can change variables to simplify this. Let me set $n = j + k$. Then, for each fixed n, j and k can range from 1 to $n-1$, and $k = n - j$. So, perhaps I can rewrite the double sum as a single sum over n, where each term is the sum over j from 1 to $n-1$ of $\frac{1}{(j+n-j)^3}$. Let me write that down: $\sum_{j=1}^{\infty} \sum_{k=1}^{\infty} \frac{1}{(j+k)^3} = \sum_{n=2}^{\infty} \sum_{j=1}^{n-1} \frac{1}{n^3}.$ Now, the double sum becomes $\sum_{n=2}^{\infty} \frac{1}{n^3} \sum_{j=1}^{n-1} 1$. The inner sum is just adding $\frac{1}{n^3}$ a total of $(n-1)$ times. So, the inner sum simplifies to $\frac{1}{n^3} \cdot (n-1)$. So now, the double sum becomes $\sum_{n=2}^{\infty} \frac{n-1}{n^3} = \sum_{n=2}^{\infty} \left(\frac{n}{n^3} - \frac{1}{n^3} \right) = \sum_{n=2}^{\infty} \left(\frac{1}{n^2} - \frac{1}{n^3} \right)$. So, this is equal to $\sum_{n=2}^{\infty} \frac{1}{n^2} - \sum_{n=2}^{\infty} \frac{1}{n^3}$. But I know that $\sum_{n=1}^{\infty} \frac{1}{n^2} = p$ and $\sum_{n=1}^{\infty} \frac{1}{n^3} = q$. So, if I subtract the $n=1$ terms from both series, I can express the sums starting from $n=2$. Let's compute that: $\sum_{n=2}^{\infty} \frac{1}{n^2} = p - \frac{1}{1^2} = p - 1$ $\sum_{n=2}^{\infty} \frac{1}{n^3} = q - \frac{1}{1^3} = q - 1$ So, substituting back into our expression: $\sum_{j=1}^{\infty} \sum_{k=1}^{\infty} \frac{1}{(j+k)^3} = (p - 1) - (q - 1) = p - q.$ So, the double sum simplifies to $p - q$. Therefore, the answer is $p - q$.	15
QwQ-32B Okay, so I need to find a way to express the double sum $\sum_{j=1}^{\infty} \sum_{k=1}^{\infty} \frac{1}{(j+k)^3}$ in terms of $\sum_{k=1}^{\infty} \frac{1}{k^2}$ and $\sum_{k=1}^{\infty} \frac{1}{k^3}$. Hmm, let me think about how to approach this. First, I remember that double sums can sometimes be simplified by changing variables or switching the order of summation. Maybe I can let $n = j + k$, so that for each n, the terms where $j + k = n$ contribute to the sum. That might help me rewrite the double sum as a single sum over n, with each term multiplied by the number of pairs (j, k) that add up to n. Let me try that. Let's set $n = j + k$. Then for each $n \geq 2$, since j and k are at least 1, the number of pairs (j, k) such that $j + k = n$ is $(n-1)$. For example, when $n=2$, there's only one pair (1,1); when $n=3$, there are two pairs (1,2) and (2,1); and so on. So the double sum can be rewritten as $\sum_{n=2}^{\infty} \frac{(n-1)}{n^3}.$ Wait, is that right? Let me check. Each term $\frac{1}{(j+k)^3}$ for each pair (j, k) would be grouped by $n = j + k$, so each n contributes $(n-1)$ terms each of $\frac{1}{n^3}$. Therefore, the total contribution for each n is $\frac{(n-1)}{n^3}$. So the entire double sum becomes $\sum_{n=2}^{\infty} \frac{(n-1)}{n^3}$. Now I need to express this in terms of p and q. Let me see. Let's first write out the sum: $\sum_{n=2}^{\infty} \frac{(n-1)}{n^3} = \sum_{n=2}^{\infty} \left(\frac{n}{n^3} - \frac{1}{n^3} \right) = \sum_{n=2}^{\infty} \left(\frac{1}{n^2} - \frac{1}{n^3} \right).$ Ah, that simplifies to the difference between two sums. Let's split that into two separate sums: $\sum_{n=2}^{\infty} \frac{1}{n^2} - \sum_{n=2}^{\infty} \frac{1}{n^3}.$ Now, notice that these are just the tails of the series for p and q. Since $p = \sum_{k=1}^{\infty} \frac{1}{k^2}$, then $\sum_{n=2}^{\infty} \frac{1}{n^2} = p - \frac{1}{1^2} = p - 1$. Similarly, $q = \sum_{k=1}^{\infty} \frac{1}{k^3}$, so $\sum_{n=2}^{\infty} \frac{1}{n^3} = q - \frac{1}{1^3} = q - 1$. Therefore, substituting back into the expression: $\sum_{j=1}^{\infty} \sum_{k=1}^{\infty} \frac{1}{(j+k)^3} = (p - 1) - (q - 1) = p - q.$ So, the double sum simplifies to $p - q$.	387
Let's summarize the above thought process and provide the result. The double sum can be rewritten by considering the sum over $n = j + k$. For each $n \geq 2$, there are $(n-1)$ pairs (j, k) such that $j + k = n$. This transforms the double sum into a single sum: $\sum_{n=2}^{\infty} \frac{(n-1)}{n^3}.$ Breaking this into two separate sums: $\sum_{n=2}^{\infty} \frac{1}{n^2} - \sum_{n=2}^{\infty} \frac{1}{n^3}.$ Recognizing these as the tails of the series for p and q, we subtract the first terms: $\sum_{n=2}^{\infty} \frac{1}{n^2} = p - 1$ $\sum_{n=2}^{\infty} \frac{1}{n^3} = q - 1$ Thus, the double sum simplifies to $p - q$.	260
To express the double sum $\sum_{j=1}^{\infty} \sum_{k=1}^{\infty} \frac{1}{(j+k)^3}$ in terms of $\sum_{k=1}^{\infty} \frac{1}{k^2}$ and $\sum_{k=1}^{\infty} \frac{1}{k^3}$, we start by changing variables. Let $n = j + k$. For each $n \geq 2$, the number of pairs (j, k) such that $j + k = n$ is $(n-1)$. This allows us to rewrite the double sum as $\sum_{n=2}^{\infty} \frac{(n-1)}{n^3}.$ Recognizing these as the tails of the series for p and q, we have: $\sum_{n=2}^{\infty} \frac{1}{n^2} = p - 1$ $\sum_{n=2}^{\infty} \frac{1}{n^3} = q - 1$ Substituting these into the expression gives: $\sum_{j=1}^{\infty} \sum_{k=1}^{\infty} \frac{1}{(j+k)^3} = (p - 1) - (q - 1) = p - q.$	128

图 18: 一个展示 QwQ-32B 中存在多个 $\langle \text{think} \rangle$ 标记的示例。对于两个模型，绿色思考块都是以得出正确结论。然而，QwQ-32B 继续进行了一个额外的总结（红色块）并生成了自己的 $\langle \text{think} \rangle$ 标记。基于以上内容，它得出了结论。